

Aurix

Microcontroller Infineon Software Architecture

User's Manual

SPI Handler Driver

Release v4.5

2016-08

Edition 2016-08

**Published by
Infineon Technologies AG
81726 Munich, Germany**

**© 2016 Infineon Technologies AG
All Rights Reserved.**

LEGAL DISCLAIMER

THE INFORMATION GIVEN IN THIS DOCUMENT IS GIVEN AS A HINT FOR THE IMPLEMENTATION OF THE INFINEON TECHNOLOGIES COMPONENT ONLY AND SHALL NOT BE REGARDED AS ANY DESCRIPTION OR WARRANTY OF A CERTAIN FUNCTIONALITY, CONDITION OR QUALITY OF THE INFINEON TECHNOLOGIES COMPONENT. THE RECIPIENT OF THIS DOCUMENT MUST VERIFY ANY FUNCTION DESCRIBED HEREIN IN THE REAL APPLICATION. INFINEON TECHNOLOGIES HEREBY DISCLAIMS ANY AND ALL WARRANTIES AND LIABILITIES OF ANY KIND (INCLUDING WITHOUT LIMITATION WARRANTIES OF NON-INFRINGEMENT OF INTELLECTUAL PROPERTY RIGHTS OF ANY THIRD PARTY) WITH RESPECT TO ANY AND ALL INFORMATION GIVEN IN THIS DOCUMENT.

Information

For further information on technology, delivery terms and conditions and prices, please contact the nearest Infineon Technologies Office (www.infineon.com).

Warnings

Due to technical requirements, components may contain dangerous substances. For information on the types in question, please contact the nearest Infineon Technologies Office.

Infineon Technologies components may be used in life-support devices or systems only with the express written approval of Infineon Technologies, if a failure of such components can reasonably be expected to cause the failure of that life-support device or system or to affect the safety or effectiveness of that device or system. Life support devices or systems are intended to be implanted in the human body or to support and/or maintain and sustain and/or protect human life. If they fail, it is reasonable to assume that the health of the user or other persons may be endangered.

Date	Version	Change Description
2016-08-10	4.5	Added point 10 in Section 9.2 based on SAS_MCAL_SPI_9863 Added safety marker related information in Sections 4.6.8, 6.2.1, 6.2.4, 6.2.6 and 6.2.7 as per JIRA 51261-288. Updated to fix review comments: Added JIRA ID in version history Updated Sections as part of Protected mode support implementation: Section 2.2 : Interface diagram updated Section 3.1 File structure updated Sections 4.2.19 to 4.2.22: SpiGeneral Configuration parameters added Sections 6.2 to 6.4 IO mode row of APIs updated and origin added Section 8.1.1 : Example usage updated
2016-06-22	4.4	Updated Section 9.2 : Updated point 1 in Functional limitation as part of JIRA 0000051261-226
2015-11-29	4.3	Updated Section 9.1 : Updated Interrupt priority information
2015-11-03	4.2	1. Updated Section 4.7.6, wrt behaviour CS_VIA_GPIO with SpiChannelBasedCS (JIRA 0000051108-170). 2. Updated Section 7.1, Added Safety Error SPI_E_INTR_SRC_ERROR Interrupt Source Check failure, also updated Section 6.4, Service Id for ISRs added. 3. Updated API's table, wrt I/O mode, also Interface diagram in Section 2.2 (Usermode/Supervisor mode)
2015-09-09	4.1	Added Safety Error, SPI_E_CLC_ERROR (updated, Table 159 and 161) Updated Debug Macro used for Testing (Section 7.3)
2015-07-22	4.0	Updated review comments, wrt Section 6.4
2015-07-07	3.9	Updated table description in Section 10.2 (JIRA 0000051261-140) Updated Section 6.4, with interrupt priority recommendation (JIRA 0000051124-142)
2015-05-28	3.8	Updated review comments
2015-05-26	3.7	Updated Section 9.2, wrt multiple configuration set and delays Updated section, 6.2.5, added Table 130 (JIRA 0000051047-130)
2015-05-14	3.6	Updated review comments
2015-05-13	3.5	Updated section 9.1 (JIRA 0000051047-86)
2015-04-29	3.4	Review comments updated
2015-04-27	3.3	Updated, section 4.2.3 (JIRA 0000051124-60) section 4.6.4, 4.6.5 (JIRA 0000051124-78, 0000051023-33) section 6.4, general notes updated (JIRA 0000051124-80) Added, section 4.2.17, 4.2.18 (JIRA 0000051124-52) section 8.1.5, Spi slave usage
2015-03-05	3.2	Updated for review comments REV_007050
2015-02-12	3.1	-- Section 10 modified for Tc21x derivatives -- Updated for JIRA 0000051050-4
2015-01-09	3.0	Updated for review comments REV_006746

CONFIDENTIAL

Date	Version	Change Description
2015-01-07	2.4	SpiParitySupport added for : AI00252435
2014-11-10	2.3	Updated for review comments REV_006430
2014-11-05	2.2	AI00251090: IO mode added for all API's AI00249920: SpiAssignedHwChannel description updated. AI00247851: Section 4.3 added.
2014-09-05	2.1	-- Section 10.2 updated for TC22x
2014-08-01	2.0	-- Updated review comments REV_005894
2014-07-17	1.19	-- Updated for UTP AI00239337 to support SpiAsyncParallelTransmit.
2014-07-17	1.18	-- Updated Configuration Limitation for UTP AI00246367
2014-07-14	1.17	-- Removed Safety information chapter as per review comments REV_005550
2014-05-26	1.16	-- Updated Section 10.1 for UTP AI00239711 -- Updated for UTP AI00243434 and UTP Task AI00243494 -- Removed Author column in Document history table. -- Debug macros effect explained in section 7.3
2014-05-02	1.15	-- Removed slave limitation for SpiChannelBasedCS -- Traceability updated.
2014-04-01	1.14	-- Corrected Section 11.2.12
2014-03-25	1.13	-- Updated Traceability to reduce Manual coverage. -- Added Tc267 derivate in Chapter 11. -- SpiChannelBasedCS moved to Job container and limitations updated. -- SoftwareSafetyInformation reference updated.
2014-02-24	1.12	-- Section 11. Tables updated for more Derivates. -- AI00237720: Channel based chip select implemented.
2014-02-05	1.11	Updated review comments REV_005166
2014-01-14	1.10	-- Slave implemented and limitation removed.
2013-12-12	1.9	-- Updated for AI00233215 and AI00237741
2013-11-12	1.8	-- Updated for review comments REV_005010
2013-10-31	1.7	-- Updated for Safety support.
2013-08-06	1.6	-- Made as single source UM for both autosar versions. -- Section 7.2 introduced for UTP AI00193369. -- Figures for "Interfaces of SPI Handler Driver" and "File Structure" updated. -- Section 10 updated with 29x, 27x, and 26x pins. -- Safety containers and parameters mentioned.
2013-07-08	1.5	--Changed elements in SpiHwDmaConfiguration as reference to DMA channels from Mcu. --Section 10 updated for Tc26x
2013-03-12	1.4	Updated for TC27x B-step
2012-11-11	1.3	Updated for AS403, UTP AI00081891
2012-07-30	1.2	Updated review comments
2012-05-14	1.1	Updated for TC27x
2011-10-21	1.0	Updated review comments.
2011-09-14	0.2	Document updated for TC2Dx .
2011-03-11	0.1	Initial draft version

We Listen to Your Comments

Is there any information in this document that you feel is wrong, unclear or missing?

Your feedback will help us to continuously improve the quality of this document.

Please send your proposal (including a reference to this document) to:

mcdocu.comments@infineon.com



Table of Contents	Page
1 Introduction	17
1.1 Scope	17
1.2 Abbreviations.....	17
1.3 Reference	18
2 SPI Handler Driver Overview.....	19
2.1 Software Hardware Mapping.....	19
2.1.1 QSPIn.....	20
2.1.2 PORT	21
2.1.3 System Clock	21
2.1.4 DMA	21
2.1.5 IRQ	21
2.2 Software Driver Description	21
3 File Structure	24
3.1 Module File Structure	24
4 Configuration Documentation	26
4.1 Configuration Concept	26
4.1.1 Configuration Class	26
4.1.2 Configuration Variant	26
4.1.3 How to read the Configuration Class field.....	26
4.2 Container: SpiGeneral.....	27
4.2.1 SpiDevErrorDetect	27
4.2.2 SpiVersionInfoApi.....	27
4.2.3 SpiLevelDelivered	28
4.2.4 SpiChannelBuffersAllowed.....	29
4.2.5 SpiInterruptibleSeqAllowed	29
4.2.6 SpiHwStatusApi	30
4.2.7 SpiCancelApi.....	30
4.2.8 SpiMaxJobTriggerQueueLength	30
4.2.9 SpiPBFixedAddress	31
4.2.10 SpiDebugSupport	31
4.2.11 SpiSupportConcurrentSyncTransmit	32
4.2.12 SpiSyncTransmitTimeoutDuration	32
4.2.13 SpiSlaveSelectRef	33
4.2.14 SpiSlaveFlipTransferStart	33
4.2.15 SpiSafetyMasterRef	34
4.2.16 SpiAsyncParallelTransmit	34
4.2.17 SpiBaudrateConfigAtRuntime	35
4.2.18 SpiResetSfrAtInit.....	35
4.2.19 SpiUserModelInitApiEnable	36
4.2.20 SpiUserModeDeInitApiEnable.....	36
4.2.21 SpiUserModeRuntimeApiEnable.....	37
4.2.22 SpiRunningInUser0Mode	37
4.3 Container: SpiMainFunctionPeriod	37
4.3.1 SpiMainFunctionPeriod	38
4.4 Container: SpiSafety	38
4.4.1 SpiSafetyEnable	38
4.4.2 SpiInitCheckApi.....	38
4.5 Container: SpiDriver	39
4.5.1 SpiMaxChannel.....	39
4.5.2 SpiMaxJob.....	39
4.5.3 SpiMaxSequence	40
4.5.4 SpiSystemclock.....	40
4.6 Container: SpiChannel.....	40
4.6.1 SpiChannelType.....	40
4.6.2 SpiDataWidth	41
4.6.3 SpiTransferStart	41

Table of Contents	Page
4.6.4 SpiNbBuffers	42
4.6.5 SpiEbMaxLength	43
4.6.6 SpiDefaultData	43
4.6.7 SpiChannelId	44
4.6.8 SpiSafetyBufferMarker	44
4.7 Container: SpiJob	44
4.7.1 SpiJobPriority	45
4.7.2 SpiJobEndNotification	45
4.7.3 SpiDeviceAssignment	45
4.7.4 SpiJobId	46
4.7.5 SpiHwUnitSynchronous	46
4.7.6 SpiChannelBasedCS	47
4.8 Container: SpiChannelList	47
4.8.1 SpiChannelIndex	47
4.8.2 SpiChannelAssignment	48
4.9 Container: SpiExternalDevice	48
4.9.1 SpiCsIdentifier	48
4.9.2 SpiEnableCs	49
4.9.3 SpiCsPolarity	49
4.9.4 SpiCsSelection	49
4.9.5 SpiBaudrate	50
4.9.6 SpiTimeClk2CS	50
4.9.7 SpiShiftClockIdleLevel	51
4.9.8 SpiDataShiftEdge	51
4.9.9 SpiIdleTime	52
4.9.10 SpiTrailingTime	52
4.9.11 SpiAutoCalcDelayParams	52
4.9.12 SpiAutoCalcBaudParams	53
4.9.13 SpiQSpiParamTq	53
4.9.14 SpiQSpiParamQ	54
4.9.15 SpiQSpiParamA	54
4.9.16 SpiQSpiParamB	55
4.9.17 SpiQSpiParamC	55
4.9.18 SpiQSpiParamTPre	56
4.9.19 SpiQSpiParamTrail	56
4.9.20 SpiQSpiParamIPre	57
4.9.21 SpiQSpiParamIdle	57
4.9.22 SpiQSpiParamLPre	58
4.9.23 SpiQSpiParamLead	58
4.9.24 SpiParitySupport	59
4.10 Container: SpiHwUnit	59
4.10.1 SpiAssignedHwModule	59
4.10.2 SpiAssignedHwChannel	60
4.11 Container: SpiCsGpio	60
4.11.1 SpiCsGpioPortSelection	60
4.11.2 SpiCsGpioPinSelection	61
4.12 Container: SpiSequence	61
4.12.1 SpiInterruptibleSequence	61
4.12.2 SpiSeqEndNotification	62
4.12.3 SpiSequenceld	62
4.12.4 SpiJobAssignment	63
4.13 Container: SpiHwConfiguration	64
4.13.1 Container: SpiHardwareConfigurationQspix	64
4.13.1.1 Container: SpiClockConfiguration	64
4.13.1.2 Container: SpiHwPinConfiguration	65
4.13.1.3 Container: SpiHwDmaConfiguration	66
4.14 Container: SpiDemEventParameterRefs	67
4.14.1 SPI_E_HARDWARE_ERROR	67
SPI_E_HARDWARE_ERROR	67

Table of Contents	Page
4.14.2 SPI_E_DMA_TRANSFER.....	67
4.15 Container: SpiPublishedInformation	68
4.15.1 SpiMaxHwUnit.....	68
4.16 Container: CommonPublishedInformation	68
4.16.1 ArMajorVersion.....	68
4.16.2 ArMinorVersion.....	69
4.16.3 ArPatchVersion	69
4.16.4 SwMajorVersion	69
4.16.5 SwMinorVersion	70
4.16.6 SwPatchVersion	70
4.16.7 VendorId.....	70
4.16.8 ModuleId.....	71
4.16.9 Release	71
4.17 Derived Configuration Parameters.....	72
4.17.1 SPI_MAXIMUM_SEQUENCE	72
4.17.2 SPI_MAXIMUM_JOB	72
4.17.3 SPI_MAXIMUM_CHANNEL.....	73
4.17.4 SPI_MAXIMUM_IB_BUFFER_SIZE	73
4.17.5 SPI_MAXIMUM_EB_CHANNELS.....	73
4.17.6 SPI_MAXIMUM_IB_CHANNELS	74
4.17.7 SPI_WRITELOCK_INDEX	74
4.17.8 SPI_SEQUENCE_ASIL_ARRAY_INDEX	75
4.17.9 SPI_SEQUENCE_QM_ARRAY_INDEX	75
4.17.10 SPI_JOB_ASIL_ARRAY_INDEX	76
4.17.11 SPI_JOB_QM_ARRAY_INDEX	76
4.17.12 SPI_JOB_END_NOTIF_REQ	77
4.17.13 SPI_SEQ_END_NOTIF_REQ.....	77
4.17.14 SPI_QSPiz_USED	77
4.17.15 SPI_QSPiz_MRIS_SEL	78
4.17.16 SPI_QSPiz_HWTYPE.....	78
4.17.17 SPI_MAX_HW_UNIT	79
4.17.18 SPI_DEBUG_SUPPORT	79
4.17.19 SPI_SUPPORT_CONCURRENT_SYNC_TRANSMIT.....	80
4.17.20 SPI_SYNCTRANSMIT_TIMEOUT_COUNTER.....	80
4.17.21 SPI_MASTER_MODULES_USED.....	81
4.17.22 SPI_ASIL_MASTER_MODULE_USED	81
4.17.23 SPI_NUM_ASIL_MASTER_MODULES	81
4.17.24 SPI_NUM_QM_ASYNC_MASTERS.....	82
4.17.25 SPI_NUM_ASIL_SLAVE_MODULES	82
4.17.26 SPI_SLAVE_MODULE_INDEX	83
4.17.27 SPI_SLAVE_ENABLE.....	83
4.17.28 SPI_CONFIG_COUNT.....	83
5 Published Parameters	85
5.1 VendorId.....	85
5.2 ModuleId.....	85
5.3 ARMajorVersion	85
5.4 ARMinorVersion	85
5.5 ARPatchVersion	86
5.6 SwMajorVersion	86
5.7 SwMinorVersion	86
5.8 SwPatchVersion	86
5.9 VendorApilInfix.....	86
5.10 Release	87
6 API Documentation	88
6.1 API Type Definitions.....	88
6.1.1 Type Definition Spi_ConfigType.....	88
6.1.2 Type Definition Spi_StatusType.....	88

Table of Contents	Page
6.1.3 Type Definition Spi_JobResultType	88
6.1.4 Type Definition Spi_SeqResultType	89
6.1.5 Type Definition Spi_DataType	89
6.1.6 Type Definition Spi_NumberOfDataType	90
6.1.7 Type Definition Spi_ChannelType	90
6.1.8 Type Definition Spi_JobType	90
6.1.9 Type Definition Spi_SequenceType	90
6.1.10 Type Definition Spi_HWUnitType	91
6.1.11 Type Definition Spi_AsyncModeType	91
6.2 API Functions	91
6.2.1 Spi_Init	91
6.2.2 Spi_InitCheck	92
6.2.3 Spi_DeInit	93
6.2.4 Spi_WriteIB	94
6.2.5 Spi_AsyncTransmit	95
6.2.6 Spi_ReadIB	102
6.2.7 Spi_SetupEB	103
6.2.8 Spi_GetStatus	105
6.2.9 Spi_GetJobResult	106
6.2.10 Spi_GetSequenceResult	107
6.2.11 Spi_GetVersionInfo	108
6.2.12 Spi_SyncTransmit	109
6.2.13 Spi_GetHWUnitStatus	110
6.2.14 Spi_Cancel	111
6.2.15 Spi_SetAsyncMode	112
6.2.16 Spi_SlaveSetupEB	113
6.2.17 Spi_SlaveReadPrepare	114
6.3 Scheduled functions	115
6.3.1 Spi_MainFunction_Handling	115
6.4 Interrupt Handling in SPI	116
6.4.1 Spi_IsrDmaQspiRx	117
6.4.2 Spi_IsrDmaQspiTx	118
6.4.3 Spi_IsrQspiError	118
6.4.4 Spi_IsrDmaError	119
6.4.5 Spi_IsrQspiPt	119
6.4.6 Spi_IsrQspiUsr	120
6.4.7 Spi_IsrCheckDmaError	121
6.5 Exclusive Area (EA) Information	121
6.5.1 Spi_Init	121
6.5.2 Spi_DeInit	121
6.5.3 Spi_WriteIB	122
6.5.4 Spi_AsyncTransmit	122
6.5.5 Spi_SyncTransmit (AS4.x)	122
6.5.6 Spi_GetSequenceResult	122
6.5.7 Spi_Cancel	123
7 Error Classification	124
7.1 Error Classification and Error codes	124
7.2 Description of Production errors	125
7.3 Debug Macro used for Testing	126
8 Example Usage	127
8.1 Configuration of the driver	127
8.1.1 Sample Configuration of the Driver	127
8.1.2 SPI Configuration	131
8.1.3 Spi Driver Initialization	131
8.1.4 Start SPI Transmission to access EEPROM	132
8.1.5 Spi Slave Usage	133
9 Limitations and Assumptions	135

Table of Contents	Page
9.1 Assumptions and Deviations from Software Specification.....	135
9.2 Limitations	135
9.3 Hardware Features not supported	136
10 Used Aurix Derivates	137
10.1 QSPI Modules Availability	137
10.2 Port-Pins mapping.....	138
10.2.1 TC299 (516pin) Derivative Port-Pin mapping	138
10.2.2 TC298 (416pin) Derivative Port-Pin mapping	140
10.2.3 TC297 (292pin) Derivative Port-Pin mapping	142
10.2.4 TC277 (292pin) Derivative Port-Pin mapping	144
10.2.5 TC275 (176pin) Derivative Port-Pin mapping	146
10.2.6 TC267 (292pin) Derivative Port-Pin mapping	148
10.2.7 TC265 (176pin) Derivative Port-Pin mapping	150
10.2.8 TC264/TC244 (144pin) Derivative Port-Pin mapping	152
10.2.9 TC237 (292pin) Derivative Port-Pin mapping	154
10.2.10 TC234 (144pin) Derivative Port-Pin mapping	156
10.2.11 TC233 (100pin) Derivative Port-Pin mapping	158
10.2.12 TC224/TC214 (144pin) Derivative Port-Pin mapping	159
10.2.13 TC223/TC213 (100pin) Derivative Port-Pin mapping	161
10.2.14 TC222/TC212 (80pin) Derivative Port-Pin mapping	162

List of Figures**Page**

Figure 1	Software Hardware Mapping of SPI HandlerDriver.	20
Figure 2	Interfaces of SPI Handler Driver	23
Figure 3	File Structure of SPI Handler Driver	24
Figure 4	Sequences, Job, Channel links.....	64
Figure 5	Handling Notification functions.....	98
Figure 6	Use case 1: representing the interruptible sequence	99
Figure 7	Use case 2: representing the interruptible and non-interruptible sequence	100
Figure 8	Use case 3: Difference between SpiAsyncParallelTransmit ON/OFF	101
Figure 9	Spi Slave connections	133

List of Tables

Page

Table 1	Abbreviations used in this Document.....	17
Table 2	File Contents	24
Table 3	Configuration Class Example 1.....	27
Table 4	Configuration Class Example 2.....	27
Table 5	Development Error Tracer Configuration	27
Table 6	Module Version Information API Configuration	27
Table 7	SPI Level Delivered.....	28
Table 8	SPI Channel Buffers Allowed	29
Table 9	SPI Interruptible Sequence Allowed	29
Table 10	SPI HW Status API	30
Table 11	SPI Cancel API.....	30
Table 12	SPI Maximum Job Trigger Queue Length.....	30
Table 13	SPI Post build fixed address feature	31
Table 14	SpiDebugSupport.....	31
Table 15	SpiSupportConcurrentSyncTransmit	32
Table 16	SpiSyncTransmitTimeoutDuration	32
Table 17	SpiSlaveSelectRef Configuration	33
Table 18	SpiSlaveFlipTransferStart Configuration.....	33
Table 19	SpiSafetyMasterRef Configuration.....	34
Table 1	SpiAsyncParallelTransmit Configuration.....	34
Table 2	SpiBaudrateConfigAtRuntime Configuration.....	35
Table 3	SpiResetSfrAtInit Configuration	35
Table 4	SpiUserModelInitApiEnable	36
Table 5	SpiUserModeDeInitApiEnable.....	36
Table 6	SpiUserModeRuntimeApiEnable.....	37
Table 7	SpiRunningInUser0Mode	37
Table 8	SpiMainFunctionPeriod Configuration	38
Table 9	SpiSafetyEnable Configuration	38
Table 10	SpiInitCheckApi Configuration	38
Table 11	SPI Maximum channel	39
Table 12	SPI Maximum jobs	39
Table 13	SPI Maximum sequence	40
Table 14	SpiSystemClock	40
Table 15	SPI Channel Type	40
Table 16	SpiDataWidth	41
Table 17	SPI Transfer Start	41
Table 18	Number of IB Buffers.....	42
Table 19	Number of EB Buffers	43
Table 20	SPI Default Data	43
Table 21	SPI Channel ID.....	44
Table 22	SpiSafetyBufferMarker Configuration	44
Table 23	SPI Job Priority.....	45
Table 24	SPI Job End Notification	45
Table 25	SPI External Device assignment.....	45
Table 26	SPI Job ID	46
Table 27	SpiHwUnitSynchronous	46
Table 28	SpiChannelBasedCS Configuration	47
Table 29	SpiChannelIndex.....	47
Table 30	SPI Channel Linking.....	48
Table 31	SPI CS Identifier.....	48
Table 32	SPI Enable CS	49
Table 33	SPI CS Polarity.....	49
Table 34	SpiCsSelection	49
Table 35	SPI Baudrate	50
Table 36	SPI Time CLK2CS	50
Table 37	Shift Clock Idle Level.....	51
Table 38	Data Shift Edge	51
Table 39	SPI Idle Time.....	52

List of Tables
Page

Table 40	SPI Training Time	52
Table 41	SpiAutoCalcDelayParams	52
Table 42	SpiAutoCalcBaudParams	53
Table 43	SpiQSpiParamTq	53
Table 44	SpiQSpiParamQ	54
Table 45	SpiQSpiParamA	54
Table 46	SpiQSpiParamB	55
Table 47	SpiQSpiParamC	55
Table 48	SpiQSpiParamTPre	56
Table 49	SpiQSpiParamTrail	56
Table 50	SpiQSpiParamIPre	57
Table 51	SpiQSpiParamIdle	57
Table 52	SpiQSpiParamLPre	58
Table 53	SpiQSpiParamLead	58
Table 54	SPI Parity Support	59
Table 55	SpiAssignedHwModule	59
Table 56	SpiAssignedHwChannel	60
Table 57	Chip select Gpio Port selection	60
Table 58	Chip select Gpio Pin selection	61
Table 59	SPI Sequence Interruptible	61
Table 60	SPI Sequence End Notification	62
Table 61	SPI Sequence ID	62
Table 62	SPI Job Linking	63
Table 63	SPI Sleep Enable	64
Table 64	SPI Hw Pin MISO	65
Table 65	SPI Slave Hw Pin MOSI	65
Table 66	SPI Slave Hw Pin SCLK	66
Table 67	SPI Hw Dma Channel Reception Ref	66
Table 68	SPI HW Dma Channel Transmission Ref	66
Table 69	DEM for SPI Receive error	67
Table 70	DEM for SPI DMA Transfer Error	67
Table 71	SpiMaxHwUnit	68
Table 72	ArMajorVersion	68
Table 73	ArMinorVersion	69
Table 74	ArPatchVersion	69
Table 75	SwMajorVersion	69
Table 76	SwMinorVersion	70
Table 77	SwPatchVersion	70
Table 78	VendorId	70
Table 79	ModuleId	71
Table 80	Release	71
Table 81	SPI Maximum Sequence	72
Table 82	SPI Maximum Job	72
Table 83	SPI Maximum Channel	73
Table 84	SPI Maximum IB Buffer Size	73
Table 85	SPI Maximum EB Channels	73
Table 86	SPI Maximum IB Channels	74
Table 87	SPI Write Lock Index	74
Table 88	SPI ASIL Sequence Array Index	75
Table 89	SPI QM Sequence Array Index	75
Table 90	SPI ASIL Job Array Index	76
Table 91	SPI QM Job Array Index	76
Table 92	Spi Job End Notification Required	77
Table 93	SPI QSPIz Module used	77
Table 94	SPI_QSPIz_MRIS_SEL	78
Table 95	SPI_QSPIz_HWTYPE	78
Table 96	SPI_MAX_HW_UNIT	79
Table 97	SPI_DEBUG_SUPPORT	79
Table 98	SPI_SUPPORT_CONCURRENT_SYNC_TRANSMIT	80

List of Tables

Page

Table 99	SPI_SYNCTRANSMIT_TIMEOUT_COUNTER	80
Table 100	SPI_MASTER_MODULES_USED	81
Table 101	SPI_ASIL_MASTER_MODULE_USED	81
Table 102	SPI_NUM_ASIL_MASTER_MODULES	81
Table 103	SPI_NUM_QM_ASYNC_MASTERS	82
Table 104	SPI_NUM_ASIL_SLAVE_MODULES	82
Table 105	SPI_SLAVE_MODULE_INDEX	83
Table 106	SPI_SLAVE_ENABLE	83
Table 107	SPI_CONFIG_COUNT	83
Table 108	Vendor ID	85
Table 109	Module ID	85
Table 110	AUTOSAR Major Version	85
Table 111	AUTOSAR Minor Version	85
Table 112	AUTOSAR Revision Version	86
Table 113	Software Major Version	86
Table 114	Software Minor Version	86
Table 115	Software Patch Version	86
Table 116	VendorApilInfix	86
Table 117	Release	87
Table 118	Type Definition Spi_ConfigType	88
Table 119	Type Definition Spi_StatusType	88
Table 120	Type Definition Spi_JobResultType	88
Table 121	Type Definition Spi_SeqResultType	89
Table 122	Type Definition spi_Datatype	89
Table 123	Type Definition Spi_NumberOfDataType	90
Table 124	Type Definition Spi_ChannelType	90
Table 125	Type Definition Spi_JobType	90
Table 126	Type Definition Spi_SequenceType	90
Table 127	Type Definition Spi_HWUnitType	91
Table 128	Type Definition Spi_AsyncModeType	91
Table 129	Specification of API Spi_Init	91
Table 130	Specification of API Spi_InitCheck	92
Table 131	Specification of Spi_DeInit	93
Table 132	Specification of Spi_WriteIB	94
Table 133	Specification of Spi_AsyncTransmit	95
Table 134	Asynchronous transmission with different Chip Select configurations	101
Table 135	Specification of API Spi_ReadIB	102
Table 136	Specification of API Spi_SetupEB	103
Table 137	Specification of API Spi_GetStatus	105
Table 138	Specification of API Spi_GetJobResult	106
Table 139	Specification of API Spi_GetSequenceResult	107
Table 140	Specification of API Spi_GetVersionInfo	108
Table 141	Specification of API Spi_SyncTransmit	109
Table 142	Specification of API Spi_GetHWUnitStatus	110
Table 143	Specification of API Spi_Cancel	111
Table 144	Specification of API Spi_SetAsyncMode	112
Table 145	Specification of API Spi_SlaveSetupEB	113
Table 146	Specification of API Spi_SlaveReadPrepare	114
Table 147	Specification of API Spi_MainFunction	115
Table 148	Specification of API Spi_IsrDmaQspiRx	117
Table 149	Specification of API Spi_IsrDmaQspiTx	118
Table 150	Specification of API Spi_IsrQspiError	118
Table 151	Specification of API Spi_IsrDmaError	119
Table 152	Specification of API Spi_IsrQspiPt	119
Table 153	Specification of API Spi_IsrQspiUsr	120
Table 154	Specification of API Spi_IsrCheckDmaError	121
Table 155	Specification of EA Spi_Init	121
Table 156	Specification of EA Spi_DeInit	121
Table 157	Specification of EA Spi_WriteIB	122

List of Tables

Page

Table 158	Specification of EA Spi_AsyncTransmit.....	122
Table 159	Specification of EA Spi_SyncTransmit.....	122
Table 160	Specification of EA Spi_GetSequenceResult	122
Table 161	Specification of EA Spi_Cancel	123
Table 162	DET and DEM Error Classification and Error Codes	124
Table 163	Safety Error Classification and Error Codes	124
Table 164	Description of Production errors.	125
Table 165	Description of Debug Macros.....	126
Table 166	Aurix Derivates Information.....	137
Table 167	QSPI MRST(MISO), MTSR(MOSI), SCLK and Chip Select Pin Connections for TC299	138
Table 168	QSPI MRST(MISO), MTSR(MOSI), SCLK and Chip Select Pin Connections for TC298	140
Table 169	QSPI MRST(MISO), MTSR(MOSI), SCLK and Chip Select Pin Connections for TC297	142
Table 170	QSPI MRST(MISO), MTSR(MOSI), SCLK and Chip Select Pin Connections for TC277	144
Table 171	QSPI MRST(MISO), MTSR(MOSI), SCLK and Chip Select Pin Connections for TC275	146
Table 172	QSPI MRST(MISO), MTSR(MOSI), SCLK and Chip Select Pin Connections for TC267	148
Table 173	QSPI MRST(MISO), MTSR(MOSI), SCLK and Chip Select Pin Connections for TC265	150
Table 174	QSPI MRST(MISO), MTSR(MOSI), SCLK and Chip Select Pin Connections for TC264/TC244 152	
Table 175	QSPI MRST(MISO), MTSR(MOSI), SCLK and Chip Select Pin Connections for TC237	154
Table 176	QSPI MRST(MISO), MTSR(MOSI), SCLK and Chip Select Pin Connections for TC234	156
Table 177	QSPI MRST(MISO), MTSR(MOSI), SCLK and Chip Select Pin Connections for TC233	158
Table 178	QSPI MRST(MISO), MTSR(MOSI), SCLK and Chip Select Pin Connections for TC224/TC214 159	
Table 179	QSPI MRST(MISO), MTSR(MOSI), SCLK and Chip Select Pin Connections for TC223/TC213 161	
Table 180	QSPI MRST(MISO), MTSR(MOSI), SCLK and Chip Select Pin Connections for TC222/TC212 162	

List of Code Listings

Page

Code Listing 1	Interrupt Priority configuration for QSPI2 node (Irq_Cfg.h)	116
Code Listing 2	SPI Configuration Header file	127
Code Listing 3	SPI Channel Configuration Parameter	128
Code Listing 4	SPI Job Configuration Parameter.....	128
Code Listing 5	SPI Sequence Configuration Parameter	129
Code Listing 6	Hardware Module and Dma configuration.....	130
Code Listing 7	Run time Configurable Parameters	131
Code Listing 8	SPI Driver Initialization	131
Code Listing 9	SPI EEPROM Access.....	132
Code Listing 10	Spi Slave readback calling procedure(Master Receive data, half-duplex)	134

1 Introduction

This user manual provides information regarding the functionality, configuration parameters and API implementation for the SPI handler driver.

The SPI handler driver is built according to the AUTOSAR specification, refer [\[2\]](#)

This user manual is intended to help the user to get acquainted with the SPI handler driver implementation for the Aurix family of microcontrollers. It is the documentation that describes how to use the SPI handler driver.

1.1 Scope

This document addresses the following features of the SPI handler driver implementation,

- Aurix microcontroller HW peripherals used to realize the driver
- File structure of SPI handler driver
- Configuration parameters of the driver
- Parameters published by SPI handler driver
- API specification
- Code snippets illustrating the common use cases.
- Limitations and assumptions

This documentation applies to the SPI handler driver variant: Variant PB.

Parameters with "Pre-compile time", "Link time" and "Post-build time" are allowed in Variant PB.

1.2 Abbreviations

The abbreviations used inside this document are explained in [Table 1](#).

Table 1 Abbreviations used in this Document

Abbreviations	Explanation
μC	Microcontroller
μs	Micro Second
API	Application Programming Interface
AUTOSAR	AUTomotive Open System ARchitecture
CS	Chip Select
DEM	Diagnostic Error Manager
DET	Development Error Tracer
DMA	Direct Memory Access
EB	External Buffer
ECU	Electronic Control Unit
HW	Hardware
IB	Internal Buffer
MCAL	MicroController Abstraction Layer
MCU	MicroController Unit
MISO	Master Input Slave Output

Abbreviations	Explanation
MOSI	Master Output Slave Input
MRST	Master Receive Slave Transmit (MISO)
MTSR	Master Transmit Slave Receive (MOSI)
RAM	Random Access Memory
ROM	Read Only Memory
SFR	Special Function Register
SCLK	Serial Clock
SLSO	Slave Select Output
SPI	Serial peripheral Interface
QSPI	Queued SPI
SW	Software
GPIO	General purpose input output

1.3 Reference

This section lists down all relevant documents for this user manual.

- [1] Software Requirements:
AUTOSAR_SRS_SPI_HandlerDriver.pdf (AS 4.0.3),
- [2] Software Specifications:
AUTOSAR_SWS_SPIHandlerDriver.pdf , (AS 4.0.3)
- [3] Target User Manuals : tc29xB_um_v1.2_2.pdf, tc27xC_um_v2.0OM.pdf, tc27xD_um_v2.2.pdf,
tc26xB_um_v1.3.pdf, tc23x_tc22x_um_v1.1.pdf
- [4] User's Manual MCU Driver Aurix_MC-ISAR_UM_MCUDriver.pdf
- [5] User's Manual PORT Driver Aurix_MC-ISAR_UM_PORTDriver.pdf
- [6] User's Manual IRQ Driver Aurix_MC-ISAR_UM_IRQDriver.pdf
- [7] Hardware Errata Analysis: TC27x_MC-ISAR_HWErtaAnalysis.xls
- [8] Hardware Errata Analysis: TC26x_MC-ISAR_HWErtaAnalysis.xls
- [9] Hardware Errata Analysis: TC29x_MC-ISAR_HWErtaAnalysis.xls
- [10] Hardware Errata Analysis: TC23x_MC-ISAR_HWErtaAnalysis.xls
- [11] Hardware Errata Analysis: TC22x_TC21x_MC-ISAR_HWErtaAnalysis.xls

2 SPI Handler Driver Overview

The Aurix QSPI supports full-duplex and half-duplex synchronous communication. Data width, shift direction, clock polarity and phase are programmable. This allows communication with SPI-compatible devices.

The Aurix families of microcontroller provide QSPI modules to implement synchronous serial communication. These modules are used to realize the SPI handler driver.

The required QSPI is configured using the configuration parameters described in chapter [\[4\]](#)

The driver provides APIs to transmit the data in asynchronous mode and synchronous mode. The SPI handler driver provides set of APIs to read the status of sequence and job.

The APIs provided by the SPI handler driver is described in chapter [\[6.2\]](#).

2.1 Software Hardware Mapping

This section introduces the user to the HW features used for implementation. The [Figure 1](#) illustrates the hardware modules and their interaction with the software modules.

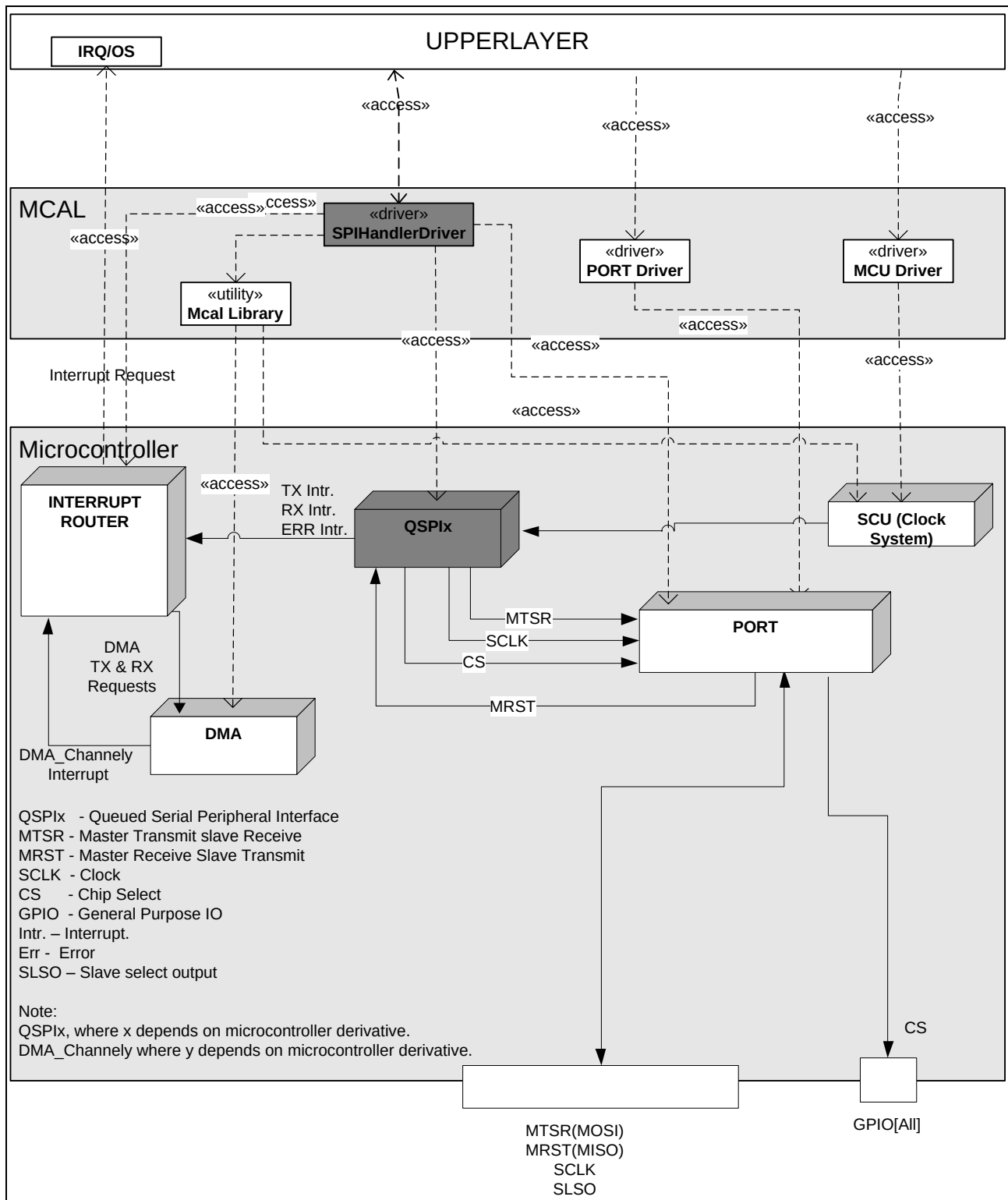


Figure 1 Software Hardware Mapping of SPI HandlerDriver.

The different HW modules needed by the driver and their purpose is explained below.

2.1.1 QSPIn

Refer [\[10.1\]](#) for the QSPI's provided by the Aurix families of microcontroller. The SPI Handler driver configures the QSPIn to transmit data in the asynchronous mode and synchronous mode. The QSPI uses

the service of DMA to transmit data in the asynchronous mode. SPI driver handler configures the request for the DMA channel to trigger the DMA transaction.

The SPI handler driver configures the SPI interrupts and DMA Channel interrupts. The SPI handler driver provides notifications to the upper layer informing about the completion of the sequence and completion of jobs.

2.1.2 PORT

The PORT Peripheral controls all access to the Pins required by SPI. Port pin direction and mode selection is setup in the PORT driver. Section [10.2] shows the port pins mapped to QSPI peripheral

Chip Select Pins: In addition to given SLSO pins, all the available GPIO pin/s can be used as potential ChipSelect pins. The Configuration of Chipselect for GPIO pins shall be done in Spi driver.

It is important to note, that the PORT driver must be initialized for the pins required by SPI. Refer [5] PORT user manual for details about PORT configuration.

2.1.3 System Clock

Qspi peripheral clock and baud rate are derived from system clock. The system clock is set up via MCU driver. It is mandatory that user sets up an appropriate system clock.

Refer [4] MCU user manual for details about MCU configuration.

2.1.4 DMA

The SPI uses the service of the DMA for data transmission in Asynchronous mode. The Dma Channels used by Spi are Configurable and the allocation of the same is done in MCU. Each channel can work in software triggered mode or hardware triggered mode. For SPI to use DMA channel, the channel should be configured as hardware trigger type.

In this implementation the Combined Software/Hardware Controlled Mode will be used. Hence the first DMA transfer will be triggered by software when setting the corresponding request channel bit in the Software Transaction Request Register, while the hardware requests are still disabled. The subsequent DMA transfers are triggered by the corresponding hardware request, this happens after the hardware requests have been enabled.

The QSPI Peripheral's RX or TX Interrupt Line is used to trigger the DMA transactions. Each QSPI Peripheral uses two DMA channels to trigger reception and transmission respectively.

The DMA Mechanism is used for the Asynchronous Transmission mode. It is not used during Synchronous transmission mode. The SPI handler driver initializes the DMA channels used.

2.1.5 IRQ

IRQ driver is responsible for initializing the priority of interrupts required by SPI and DMA.

2.2 Software Driver Description

The below figure (Interfaces of SPI Handler Driver) shows the Application Programming Interface (API) for the SPI handler driver and its interaction with other software modules.

- Development Error Tracer (DET) is required if DET is enabled for the SPI handler driver.
- The DEM is also required for handling the production errors.
- The scheduler is required for calling the Spi_MainFunction_Handling in the polling mode for LEVEL2 driver

The APIs of the driver are explained in detail in chapter [6.2]

The configuration data of the driver contains SPI driver configuration and SpiGeneral configuration. The SPI driver configuration is made up of several channels, jobs, sequences, external device (Slave), and QSPI hardware configurations. The individual containers and their configuration parameters are explained in detail in chapter [\[4\]](#)

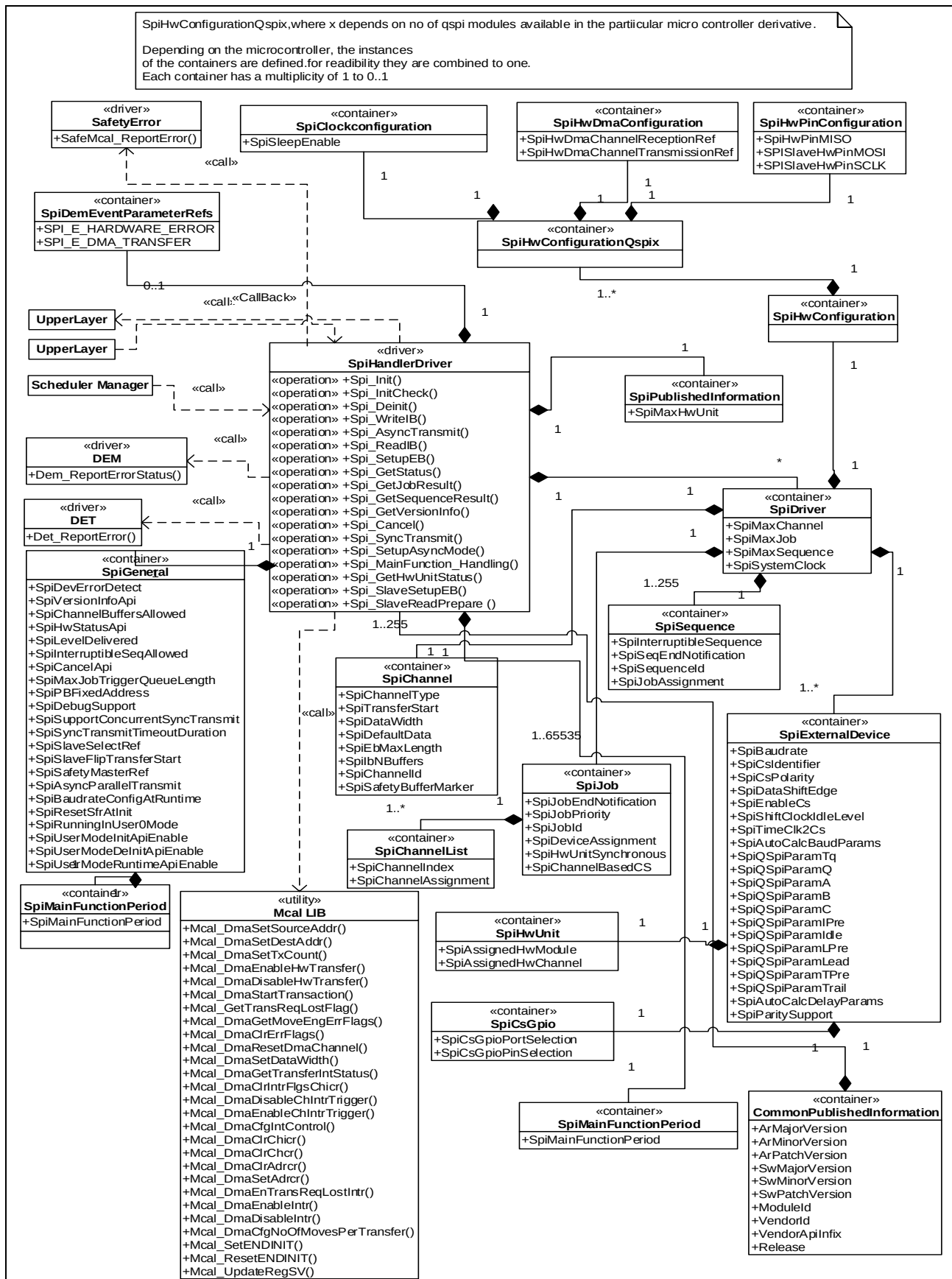


Figure 2 Interfaces of SPI Handler Driver

3 File Structure

This section provides details about the SPI handler driver files and other related files.

3.1 Module File Structure

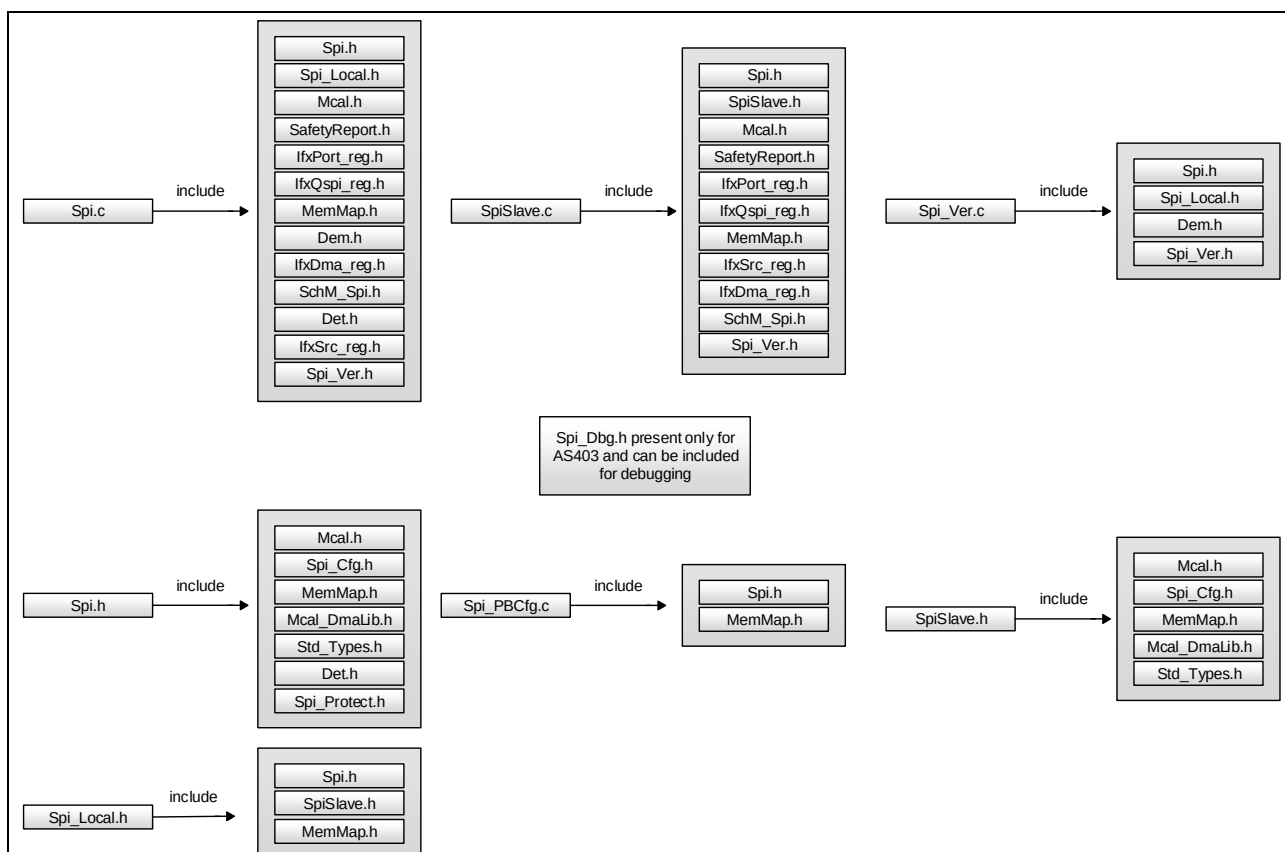


Figure 3 File Structure of SPI Handler Driver

File Contents

File Name	File Contents
Platform_Types.h	Platform dependencies are declared here
Std_Types.h	Standard data types to be used are declared here. This file in turn includes compiler.h
Compiler.h	Compiler dependencies are abstracted here. This file in turn includes Compiler_Cfg.h
Compiler_Cfg.h	Contains abstraction of compiler specific keywords used for addressing data and code within declaration and definition
MemMap.h	Mapping of code and data (variables, constant variables) to specific memory sections
IfxQspi_reg.h	Includes the register definition file for Qspi
Mcal.h	Provides library functions for MCAL layer
IfxSrc_reg.h	Includes the register definition file for IR
SchM_Spi.h	Functions to enable/disable interrupts are declared here
Det.h	Provides the functionality of Development Error Tracer

File Name	File Contents
Dem.h	Provides the functionality of Diagnostic Event Manager
Safety_Report.h	Provides the functionality of Safety Error Reporting
Spi_Dbg.h	Provides the debug variables (only in AS4.x)
Spi.h	Spi Driver data types, API Functions and Interrupt handlers are exported for the user
Spi.c	Spi Driver Function implementation
Spi_Cfg.h	Contains Driver precompile configuration parameters
Spi_PBCfg.c	Contains Driver postbuild configuration parameters
SpiSlave.c	Contains slave specific APIs (for safety readback purpose)
SpiSlave.h	Provides header definitions for slave implementation
IfxDma_reg.h	Includes the register definition file for Dma
IfxPort_reg.h	Includes the register definition file for Port
Spi_Ver.h	File include prototypes of local SchM calls(Only in AS321)

User of the SPI handler driver needs to include only Spi.h file to import the functionality of SPI handler driver.

4 Configuration Documentation

This chapter summarizes

Configuration concept used for structuring the driver

All configuration parameters and their description

4.1 Configuration Concept

4.1.1 Configuration Class

The development of basic software drivers involves the following development cycles:

Compiling,

Linking and

Downloading of the executable to ECU memory.

Configuration of parameters can be done in any of these process-steps: pre-compile time, link time or even post-build time. According to the process-step that does the configuration of parameters, the configuration classes are categorized as below

Pre-compile time: The pre-compile time configuration is implemented by means of compiler switches or variables affecting the total build process. Pre-compiler configuration requires the Driver to be compiled, linked and located and hence the values of pre-compile parameters must be **frozen before compilation** and therefore can be used in source code distributions of the Driver only.

Link time: The Link time configuration doesn't affect compilation stage but impacts the linkage and locating stages. The Link time parameters can be used in binary (library) distributions of the Driver and the values of the Link time parameters are specified by means of external configuration source code that must be linked to the Driver during the build process.

Post-build time: The Post-build time configuration will not affect the build process and hence Post-build time configuration is applied to Driver at runtime. Since configuration location is statically independent of the Driver functionality itself but linked dynamically (e.g. at Driver initialization), post-build time configuration is used for re-programmable configuration data.

4.1.2 Configuration Variant

Based on the most suitable configuration class that may be needed for the configuration parameters of the Driver, a Driver can be delivered as the following variants

Variant PC: This variant is limited to pre-compile configuration parameters only. The intention of this variant is to optimize the parameters configuration for a source code delivery.

Variant LT: This variant allows a mix of pre-compile time- link time configuration parameters. The intention of this variant is to optimize the parameters configuration for an object code delivery.

Variant PB: This variant allows a mix of pre-compile time-, post build-time configuration parameters. The intention of this variant is to optimize the configuration parameters for a re-loadable binary.

The SPI Handler Driver is delivered as a Variant PB.

4.1.3 How to read the Configuration Class field

The examples cited below help the user to identify the configuration class of the parameter for a given delivery type.

Note: Topics [4.1.1](#), [4.1.2](#) should be read for understanding the configuration class field

Table 3 Configuration Class Example 1

Configuration Class:	Pre-compile	x	All Variants
	Link Time	--	--
	Post build	--	--

'x' indicates that the parameter is implemented as a pre-compile parameter in all types of deliveries i.e. Variant PB, Variant LT, Variant PC.

Table 4 Configuration Class Example 2

Configuration Class:	Pre-compile	x	Variant PC
	Link Time	--	--
	Post build	x	Variant PB

'x' indicates that the parameter is implemented as a

- pre-compile parameter if the delivery type is Variant PC delivery
- post-build parameter if the delivery type is a Variant PB delivery

4.2 Container: SpiGeneral

This container contains the general configuration (parameters) of the SPI Handler/Driver.

4.2.1 SpiDevErrorDetect

Table 5 Development Error Tracer Configuration

Syntax:	SpiDevErrorDetect		
Source:	This parameter is defined by the AUTOSAR, refer [2] .		
File:	Spi.xdm/.bmd		
Range:	Values: TRUE: Enabled FALSE: Disabled Default value: FALSE		
Configuration Class	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	Switches the development error detection ON or OFF.		
SFRs Modified:	None		
Scope:	Driver		
Dependency:	None		

4.2.2 SpiVersionInfoApi

Table 6 Module Version Information API Configuration

Syntax:	SpiVersionInfoApi		
Source:	This parameter is defined by the AUTOSAR, refer [2]		
File:	Spi.xdm/.bmd		
Range:	Values: TRUE: Version info API enabled		

	FALSE: Version Info API disabled Default value: FALSE		
Configuration Class	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
SFRs Modified:	None		
Description:	Pre-processor switch to enable / disable the API to read out the driver version information		
Scope:	Driver		
Dependency:	None		

4.2.3 SpiLevelDelivered

Table 7 SPI Level Delivered

Syntax:	SpiLevelDelivered		
Source:	This parameter is defined by the AUTOSAR, refer [2]		
File:	Spi.xdm/.bmd		
Range:	Values: 0 - Simple synchronous SPI Handler/Driver functionalities are selected. 1 - Basic asynchronous SPI Handler/Driver functionalities are selected. 2 - Enhanced SPI Handler/Driver functionalities are selected. Default value: 1		
Configuration Class	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
SFRs Modified:	None		
Description:	This parameter is to select the SPI Handler/Driver level of scalable functionality. This parameter shall be configured with one of the 3 values according to the described levels: 0, 1 or 2. SpiLevelDelivered is 0 (LEVEL0): The driver handles only simple synchronous transmission on all available SPI busses. SpiLevelDelivered is 1(LEVEL1): The driver handles only simple asynchronous transmission on all available SPI busses. The asynchronous behaviour is implemented using interrupt mechanism. SpiLevelDelivered is 2(LEVEL2): The LEVEL2 which is an enhanced behavior of the driver will handle both synchronous and asynchronous transmissions. In the asynchronous transmission mode of LEVEL2, the transmission can be achieved using POLLING mode or INTERRUPT mode. The transmission mode is selected by using the API Spi_SetAsyncMode. In case of SPI_LEVEL_DELIVERED 2, following combinations wrt synchronous and asynchronous jobs are possible, - Configuration with atleast one synchronous and atleast one asynchronous jobs. i.e., both synchronous and asynchronous services are available (as per requirement SPI128). - Configuration with atleast one asynchronous job and zero synchronous jobs. i.e., asynchronous service is available and synchronous services are not		

	available
	Note: In LEVEL 2, configuration with atleast one synchronous job and zero asynchronous jobs is not supported wherein; Spi_SyncTransmit API would not be available.
Scope:	Driver
Dependency:	None

4.2.4 SpiChannelBuffersAllowed

Table 8 SPI Channel Buffers Allowed

Syntax:	SpiChannelBuffersAllowed		
Source:	This parameter is defined by the AUTOSAR, refer [2] .		
File:	Spi.xdm/.bmd		
Range:	Values: 0 - The usage 0, only internal buffers are selected in handler/driver. 1 - The usage 1, only external buffers are selected in handler/driver. 2 - The usage 2, both internal and external buffers are selected in handler/driver. Default value: 2		
Configuration Class	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
SFRs Modified:	None		
Description:	This parameter is to select the SPI handler/driver channel buffer usage. This parameter shall be configured with one of the 3 values according to the described usage: 0, 1 or 2.		
Scope:	Driver		
Dependency:	None		

4.2.5 SpiInterruptibleSeqAllowed

Table 9 SPI Interruptible Sequence Allowed

Syntax:	SpiInterruptibleSeqAllowed		
Source:	This parameter is defined by the AUTOSAR, refer [2] .		
File:	Spi.xdm/.bmd		
Range:	Significant only if SpiLevelDelivered is 1 or 2 Values: TRUE – Interruptible sequence allowed FALSE – Interruptible sequence not allowed Default value: FALSE		
Configuration Class	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
SFRs Modified:	None		
Description:	Switches the interruptible sequences handling functionality ON or OFF.		
Scope:	Driver		

Dependency:	SpiLevelDelivered
-------------	-------------------

4.2.6 SpiHwStatusApi

Table 10 SPI HW Status API

Syntax:	SpiHwStatusApi		
Source:	This parameter is defined by the AUTOSAR, refer [2] .		
File:	Spi.xdm/.bmd		
Range:	Values: TRUE - Spi_GetHWUnitStatus function is available to user FALSE – Spi_GetHWUnitStatus function is not available to user Default value: FALSE		
Configuration Class	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
SFRs Modified:	None		
Description:	Switches the Spi_GetHWUnitStatus function ON or OFF.		
Scope:	Driver		
Dependency:	None		

4.2.7 SpiCancelApi

Table 11 SPI Cancel API

Syntax:	SpiCancelApi		
Source:	This parameter is defined by the AUTOSAR, refer [2] .		
File:	Spi.xdm/.bmd		
Range:	Values: TRUE - Spi_Cancel function is available to user FALSE – Spi_Cancel function is not available to user Default value: FALSE		
Configuration Class	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
SFRs Modified:	None		
Description:	Switches the Spi_Cancel function ON or OFF.		
Scope:	Driver		
Dependency:	None		
Dependency:	None		

4.2.8 SpiMaxJobTriggerQueueLength

Table 12 SPI Maximum Job Trigger Queue Length

Syntax:	SpiMaxJobTriggerQueueLength
Source:	This parameter is microcontroller specific

File:	Spi.xdm/.bmd		
Range:	Significant only if SpiLevelDelivered is 1 or 2 Values: 1 - 65535 Default value: 2		
Configuration Class	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
SFRs Modified:	None		
Description:	This parameter specifies the maximum length of the job and sequence queue. In the asynchronous mode (SpiLevelDelivered is 1 or 2) Every call of the function Spi_AsyncTransmit, the sequence IDs and its corresponding job IDs are filled in the queue (sequence queue and job queue). The queue is filled until the queue length reaches SPI_MAX_JOB_TRIG_Q_LENGTH. Note: The SPI Job queue and the sequence queue are the circular queue, the jobs and the sequences are not removed from the queue once they are serviced. The user should be aware of the queue size and ideally the queue size should be maximum jobs configured		
Scope:	Driver		
Dependency:	None		
Dependency:	SpiLevelDelivered is 1 or 2		

4.2.9 SpiPBFixedAddress

Table 13 SPI Post build fixed address feature

Syntax:	SpiPBFixedAddress		
Source:	This parameter is microcontroller specific		
File:	Spi.xdm/.bmd		
Range:	Values: TRUE – Fixed address feature used FALSE – Fixed address feature not used Default value: FALSE		
Configuration Class	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
SFRs Modified:	None		
Description:	The fixed address feature supports the optimization of a post build configuration with a fixed address by avoiding unnecessary indirection.		
Scope:	Driver		
Dependency:	None		
Dependency:	None		

4.2.10 SpiDebugSupport

Table 14 SpiDebugSupport

Syntax:	SpiDebugSupport
----------------	-----------------

Source:	This parameter is defined by the AUTOSAR, refer [2] .		
File:	Spi.xdm/.bmd		
Range:	Values: TRUE – Debug support enabled FALSE – Debug support Disabled Default value: FALSE		
Configuration Class	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
SFRs Modified:	None		
Description:	Pre-processor switch for enabling/disabling the debugging support		
Scope:	Driver		
Dependency:	None		

4.2.11 SpiSupportConcurrentSyncTransmit

Table 15 SpiSupportConcurrentSyncTransmit

Syntax:	SpiSupportConcurrentSyncTransmit		
Source:	This parameter is defined by the AUTOSAR, refer [2] .		
File:	Spi_Cfg.h		
Range:	TRUE – Concurrent Sync Transmit on multiple buses allowed FALSE – Concurrent Sync Transmit on multiple buses not allowed Default: FALSE		
Configuration Class	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
SFRs Modified:	None		
Description:	Specifies whether concurrent Spi_SyncTransmit() calls for different sequences shall be configurable		
Scope:	Driver		
Dependency:	Applicable only if SpiLevelDelivered = 0 or 2		

4.2.12 SpiSyncTransmitTimeoutDuration

Table 16 SpiSyncTransmitTimeoutDuration

Syntax:	SpiSyncTransmitTimeoutDuration		
Source:	This parameter is microcontroller specific		
File:	Spi_Cfg.h		
Range:	Values: 0x1 – 0xFFFFFFFF Default value: 0x000FFFFF		
Configuration Class	Pre-Compile	x	All Variants
	Link Time	--	--

	Post Build	--	--
SFRs Modified:	None		
Description:	The parameter is used as timeout loop counter while waiting for data transmission/reception of 1 data element.		
Scope:	Driver		
Dependency:	None		

4.2.13 SpiSlaveSelectRef

Table 17 SpiSlaveSelectRef Configuration

Syntax:	SpiSlaveSelectRef		
Source:	This parameter is microcontroller specific		
File:	Spi.xdm/.bmd		
Range:	Values: SPI_SLAVE_UNUSED QSPIz Default value: SPI_SLAVE_UNUSED Range of z depends on micro controller derivative. Refer [10.1] for QSPI module availability for specific derivative.		
Configuration Class	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
SFRs Modified:	None		
Description:	Reference to the QSPI module which is used as slave. If SpiSafetyMasterRef is - QSPI0 then valid slave module must be any of QSPI1 to z OR SPI_SLAVE_UNUSED - If SpiSafetyMasterRef is QSPI1 to z then valid slave module must be QSPI0 OR SPI_SLAVE_UNUSED		
Scope:	Driver		
Dependency:	SpiSafetyEnable and SpiSafetyMasterRef		

4.2.14 SpiSlaveFlipTransferStart

Table 18 SpiSlaveFlipTransferStart Configuration

Syntax:	SpiSlaveFlipTransferStart		
Source:	This parameter is microcontroller specific		
File:	Spi.xdm/.bmd		
Range:	Values: TRUE: Enabled FALSE: Disabled Default value: FALSE		
Configuration Class	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--

SFRs Modified:	None
Description:	Flips the MSB->LSB or LSB->MSB on slave reception depending on corresponding master module's Spichannel's SpiTransferStart configuration.
Scope:	Driver
Dependency:	SpiSafetyEnable

4.2.15 SpiSafetyMasterRef

Table 19 SpiSafetyMasterRef Configuration

Syntax:	SpiSafetyMasterRef		
Source:	This parameter is microcontroller specific		
File:	Spi.xdm/.bmd		
Range:	Values: QSPIz Default value: QSPI0 Range of z depends on micro controller derivative. Refer [10.1] for Qspi module availability for specific derivative. Note: for TC298 device the default value is QSPI1		
Configuration Class	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
SFRs Modified:	None		
Description:	Reference to the QSPI module which is used as Safety Master.		
Scope:	Driver		
Dependency:	SpiSafetyEnable		

4.2.16 SpiAsyncParallelTransmit

Table 1 SpiAsyncParallelTransmit Configuration

Syntax:	SpiAsyncParallelTransmit		
Source:	This parameter is microcontroller specific		
File:	Spi.xdm/.bmd		
Range:	Values: TRUE: Enabled FALSE: Disabled Default value: FALSE		
Configuration Class	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	Switches for SPI Parallel Asynchronous Transmission ON or OFF The Syntax of the generated parameter is, #define SPI_ASYNC_PARALLEL_TRANSMIT (STD_ON) Or #define SPI_ASYNC_PARALLEL_TRANSMIT (STD_OFF) This parameter is editable for SpiLevelDelivered = 1 and 2 Only (Async Transmission)		

	Restriction: (Autosar Requirement: SPI221_Conf): If SpiAsyncParallelTransmit is Enabled SPI sequence cannot have the multiple Jobs which belong to a different QSPI modules. However SPI sequence can have multiple Jobs which belong to the same QSPI module.
Scope:	Driver
Dependency:	SpiLevelDelivered

4.2.17 SpiBaudrateConfigAtRuntime

Table 2 SpiBaudrateConfigAtRuntime Configuration

Syntax:	SpiBaudrateConfigAtRuntime		
Source:	This parameter is microcontroller specific		
File:	Spi.xdm/.bmd		
Range:	Values: TRUE: Enable baudrate change at runtime, i.e., multiple baudrate configurations per hardware. FALSE: Disable baudrate change at runtime, i.e., single baudrate configuration per hardware Default value: FALSE		
Configuration Class	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	Pre-processor switch for setting up ECON sfr during Spi_Init()[FALSE] or before starting of a new Job during runtime[TRUE]. If parameter is enabled(TRUE), a particular hardware QSPIx, channely can be configured with different baudrates, assigned to jobs. Baudrate for these jobs are configured at runtime. If parameter is disabled(FALSE), a particular hardware QSPIx, channely can have only one baudrate configuration which is initialized during Spi_Init() API. The Syntax of the generated parameter in Spi_Cfg.h file is, #define SPI_BAUDRATE_AT_RUNTIME (STD_ON) Or #define SPI_BAUDRATE_AT_RUNTIME (STD_OFF)		
Scope:	Driver		
Dependency:	None		

4.2.18 SpiResetSfrAtInit

Table 3 SpiResetSfrAtInit Configuration

Syntax:	SpiResetSfrAtInit
Source:	This parameter is microcontroller specific
File:	Spi.xdm/.bmd
Range:	Values:

	TRUE: Enabled FALSE: Disabled Default value: FALSE		
Configuration Class	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	Pre-processor switch to enable/disable resetting SFRs used by SPI driver to its reset value in the Spi_Init() API The Syntax of the generated parameter is, #define SPI_RESET_SFR_INIT (STD_ON) Or #define SPI_RESET_SFR_INIT (STD_OFF) Note: SFRs related to interrupts are centrally reset by using Irq_ClearAllInterruptFlags API from IRQ module.		
Scope:	Driver		
Dependency:	None		

4.2.19 SpiUserModelInitApiEnable

Table 4 SpiUserModelInitApiEnable

Syntax:	SpiUserModelInitApiEnable		
Source:	This is IFX specific parameter		
File:	Spi_Cfg.h		
Range:	Values: TRUE: Enabled FALSE: Disabled Default value: FALSE		
Configuration Class	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	Pre-processor switch to enable/disable the Normal/Protected mode		
Scope:	Driver		
Dependency:	None		

4.2.20 SpiUserModeDeInitApiEnable

Table 5 SpiUserModeDeInitApiEnable

Syntax:	SpiUserModeDeInitApiEnable		
Source:	This is IFX specific parameter		
File:	Spi_Cfg.h		
Range:	Values: TRUE: Enabled FALSE: Disabled Default value: FALSE		

Configuration Class	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	Pre-processor switch to enable/disable the Normal/Protected mode		
Scope:	Driver		
Dependency:	None		

4.2.21 SpiUserModeRuntimeApiEnable

Table 6 SpiUserModeRuntimeApiEnable

Syntax:	SpiUserModeRuntimeApiEnable		
Source:	This is IFX specific parameter		
File:	Spi_Cfg.h		
Range:	Values: TRUE: Enabled FALSE: Disabled Default value: FALSE		
Configuration Class	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	Pre-processor switch to enable/disable the Normal/Protected mode		
Scope:	Driver		
Dependency:	None		

4.2.22 SpiRunningInUser0Mode

Table 7 SpiRunningInUser0Mode

Syntax:	SpiRunningInUser0Mode		
Source:	This is IFX specific parameter		
File:	Spi_Cfg.h		
Range:	Values: TRUE: Enabled FALSE: Disabled Default value: FALSE		
Configuration Class	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	Pre-processor switch to enable/disable the Normal/Protected mode		
Scope:	Driver		
Dependency:	None		

4.3 Container: SpiMainFunctionPeriod

This container contains the configuration parameter that defines the cycle time of the function Spi_MainFunction_Handling.

4.3.1 SpiMainFunctionPeriod

Table 8 SpiMainFunctionPeriod Configuration

Syntax:	SpiMainFunctionPeriod		
Source:	This parameter is microcontroller specific		
File:	Spi.xdm/.bmd		
Range:	Values: Range: 0.0000001 .. 1.0 seconds Default value: 0.01 seconds		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	--	--
Description:	This parameter defines the cycle time of the function Spi_MainFunction_Handling in seconds. The parameter is not used by the driver itself, but it is used by upper layer. Bugzilla #60168		
Scope:	Driver		
Dependency:	None		

4.4 Container: SpiSafety

This container contains the configuration parameters to describe a Safety usage.

4.4.1 SpiSafetyEnable

Table 9 SpiSafetyEnable Configuration

Syntax:	SpiSafetyEnable		
Source:	This parameter is microcontroller specific		
File:	Spi.xdm/.bmd		
Range:	Values: TRUE: Enabled FALSE: Disabled Default value: FALSE		
Configuration Class	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
SFRs Modified:	None		
Description:	Pre-processor switch to enable / disable Safety.		
Scope:	Driver		
Dependency:	None		

4.4.2 SpiInitCheckApi

Table 10 SpiInitCheckApi Configuration

Syntax:	SpiInitCheckApi
Source:	This parameter is microcontroller specific

File:	Spi.xdm/.bmd		
Range:	Values: TRUE: Enabled FALSE: Disabled Default value: FALSE		
Configuration Class	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
SFRs Modified:	None		
Description:	Switches the SpiInitCheckApi ON or OFF. Pre-processor switch to enable / disable the API SpiInitCheckApi.		
Scope:	Driver		
Dependency:	SpiSafetyEnable		

4.5 Container: SpiDriver

4.5.1 SpiMaxChannel

Table 11 SPI Maximum channel

Syntax:	SpiMaxChannel		
Source:	This parameter is defined by the AUTOSAR, refer [2]		
File:	Spi.xdm/.bmd		
Range:	1 – 255		
Configuration Class	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
SFRs Modified:	None		
Description:	This parameter is disabled in the configuration tool. The value for this parameter is gathered by the tool. This parameter holds the total number of channels configured including IB channels and EB channels.		
Scope:	Driver		
Dependency:	None		

4.5.2 SpiMaxJob

Table 12 SPI Maximum jobs

Syntax:	SpiMaxJob		
Source:	This parameter is defined by the AUTOSAR, refer [2]		
File:	Spi.xdm/.bmd		
Range:	1 – 65535		
Configuration Class	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
SFRs Modified:	None		

Description:	This parameter is disabled in the configuration tool. The value for this parameter is gathered by the tool. This parameter holds the total number of jobs configured.
Scope:	Driver
Dependency:	None

4.5.3 SpiMaxSequence

Table 13 SPI Maximum sequence

Syntax:	SpiMaxSequence		
Source:	This parameter is defined by the AUTOSAR, refer [2]		
File:	Spi.xdm/.bmd		
Range:	1 – 255		
Configuration Class	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
SFRs Modified:	None		
Description:	This parameter is disabled in the configuration tool. The value for this parameter is gathered by the tool. This parameter holds the total number of sequence configured.		
Scope:	Driver		
Dependency:	None		

4.5.4 SpiSystemclock

Table 14 SpiSystemClock

Syntax:	SpiSystemClock		
Source:	This parameter is microcontroller specific.		
File:	Spi.xdm/.bmd		
Range:	Reference to parameter of type McuClockSettingConf Default Value: None		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	All Variants
SFRs Modified:	None		
Description:	This parameter refers to the system clock configured by MCU driver.		
Scope:	Driver		
Dependency:	McuClockSettingConf		

4.6 Container: SpiChannel

This container contains the configuration parameters to describe a channel.

4.6.1 SpiChannelType

Table 15 SPI Channel Type

Syntax:	SpiChannelType
----------------	----------------

Source:	This parameter is defined by the AUTOSAR, refer [2]		
File:	Spi.xdm/.bmd		
Range:	Values: IB - Channel of internal buffer type EB - Channel of external buffer type Default value: EB		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	All Variants
SFRs Modified:	None		
Description:	This parameter defines the buffer type (EB/IB) used by the channel.		
Scope:	Driver		
Dependency:	None		

4.6.2 SpiDataWidth

Table 16 SpiDataWidth

Syntax:	SpiDataWidth		
Source:	This parameter is defined by the AUTOSAR, refer [2]		
File:	Spi.xdm/.bmd		
Range:	Values: 2..32 Default value: 8 Note: The Qspi supports the datawidth from 2 to 32 bit/s. For the baudrate of 50Mhz, the minimum data width should be 4 bit width.		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	All Variants
SFRs Modified:	DATAENTRY[0]		
Description:	This parameter defines the width of the data to be transmitted.		
Scope:	Driver		
Dependency:	None		

4.6.3 SpiTransferStart

Table 17 SPI Transfer Start

Syntax:	SpiTransferStart		
Source:	This parameter is defined by the AUTOSAR, refer [2]		
File:	Spi.xdm/.bmd		
Range:	Values: LSB - Transmission starts with the least significant bit first MSB - Transmission starts with the most significant bit first Default value: LSB		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	All Variants

SFRs Modified:	BACON
Description:	This parameter defines LSB transmitted first or MSB transmitted first.
Scope:	Driver
Dependency:	None

4.6.4 SpilbNBuffers

Table 18 Number of IB Buffers

Syntax:	SpilbNBuffers		
Source:	This parameter is defined by the AUTOSAR, refer [2]		
File:	Spi.xdm/.bmd		
Range:	Significant if SpiChannelBuffersAllowed is 0 or 2 and SpiChannelType = IB Values: 1..65535 Default value: 10		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	All Variants
SFRs Modified:	None		
Description:	This parameter contains the maximum number of data buffers in case of IB channels only. The user must note that the IB buffers are handled by SW. Depending upon the number of buffers used buffer memory is created in software. The buffer is of type Spi_DataType In case of configuration with mixed 8/16/32-bit IB channels, the 8-bit channels are allowed to configure only even number of bytes. Any channel less than or equal to 8 bit is considered as 8-bit channel, from 9 bit to 16 bit (inclusive) is considered as 16 bit channel and above 16 bit is considered as 32 bit channel. Note: In case of 32 bit data, the SpilbNBuffers value is quadrupled for the user. Note: In case of 16 bit data, the SpilbNBuffers value is doubled for the user. For Eg: if the user wishes to transmit five 16 bit data and enters 5 for SpilbNBuffers, the configuration tool would double the value of SpilbNBuffers to ten. In the case of Async Transmission mode: The maximum number of data buffer elements is limited by the capabilities of DMA transaction. For 8 bit channel data width, a maximum of 16383 data buffer elements can be transferred. For 9 to 16 bit channel data width, a maximum of 32767 (2*16383) data buffer elements can be transferred. For 16 to 32 bit channel data width, a maximum of 65535 (4*16383) data buffer elements can be transferred. Note: For a (16 to 32)-bit channels assigned to asynchronous job, maximum of 32767 data buffers should be configured, since DMA implements a circular buffer with a maximum width of 32KB which is used in asynchronous transmission only.		
Scope:	Driver		

Dependency:	SpiChannelType and SpiChannelBuffersAllowed
--------------------	---

4.6.5 SpiEbMaxLength

Table 19 Number of EB Buffers

Syntax:	SpiEbMaxLength		
Source:	This parameter is defined by the AUTOSAR, refer [2]		
File:	Spi.xdm/.bmd		
Range:	Significant if SpiChannelBuffersAllowed is 1 or 2 and SpiChannelType = EB Values: 1..65535 Default value: 511		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	All Variants
SFRs Modified:	None		
Description:	This parameter defines number of data to be transmitted for EB channel. Note: In the case of Async Transmission mode: The maximum number of data buffer elements is limited by the capabilities of DMA transaction. For 8 bit channel data width, a maximum of 16383 data buffer elements can be transferred. For 9 to 16 bit channel data width, a maximum of 32767 (2*16383) data buffer elements can be transferred. For 16 to 32 bit channel data width, a maximum of 65535 (4*16383) data buffer elements can be transferred. Note: For a (16 to 32)-bit channels assigned to asynchronous job, maximum of 32767 data buffers should be configured, since DMA implements a circular buffer with a maximum width of 32KB which is used in asynchronous transmission only.		
Scope:	Driver		
Dependency:	SpiChannelType and SpiChannelBuffersAllowed		

4.6.6 SpiDefaultData

Table 20 SPI Default Data

Syntax:	SpiDefaultData		
Source:	This parameter is defined by the AUTOSAR, refer [2]		
File:	Spi.xdm/.bmd		
Range:	Values: 0 - 4294967295. Default value: 0		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	All Variants
SFRs Modified:	DATAENTRY[0]		

Description:	This is the value that is transmitted when there are no actual values to be transmitted.
Scope:	Driver
Dependency:	SpiDataWidth

4.6.7 SpiChannelId

Table 21 SPI Channel ID

Syntax:	SpiChannelId		
Source:	This parameter is defined by the AUTOSAR, refer [2]		
File:	Spi.xdm/.bmd		
Range:	Values: 0-255. Default value: 0		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	All Variants
SFRs Modified:	None		
Description:	This is ID number assigned to SPI channel. Note: This parameter is disabled in the configuration tool. The ID is generated automatically for each channel by the configuration tool.		
Scope:	Driver		
Dependency:	None		

4.6.8 SpiSafetyBufferMarker

Table 22 SpiSafetyBufferMarker Configuration

Syntax:	SpiSafetyBufferMarker		
Source:	This parameter is microcontroller specific		
File:	Spi.xdm/.bmd		
Range:	Values: 0 to 0xFF Default value: 0xA5		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	All Variants
SFRs Modified:	None		
Description:	This parameter is used as end buffer marker for each channel. It is suggested to configure unique marker values for each channel. For 8/16/32-bit marker, the buffer marker should be at 8/16/32-bit location respectively The requirement for the Buffer marker for channels was that they should be unique, Since there can be maximum 256 channels, the range 0 - 255 suffices for all 8/16/32-bit channels. Even if the buffer width is more, marker width is restricted to 8bit only.		
Scope:	Driver		
Dependency:	SpiSafetyEnable		

4.7 Container: SpiJob

This container contains the configuration parameters to describe a job.

4.7.1 SpiJobPriority

Table 23 SPI Job Priority

Syntax:	SpiJobPriority		
Source:	This parameter is defined by the AUTOSAR, refer [2]		
File:	Spi.xdm/.bmd		
Range:	Values: 0..3 Default value: 0		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	All Variants
SFRs Modified:	None		
Description:	This parameter defines the priority of a job. In the case where some sequences are interruptible and some are non-interruptible, the jobs associated with the non-interruptible sequences have their priority made maximum (priority inversion). The non-interruptible sequences are not interrupted by any other sequences and the jobs will be queued in FIFO, therefore priority inversion would be helpful in queuing the Jobs in Spi_AsyncTransmit function		
Scope:	Driver		
Dependency:	None		

4.7.2 SpiJobEndNotification

Table 24 SPI Job End Notification

Syntax:	SpiJobEndNotification		
Source:	This parameter is defined by the AUTOSAR, refer [2]		
File:	Spi.xdm/.bmd		
Range:	Significant if SpiLevelDelivered is 1 or 2. Value: User defined notification FUNCTION_NAME(Selectable) / Address (Loadable) Default value: NULL_PTR		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	All Variants
SFRs Modified:	None		
Description:	This parameter defines the notification function name. A function by this name must be defined in an application. Note: Callback function shall have no parameters, no return value and shall be re-entrant function.		
Scope:	Driver		
Dependency:	SpiLevelDelivered		

4.7.3 SpiDeviceAssignment

Table 25 SPI External Device assignment

Syntax:	SpiDeviceAssignment		
Source:	This parameter is defined by the AUTOSAR, refer [2]		
File:	Spi.xdm/.bmd		
Range:	Reference to parameter of type SpiExternalDevice. Default value: None		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	All Variants
SFRs Modified:	None		
Description:	This parameter is a reference parameter to the external device container.		
Scope:	Driver		
Dependency:	None		

4.7.4 SpiJobId

Table 26 SPI Job ID

Syntax:	SpiJobId		
Source:	This parameter is defined by the AUTOSAR, refer [2]		
File:	Spi.xdm/.bmd		
Range:	Values: Unsigned value of size Spi_JobType. Default value: 0		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	All Variants
SFRs Modified:	None		
Description:	This ID is assigned to job. Note: This parameter is disabled in the configuration tool. The ID is generated automatically for each job by the configuration tool.		
Scope:	Driver		
Dependency:	None		

4.7.5 SpiHwUnitSynchronous

Table 27 SpiHwUnitSynchronous

Syntax:	SpiHwUnitSynchronous		
Source:	This parameter is defined by the AUTOSAR, refer [2]		
File:	Spi.xdm/.bmd		
Range:	Values: Asynchronous, Synchronous Default value: Asynchronous		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	All Variants
SFRs Modified:	None		
Description:	If SpiHwUnitSynchronous is set to "Synchronous", then the job is used in a synchronous manner and vice versa.		

	Since AS requirement says to preassigned SPI buses required for sync transmission, this parameter is created to preassign the available SPI bus to Sync or Async Refer Section 9.1 (Bugzilla 48763) for more information
Scope:	Driver
Dependency:	Applicable only if SpiLevelDelivered is 2

4.7.6 SpiChannelBasedCS

Table 28 SpiChannelBasedCS Configuration

Syntax:	SpiChannelBasedCS		
Source:	This parameter is microcontroller specific		
File:	Spi.xdm/.bmd		
Range:	Values: Disabled/ Enabled Default value: Disabled		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	All Variants
SFRs Modified:	BACON		
Description:	Enabling will toggle the Chip select after each data of the channel is transmitted. For SpiChannelBasedCS as enabled, If SpiCsSelection is configured as CS_VIA_PERIPHERAL_ENGINE, Chip select will be toggled after each data of the channel is transmitted. However, if SpiCsSelection is configured as CS_VIA_GPIO, then Chip select toggles only before/after the Job transmission. Refer, Table 134 Note: Enabling SpiChannelBasedCS would violate autosar requirement: SPI263.		
Scope:	Driver		
Dependency:	SpiExternalDevice->SpiCsSelection		

4.8 Container: SpiChannelList

This container contains the references to SPI channels and their order within the Job **SpiChannelIndex**

Table 29 SpiChannelIndex

Syntax:	SpiChannelIndex		
Source:	This parameter is defined by the AUTOSAR, refer [2]		
File:	Spi.xdm/.bmd		
Range:	0 – 254		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	All Variants
SFRs Modified:	None		

Description:	This parameter specifies the order of Channels within the Job.
Scope:	Driver
Dependency:	None

4.8.2 SpiChannelAssignment

Table 30 SPI Channel Linking

Syntax:	SpiChannelAssignment		
Source:	This parameter is defined by the AUTOSAR, refer [2]		
File:	Spi.xdm/.bmd		
Range:	Reference to a list of linked channels. Default value: None		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	All Variants
SFRs Modified:	None		
Description:	This parameter should reference to a list of linked channels.		
Scope:	Driver		
Dependency:	None		

4.9 Container: SpiExternalDevice

This container contains the configuration parameters to describe the external device (slave) properties. This container is attached/referenced to a job.

4.9.1 SpiCsIdentifier

Table 31 SPI CS Identifier

Syntax:	SpiCsIdentifier		
Source:	This parameter is defined by the AUTOSAR, refer [2]		
File:	Spi.xdm/.bmd		
Range:	Values: Reference to parameter of type PortPin Default value: None		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	All Variants
SFRs Modified:	None		
Description:	This parameter is the symbolic name to identify the chip select (CS) allocated to this job. A reference to the Port driver should be created to identify this parameter. If the user does not need a CS pin, this parameter need not be configured. Note: This parameter is not used by the Spi Driver.		
Scope:	Driver		
Dependency:	PortPin		

4.9.2 SpiEnableCs

Table 32 SPI Enable CS

Syntax:	SpiEnableCs		
Source:	This parameter is defined by the AUTOSAR, refer [2]		
File:	Spi.xdm/.bmd		
Range:	Values: FALSE – No Chipselect is enabled TRUE – Chipselect is enabled Default: FALSE		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	All Variants
SFRs Modified:	None		
Description:	This parameter handles the chip select handling functions.		
Scope:	Driver		
Dependency:	None		

4.9.3 SpiCsPolarity

Table 33 SPI CS Polarity

Syntax:	SpiCsPolarity		
Source:	This parameter is defined by the AUTOSAR, refer [2]		
File:	Spi.xdm/.bmd		
Range:	Values: LOW – Active Low chipselect HIGH – Active High Chipselect. Default: LOW		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	All Variants
SFRs Modified:	SSOC Pn_OMR (n is the port number)		
Description:	This parameter defines the active polarity of Chip Select		
Scope:	Driver		
Dependency:	SpiEnableCs		

4.9.4 SpiCsSelection

Table 34 SpiCsSelection

Syntax:	SpiCsSelection		
Source:	This parameter is defined by the AUTOSAR, refer [2] .		
File:	Spi.xdm/.bmd		

Range:	Values: CS_VIA_PERIPHERAL_ENGINE – SPI HW IP drives the chip select lines autonomously without explicit chip select control by the driver CS_VIA_GPIO- SPI driver shall drive the chip select lines explicitly as DIO/GPIO. Default: CS_VIA_PERIPHERAL_ENGINE		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	All Variants
SFRs Modified:	None		
Description:	When the Chip select handling is enabled, then this parameter specifies if the chip select is handled automatically by Peripheral HW engine or via general purpose IO by Spi driver. <i>Note: Refer Section 9.1 (Bugzilla 48762) for more information.</i> Also, if CS_VIA_GPIO is selected, then configure sufficient Lead phase delay (SpiClk2CS) to have proper Chip Select assertion.		
Scope:	Driver		
Dependency:	SpiEnableCs		

4.9.5 SpiBaudrate

Table 35 SPI Baudrate

Syntax:	SpiBaudrate		
Source:	This parameter is defined by the AUTOSAR, refer [2]		
File:	Spi.xdm/.bmd		
Range:	Values: 0Hz - 50Mhz Default value: 9600Hz		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	All Variants
SFRs Modified:	ECON		
Description:	This parameter is the communication baudrate. CAUTION: For the maximum baud rate of 50 MBaud, the minimum data width possible is four.		
Scope:	Driver		
Dependency:	Enabled only when the parameter SpiAutoCalcBaudParams is set to TRUE		

4.9.6 SpiTimeClk2CS

Table 36 SPI Time CLK2CS

Syntax:	SpiTimeClk2CS		
Source:	This parameter is defined by the AUTOSAR, refer [2]		
File:	Spi.xdm/.bmd		
Range:	Values: 0 to 0.0001Default value: 0		
Configuration	Pre-Compile	--	--

Class	Link Time	--	--
	Post Build	x	All Variants
SFRs Modified:	BACON		
Description:	This parameter is the minimum time (in seconds) between clock and chip select. This parameter is used to calculate the Qspi hw lead delay parameters LPRE and LEAD.		
Scope:	Driver		
Dependency:	Enabled only when the parameter SpiAutoCalcDelayParams is set to TRUE		

4.9.7 SpiShiftClockIdleLevel

Table 37 Shift Clock Idle Level

Syntax:	SpiShiftClockIdleLevel		
Source:	This parameter is defined by the AUTOSAR, refer [2]		
File:	Spi.xdm/.bmd		
Range:	Values: HIGH - Shift clock idle level is a high voltage level LOW - Shift clock idle level is a low voltage level Default value: LOW		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	All Variants
SFRs Modified:	ECON		
Description:	This parameter defines the idle level of shift clock. The idle level of the shift clock can be configured to be idle level low or idle level high.		
Scope:	Driver		
Dependency:	None		

4.9.8 SpiDataShiftEdge

Table 38 Data Shift Edge

Syntax:	SpiDataShiftEdge		
Source:	This parameter is defined by the AUTOSAR, refer [2]		
File:	Spi.xdm/.bmd		
Range:	Values: LEADING - First data shift edge is a leading edge TRAILING - First data shift edge is a trailing edge Default value: LEADING		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	All Variants
SFRs Modified:	ECON		
Description:	The data can be shifted on either leading edge or on trailing edge of the shift clock. This parameter defines the data shift with leading or trailing edge. NOTE: If slave module is configured to read back data then SpiDataShiftEdge		

	shall be fixed to LEADING edge.
Scope:	Driver
Dependency:	None

4.9.9 SpildleTime

Table 39 SPI Idle Time

Syntax:	SpildleTime		
Source:	This parameter is microcontroller specific		
File:	Spi.xdm/.bmd		
Range:	Values: 0 to x where x depends on the Qspi clock frequency Default value: 1.0E-7		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	All Variants
SFRs Modified:	BACON		
Description:	This parameter IDLEA / IDLEB time (in seconds) defined by QSPI hardware. This parameter is used to calculate the Qspi hw delay parameters IPRE and IDLE.		
Scope:	Driver		
Dependency:	Enabled only when the parameter SpiAutoCalcDelayParams is set to TRUE		

4.9.10 SpiTrailingTime

Table 40 SPI Training Time

Syntax:	SpiTrailingTime		
Source:	This parameter is microcontroller specific		
File:	Spi.xdm/.bmd		
Range:	Values: 0 to x where x depends on the Qspi clock frequency Default value: 1.0E-7		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	All Variants
SFRs Modified:	BACON		
Description:	This parameter Trailing time (in seconds) defined by QSPI hardware. This parameter is used to calculate the Qspi hw delay parameters TPRES and TRAIL.		
Scope:	Driver		
Dependency:	Enabled only when the parameter SpiAutoCalcDelayParams is set to TRUE		

4.9.11 SpiAutoCalcDelayParams

Table 41 SpiAutoCalcDelayParams

Syntax:	SpiAutoCalcDelayParams
Source:	This parameter is microcontroller specific
File:	Spi.xdm/.bmd

Range:	Values: TRUE: Automatically calculate the delay params. FALSE: Manually enter the Delay parameters Default value: FALSE		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	All Variants
SFRs Modified:	None		
Description:	If the parameter is set to TRUE, then the configuration tool will automatically generate the delay parameters (IPRE, IDLE, LPRE, LEAD, TPRES, TRAIL) based on the parameters SpiIdleTime, SpiTimeClk2CS, SpiTrailingTime.		
Scope:	Driver		
Dependency:	None		

4.9.12 SpiAutoCalcBaudParams

Table 42 SpiAutoCalcBaudParams

Syntax:	SpiAutoCalcBaudParams		
Source:	This parameter is microcontroller specific		
File:	Spi.xdm/.bmd		
Range:	Values: TRUE: Automatically calculate the Baudrate params. FALSE: Manually enter the Baudrate parameters Default value: FALSE		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	All Variants
SFRs Modified:	None		
Description:	If the parameter is set to TRUE, then the configuration tool will automatically generate the Baudrate parameters (TQ, Q, A, B, C) based on the parameter SpiBaudrate.		
Scope:	Driver		
Dependency:	None		

4.9.13 SpiQSpiParamTq

Table 43 SpiQSpiParamTq

Syntax:	SpiQSpiParamTq		
Source:	This parameter is microcontroller specific		
File:	Spi.xdm/.bmd		
Range:	Values: 0-255 Default value: 2		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--

	Post Build	x	All Variants
SFRs Modified:	GLOBALCON		
Description:	Global Time Quantum Length Common n-divider scaling the baud rates of all channels in direction of higher or lower baud rates. 0D division by 1 1D division by 2 ... 255D division by 256		
Scope:	Driver		
Dependency:	Enabled only when the parameter SpiAutoCalcBaudParams is set to FALSE		

4.9.14 SpiQSpiParamQ

Table 44 SpiQSpiParamQ

Syntax:	SpiQSpiParamQ		
Source:	This parameter is microcontroller specific		
File:	Spi.xdm/.bmd		
Range:	Values: 0-63 Default value: 10		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	All Variants
SFRs Modified:	ECON		
Description:	This defines the time quantum length used by A, B, and C to define the baud rate and duty cycle 000000B - 1 000001B - 2 111111B - 64		
Scope:	Driver		
Dependency:	Enabled only when the parameter SpiAutoCalcBaudParams is set to FALSE		

4.9.15 SpiQSpiParamA

Table 45 SpiQSpiParamA

Syntax:	SpiQSpiParamA		
Source:	This parameter is microcontroller specific		
File:	Spi.xdm/.bmd		
Range:	Values: 0-3 Default value: 1		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--

	Post Build	x	All Variants
SFRs Modified:	ECON		
Description:	Bit Segment 1 Length expressed in quanta of Q 00B - 1 01B - 2 10B - 3 11B - 4		
Scope:	Driver		
Dependency:	Enabled only when the parameter SpiAutoCalcBaudParams is set to FALSE		

4.9.16 SpiQSpiParamB

Table 46 SpiQSpiParamB

Syntax:	SpiQSpiParamB		
Source:	This parameter is microcontroller specific		
File:	Spi.xdm/.bmd		
Range:	Values: 0-3 Default value: 0		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	All Variants
SFRs Modified:	ECON		
Description:	Bit Segment 2 Length expressed in quanta of Q 00B - 0 01B - 1 10B - 2 11B - 3		
Scope:	Driver		
Dependency:	Enabled only when the parameter SpiAutoCalcBaudParams is set to FALSE		

4.9.17 SpiQSpiParamC

Table 47 SpiQSpiParamC

Syntax:	SpiQSpiParamC		
Source:	This parameter is microcontroller specific		
File:	Spi.xdm/.bmd		
Range:	Values: 0-3 Default value: 1		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	All Variants

SFRs Modified:	ECON
Description:	Bit Segment 3 Length expressed in quanta of Q 00B - 0 (if B=0, than C is minimum 1 per hardware) 01B - 1 10B - 2 11B - 3 Note: If SpiQSpiParamB = 0, then SpiQSpiParamC should have minimum of value 1
Scope:	Driver
Dependency:	Enabled only when the parameter SpiAutoCalcBaudParams is set to FALSE

4.9.18 SpiQSpiParamTPre

Table 48 SpiQSpiParamTPre

Syntax:	SpiQSpiParamTPre		
Source:	This parameter is microcontroller specific		
File:	Spi.xdm/.bmd		
Range:	Values: 0-7 Default value: 0		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	All Variants
SFRs Modified:	BACON		
Description:	Prescaler for the Trailing Delay Length in T_{QSPI} units 000B - 1 001B - 4 010B - 16 ... 111B - 16384		
Scope:	Driver		
Dependency:	Enabled only when the parameter SpiAutoCalcDelayParams is set to FALSE		

4.9.19 SpiQSpiParamTrail

Table 49 SpiQSpiParamTrail

Syntax:	SpiQSpiParamTrail		
Source:	This parameter is microcontroller specific		
File:	Spi.xdm/.bmd		
Range:	Values: 0-7 Default value: 1		
Configuration Class	Pre-Compile	--	--

	Link Time	--	--
	Post Build	x	All Variants
SFRs Modified:	BACON		
Description:	Trailing Delay Length Defines the length of the trailing delay, in T_{QSPI} units pre scaled with TPRES 0D - 1 units 1D - 2 unit ... 7D - 8 units		
Scope:	Driver		
Dependency:	Enabled only when the parameter SpiAutoCalcDelayParams is set to FALSE		

4.9.20 SpiQSpiParamIPre

Table 50 SpiQSpiParamIPre

Syntax:	SpiQSpiParamIPre		
Source:	This parameter is microcontroller specific		
File:	Spi.xdm/.bmd		
Range:	Values: 0-7 Default value: 1		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	All Variants
SFRs Modified:	BACON		
Description:	Prescaler for the Idle Delay Length in T_{QSPI} units 000B - 1 001B - 4 010B - 16 ... 111B - 16384		
Scope:	Driver		
Dependency:	Enabled only when the parameter SpiAutoCalcDelayParams is set to FALSE		

4.9.21 SpiQSpiParamIdle

Table 51 SpiQSpiParamIdle

Syntax:	SpiQSpiParamIdle		
Source:	This parameter is microcontroller specific		
File:	Spi.xdm/.bmd		
Range:	Values: 0-7 Default value: 1		
Configuration Class	Pre-Compile	--	--

	Link Time	--	--
	Post Build	x	All Variants
SFRs Modified:	BACON		
Description:	Idle Delay Length Defines the length of both idle delays, IDLEA and IDLEB, in T_{QSPI} units pre scaled with IPRE 0D - 1 units 1D - 2 unit ... 7D - 8 units		
Scope:	Driver		
Dependency:	Enabled only when the parameter SpiAutoCalcDelayParams is set to FALSE		

4.9.22 SpiQSpiParamLPre

Table 52 SpiQSpiParamLPre

Syntax:	SpiQSpiParamLPre		
Source:	This parameter is microcontroller specific		
File:	Spi.xdm/.bmd		
Range:	Values: 0-7 Default value: 0		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	All Variants
SFRs Modified:	BACON		
Description:	Prescaler for the Leading Delay Length in T_{QSPI} units 000B - 1 001B - 4 010B - 16 ... 111B - 16384		
Scope:	Driver		
Dependency:	Enabled only when the parameter SpiAutoCalcDelayParams is set to FALSE		

4.9.23 SpiQSpiParamLead

Table 53 SpiQSpiParamLead

Syntax:	SpiQSpiParamLead		
Source:	This parameter is microcontroller specific		
File:	Spi.xdm/.bmd		
Range:	Values: 0-7 Default value: 1		

Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	All Variants
SFRs Modified:	BACON		
Description:	Leading Delay Length Defines the length of the leading delay, in T_{QSPI} units pre scaled with LPRE 0D - 1 units 1D - 2 unit ... 7D - 8 units		
Scope:	Driver		
Dependency:	Enabled only when the parameter SpiAutoCalcDelayParams is set to FALSE		

4.9.24 SpiParitySupport

Table 54 SPI Parity Support

Syntax:	SpiParitySupport		
Source:	This parameter is microcontroller specific		
File:	Spi_PBcfg.c		
Range:	Values: Unused Even Odd Default: Unused		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	All Variants
SFRs Modified:	BACON, ECON		
Description:	This parameter handles the Parity of Transmission If this parameter is: Unused – Parity is not appended with data Transmission Even –One Parity Bit is added for Transmission to make the data as Even Parity Odd –One Parity Bit is added for Transmission to make the data as Odd Parity.		
Scope:	Driver		
Dependency:	None		

4.10 Container: SpiHwUnit

4.10.1 SpiAssignedHwModule

Table 55 SpiAssignedHwModule

Syntax:	SpiAssignedHwModule
Source:	This parameter is microcontroller specific

File:	Spi.xdm/.bmd		
Range:	Values:QSPIz Default value: QSPI0 Range of z depends on micro controller derivative. Refer [10.1] for Qspi module availability for specific derivative. Note: for TC298 device the default value is QSPI1		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	All Variants
SFRs Modified:	None		
Description:	This parameter is the symbolic name to identify the QSPI HW unit of the microcontroller.		
Scope:	Driver		
Dependency:	None		

4.10.2 SpiAssignedHwChannel

Table 56 SpiAssignedHwChannel

Syntax:	SpiAssignedHwChannel		
Source:	This parameter is microcontroller specific		
File:	Spi.xdm/.bmd		
Range:	Values:Channel0 - Channel15 Default value: Channel0		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	All Variants
SFRs Modified:	SSOC		
Description:	This parameter is the symbolic name to identify the HW channel no of the QSPI. If SpiBaudrateConfigAtRuntime is disabled, 1) Same Channel of QSPIz cannot be selected more than Once. 2) When Ch(x) is selected Ch(x+8) shall not be selected; and Vice Versa. Range of z depends on micro controller derivative.		
Scope:	Driver		
Dependency:	None		

4.11 Container: SpiCsGpio

4.11.1 SpiCsGpioPortSelection

Table 57 Chip select Gpio Port selection

Syntax:	SpiCsGpioPortSelection
Source:	This parameter is microcontroller specific
File:	Spi.xdm/.bmd

Range:	Values: 0 - 40 Default value: 1 Note : Selection of the Port No. should be as per the availability for a derivative		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	All Variants
SFRs Modified:	None		
Description:	This parameter is port number of the gpio port/pin used for Chip select		
Scope:	Driver		
Dependency:	Enabled only if SpiEnableCs = TRUE and SpiCsSelection = CS_VIA_GPIO		

4.11.2 SpiCsGpioPinSelection

Table 58 Chip select Gpio Pin selection

Syntax:	SpiCsGpioPinSelection		
Source:	This parameter is microcontroller specific		
File:	Spi.xdm/.bmd		
Range:	Values: 0 - 15 Default value: 1 Note : Selection of the Pin No. should be as per the availability for a derivative		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	All Variants
SFRs Modified:	None		
Description:	This parameter is pin number of the gpio port/pin used for Chip select		
Scope:	Driver		
Dependency:	Enabled only if SpiEnableCs = TRUE and SpiCsSelection = CS_VIA_GPIO		

4.12 Container: SpiSequence

This container contains the configuration parameters to describe a sequence.

4.12.1 SpiInterruptibleSequence

Table 59 SPI Sequence Interruptible

Syntax:	SpiInterruptibleSequence
Source:	This parameter is defined by the AUTOSAR, refer [2]
File:	Spi.xdm/.bmd
Range:	Significant only if SpiLevelDelivered is 1 OR 2 and SpiInterruptibleSequenceAllowed is TRUE. Values: TRUE: Sequence could be suspended FALSE: Sequence is not suspended

	Default value: FALSE		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	All Variants
SFRs Modified:	None		
Description:	This parameter provides an option to suspend or not to suspend the sequence by another sequence.		
Scope:	Driver		
Dependency:	SpiLevelDelivered and SpiInterruptibleSequenceAllowed		

4.12.2 SpiSeqEndNotification

Table 60 SPI Sequence End Notification

Syntax:	SpiSeqEndNotification		
Source:	This parameter is defined by the AUTOSAR, refer [2]		
File:	Spi.xdm/.bmd		
Range:	Significant only if SpiLevelDelivered is 1 OR 2 Value: User defined notification FUNCTION_NAME(Selectable) / Address (Loadable) Default value: NULL_PTR		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	All Variants
SFRs Modified:	None		
Description:	This parameter defines the notification function name. A function by this name must be defined in an application. Note: Callback function shall have no parameters, no return value and shall be re-entrant function.		
Scope:	Driver		
Dependency:	SpiLevelDelivered		

4.12.3 SpiSequenceld

Table 61 SPI Sequence ID

Syntax:	SpiSequenceld		
Source:	This parameter is defined by the AUTOSAR, refer [2]		
File:	Spi.xdm/.bmd		
Range:	Values: 0-255. Default value: 0		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	All Variants
SFRs Modified:	None		

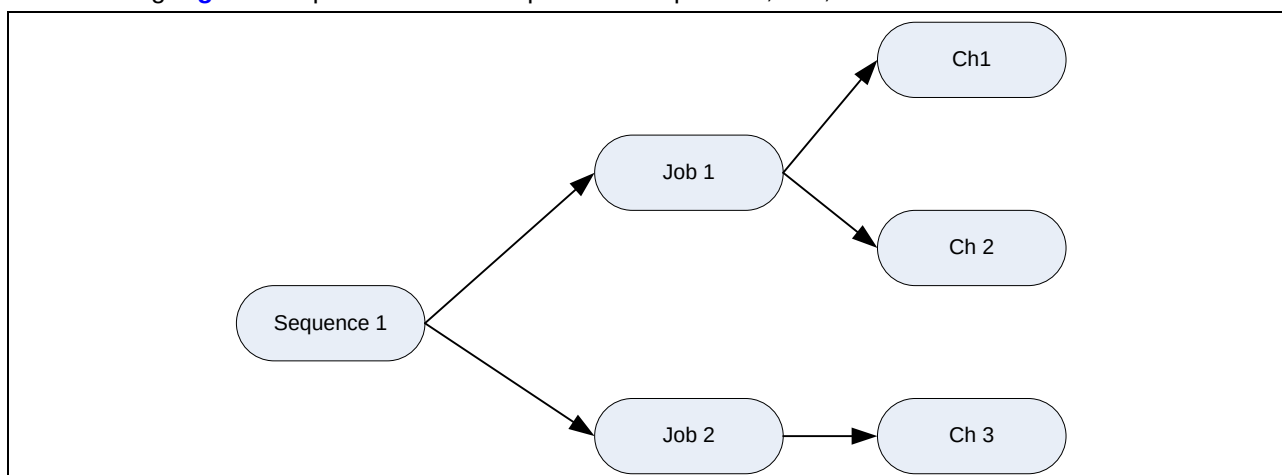
Description:	This ID is assigned to sequence. Note: This parameter is disabled in the configuration tool. The ID is generated automatically for each sequence by the configuration tool.
Scope:	Driver
Dependency:	None

4.12.4 SpiJobAssignment

Table 62 SPI Job Linking

Syntax:	SpiJobAssignment		
Source:	This parameter is defined by the AUTOSAR, refer [2]		
File:	Spi.xdm/.bmd		
Range:	Reference to a list of linked jobs. Default value: None		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	All Variants
SFRs Modified:	None		
Description:	This parameter should reference to a list of jobs. Jobs have priorities assigned. Jobs linked in a Sequence shall have decreasing priorities. That means the first Job shall have the highest priority of all Jobs within the Sequence		
Scope:	Driver		
Dependency:	None		

The following [Figure 4](#) explains the various possible Sequences, Job, Channel links



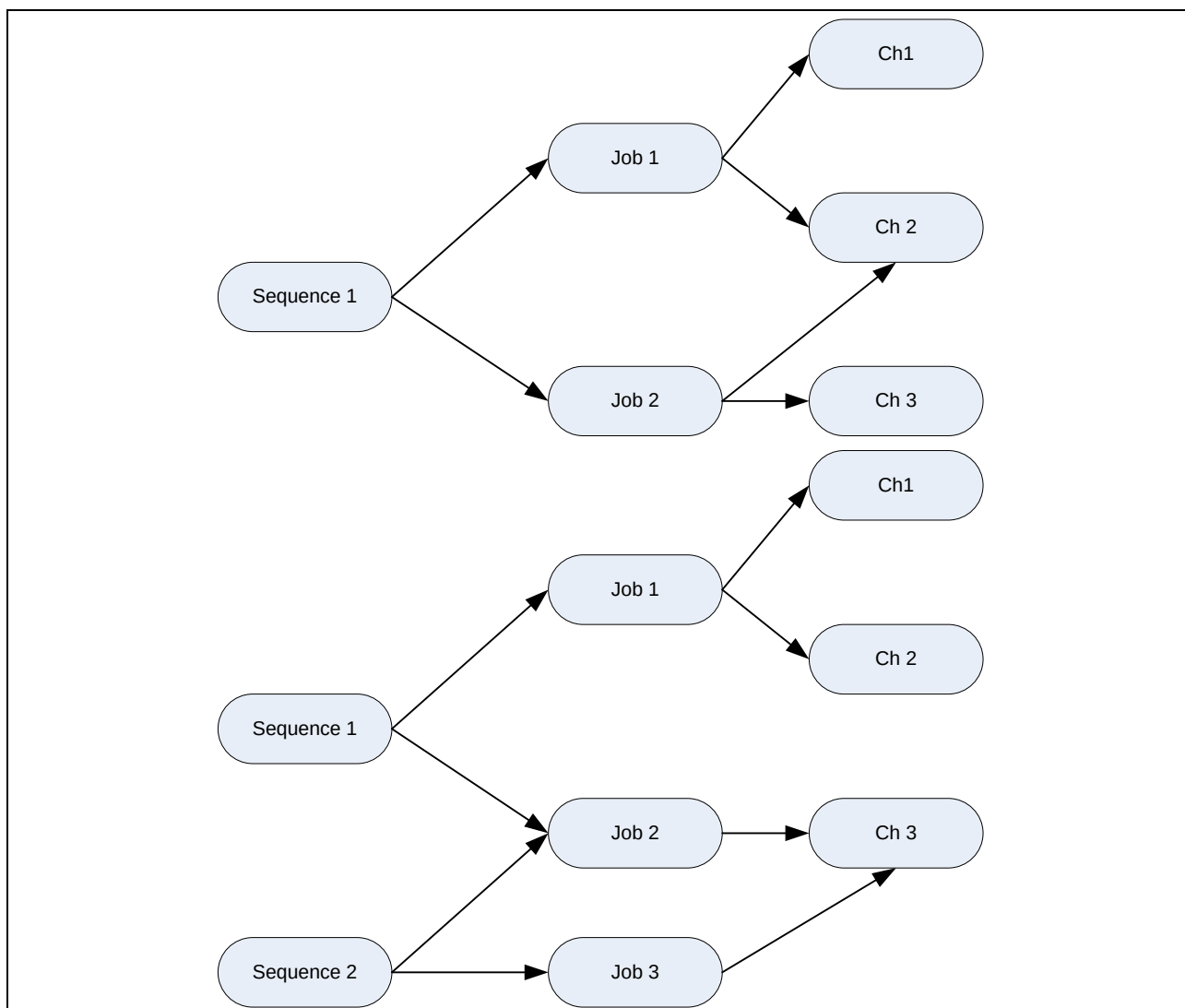


Figure 4 Sequences, Job, Channel links

4.13 Container: SpiHwConfiguration

The container contains the HW specific configuration parameters specific to configure the HW peripherals used to realize the SPI Handler driver.

4.13.1 Container: SpiHardwareConfigurationQspix

4.13.1.1 Container: SpiClockConfiguration

SpiSleepEnable

Table 63 SPI Sleep Enable

Syntax:	SpiSleepEnable
Source:	This parameter is microcontroller specific
File:	Spi.xdm/.bmd
Range:	Values:

	FALSE – QSPIx does not enter sleep mode upon sleep request from MCU. TRUE - QSPIx enters sleep mode upon sleep request from MCU. Default value: FALSE		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	All Variants
SFRs Modified:	None		
Description:	Disable/enable entering into sleep mode upon sleep request from MCU. This parameter is used during spi initialization to configure the CLC register.		
Scope:	Driver		
Dependency:	SpiHwUnit		

4.13.1.2 Container: SpiHwPinConfiguration

SpiHwPinMISO

Table 64 SPI Hw Pin MISO

Syntax:	SpiHwPinMISO		
Source:	This parameter is microcontroller specific		
File:	Spi.xdm/.bmd		
Range:	Values: It depends on micro controller derivative. Refer [10.2] for the MISO values specific to the microcontroller.		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	All Variants
SFRs Modified:	None		
Description:	Port Pin selection for MISO. This is used for the PISEL register MRIS field configuration.		
Scope:	Driver		
Dependency:	None		

SPISlaveHwPinMOSI

Table 65 SPI Slave Hw Pin MOSI

Syntax:	SPISlaveHwPinMOSI		
Source:	This parameter is microcontroller specific		
File:	Spi.xdm/.bmd		
Range:	Values: It depends on micro controller derivative. Refer [10.2] for the MOSI values specific to the microcontroller.		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	All Variants
SFRs Modified:	None		
Description:	Port Pin selection for MOSI. This is used for the PISEL register SRIS field configuration.		
Scope:	Driver		
Dependency:	None		

SPISlaveHwPinSCLK

Table 66 SPI Slave Hw Pin SCLK

Syntax:	SPISlaveHwPinSCLK		
Source:	This parameter is microcontroller specific		
File:	Spi.xdm/.bmd		
Range:	Values: It depends on micro controller derivative. Refer [10.2] for the SCLK values specific to the microcontroller.		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	All Variants
SFRs Modified:	None		
Description:	Port Pin selection for SCLK.		
Scope:	Driver		
Dependency:	None		

4.13.1.3 Container: SpiHwDmaConfiguration

SpiHwDmaChannelReceptionRef

Table 67 SPI Hw Dma Channel Reception Ref

Syntax:	SpiHwDmaChannelReceptionRef		
Source:	This parameter is microcontroller specific		
File:	Spi.xdm/.bmd		
Range:	Reference to parameter of type DmaChannel Default Value: None		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	All Variants
SFRs Modified:	None		
Description:	This parameter refers to the DmaConfiguration configured by MCU driver. DMA channel used for reception shall be provided as reference. If the DMA channel is not significant the reference shall be blank.		
Scope:	Driver		
Dependency:	None		

SpiHwDmaChannelTransmissionRef

Table 68 SPI HW Dma Channel Transmission Ref

Syntax:	SpiHwDmaChannelTransmissionRef		
Source:	This parameter is microcontroller specific		
File:	Spi.xdm/.bmd		
Range:	Reference to parameter of type DmaChannel Default Value: None		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--

	Post Build	x	All Variants
SFRs Modified:	None		
Description:	This parameter refers to the DmaConfiguration configured by MCU driver. DMA channel used for transmission shall be provided as reference. If the DMA channel is not significant the reference shall be blank.		
Scope:	Driver		
Dependency:	None		

4.14 Container: SpiDemEventParameterRefs

This container contains the DEM configuration.

4.14.1 SPI_E_HARDWARE_ERROR

Table 69 DEM for SPI Receive error

Syntax:	SPI_E_HARDWARE_ERROR		
Source:	This parameter is microcontroller specific		
File:	Spi.xdm/.bmd		
Range:	Reference to parameter of type DemEventParameter Default Value: None		
Configuration Class	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
SFRs Modified:	None		
Description:	Reference to the dem event parameter which shall be issued when the "Spi receive error" has occurred.		
Scope:	Driver		
Dependency:	None		

4.14.2 SPI_E_DMA_TRANSFER

Table 70 DEM for SPI DMA Transfer Error

Syntax:	SPI_E_DMA_TRANSFER		
Source:	This parameter is microcontroller specific		
File:	Spi.xdm/.bmd		
Range:	Reference to parameter of type DemEventParameter Default Value: None		
Configuration Class	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
SFRs Modified:	None		
Description:	Reference to the dem event parameter which shall be issued when the "DMA transfer		

	error" has occurred.
Scope:	Driver
Dependency:	None

4.15 Container: SpiPublishedInformation

This container holds all SPI specific published information parameters.

4.15.1 SpiMaxHwUnit

Table 71 SpiMaxHwUnit

Syntax:	SpiMaxHwUnit		
Source:	This parameter is defined by the AUTOSAR, refer [2] .		
File:	Spi.xdm/.bmd		
Range:	Values:x where x depends on the maximum no of QSPI modules available for that derivative. Default value: 4 (for TC27x)		
Configuration Class	Published Information	x	All Variants
SFRs Modified:	None		
Description:	Number of different SPI hardware microcontroller peripherals (units/buses) available and handled by this SPI Handler/Driver module.		
Scope:	Driver		
Dependency:	None		

4.16 Container: CommonPublishedInformation

4.16.1 ArMajorVersion

Table 72 ArMajorVersion

Name:	ArMajorVersion.		
Source:	This parameter is defined by Autosar		
File:	-		
Default:	3		
Range:	0 - 65535		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	--	--
SFRs Modified:	None		
Description:	This parameter provides the major version of the Autosar Specification.		
Scope:	Driver		
Dependency:	None		

4.16.2 ArMinorVersion

Table 73 ArMinorVersion

Name:	ArMinorVersion.		
Source:	This parameter is defined by Autosar		
File:	-		
Default:	2		
Range:	0 - 65535		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	--	--
SFRs Modified:	None		
Description:	This parameter provides the minor version of the Autosar Specification.		
Scope:	Driver		
Dependency:	None		

4.16.3 ArPatchVersion

Table 74 ArPatchVersion

Name:	ArPatchVersion.		
Source:	This parameter is defined by Autosar		
File:	-		
Default:	0		
Range:	0 - 65535		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	--	--
SFRs Modified:	None		
Description:	This parameter provides the patch version of the Autosar Specification.		
Scope:	Driver		
Dependency:	None		

4.16.4 SwMajorVersion

Table 75 SwMajorVersion

Name:	SwMajorVersion.		
Source:	This parameter is defined by Autosar		
File:	-		
Default:	2		
Range:	0 - 65535		
Configuration class	Pre-compile	--	--
	Link Time	--	--

	Post build	--	--
SFRs Modified:	None		
Description:	This parameter provides the major version of the Software Driver		
Scope:	Driver		
Dependency:	None		

4.16.5 SwMinorVersion

Table 76 SwMinorVersion

Name:	SwMinorVersion.		
Source:	This parameter is defined by Autosar		
File:	-		
Default:	1		
Range:	0 - 65535		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	--	--
SFRs Modified:	None		
Description:	This parameter provides the minor version of the Software Driver		
Scope:	Driver		
Dependency:	None		

4.16.6 SwPatchVersion

Table 77 SwPatchVersion

Name:	SwPatchVersion.		
Source:	This parameter is defined by Autosar		
File:	-		
Default:	0		
Range:	0 - 65535		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	--	--
SFRs Modified:	None		
Description:	This parameter provides the patch version of the Software Driver		
Scope:	Driver		
Dependency:	None		

4.16.7 VendorId

Table 78 VendorId

Name:	VendorId
--------------	----------

Source:	This parameter is defined by Autosar		
File:	-		
Default:	17		
Range:	0 - 65535		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	--	--
SFRs Modified:	None		
Description:	This parameter provides the VendorId		
Scope:	Driver		
Dependency:	None		

4.16.8 ModuleId

Table 79 ModuleId

Name:	ModuleId		
Source:	This parameter is defined by Autosar		
File:	-		
Default:	83		
Range:	0 - 65535		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	--	--
SFRs Modified:	None		
Description:	This parameter provides the ModuleId		
Scope:	Driver		
Dependency:	None		

4.16.9 Release

Table 80 Release

Name:	Release		
Source:	This parameter is defined by Autosar		
File:	-		
Default:	_TRICORE_TC277 / _TRICORE_TC275		
Range:	NA		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	--	--
SFRs Modified:	None		
Description:	This parameter provides the Release ID		
Scope:	Driver		

Dependency:	None
-------------	------

4.17 Derived Configuration Parameters

These parameters represent the additional information required by the Driver and automatically calculated by the configuration tool.

*Note: These derived parameters are **precompile** parameters and hence the parameters in the “Dependency” field should not be modified after the build process.*

4.17.1 SPI_MAXIMUM_SEQUENCE

Table 81 SPI Maximum Sequence

Name:	SPI_MAXIMUM_SEQUENCE		
Source:	This is a derived configuration parameter		
File:	Spi_Cfg.h		
Range:	1 – 255		
Configuration Class	Pre-Compile	x	All Variants
	Link Time	-	-
	Post Build	-	-
Description:	SPI Maximum Sequence is the maximum among the total number of sequences configured in each Spi configuration. For example if there are two post build configurations with 2 and 3 sequences respectively, SPI_MAXIMUM_SEQUENCE must be configured as 3. The generated value is between 0 to 254 (Maximum of 255 sequences) and 0xFF is used as delimiter.		
Scope:	Driver		
Dependency:	None		

4.17.2 SPI_MAXIMUM_JOB

Table 82 SPI Maximum Job

Syntax:	SPI_MAXIMUM_JOB		
Source:	This is a derived configuration parameter		
File:	Spi_Cfg.h		
Range:	This parameter is calculated based on the number of Jobs configured, container: SpiJobConfiguration Value: 1 – 65535 Default value: None		
Configuration Class	Pre-Compile	x	All Variants
	Link Time	-	-
	Post Build	-	-
Description:	SPI maximum job is the maximum among the total number of jobs configured in each SPI configuration. For example if there are two post build configurations with 2 and 3 jobs respectively, SPI_MAXIMUM_JOB would be taken as 3.		
Scope:	Driver		

Dependency:	Container: SpiJobConfiguration
--------------------	--------------------------------

4.17.3 SPI_MAXIMUM_CHANNEL

Table 83 SPI Maximum Channel

Syntax:	SPI_MAXIMUM_CHANNEL		
Source:	This is a derived configuration parameter		
File:	Spi_Cfg.h		
Range:	This parameter is calculated based on the number of channel configured, container: SpiChannelConfiguration Value: 1 – 255 Default value: None		
Configuration Class	Pre-Compile	x	All Variants
	Link Time	-	-
	Post Build	-	-
Description:	SPI maximum channel is the maximum among the total number of channels configured in each SPI configuration (This is the sum of both IB and EB channels).		
Scope:	Driver		
Dependency:	Container: SpiJobConfiguration		

4.17.4 SPI_MAXIMUM_IB_BUFFER_SIZE

Table 84 SPI Maximum IB Buffer Size

Name:	SPI_MAXIMUM_IB_BUFFER_SIZE		
Source:	This is a derived configuration parameter		
File:	Spi_Cfg.h		
Range:	A buffer can range between 1 and 65535 i.e.: Sizeof(Spi_NumberOfDataType)		
Configuration Class	Pre-Compile	X	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	It is the maximum among the total number of IB buffer size configured in each Spi configuration. For example if there are two post build configurations, one having 3 IB channels with total buffer size 30 and another having 4 IB channels with total buffer size 40, SPI_MAXIMUM_IB_BUFFER_SIZE must be configured as 40. The IB Buffer is realized by software implementation. This parameter provides the buffer size to be used based on the number of IB Channels configured and their buffer size.		
Scope:	Driver		
Dependency:	None		

4.17.5 SPI_MAXIMUM_EB_CHANNELS

Table 85 SPI Maximum EB Channels

Name:	SPI_MAXIMUM_EB_CHANNELS
--------------	-------------------------

Source:	This is a derived configuration parameter		
File:	Spi_Cfg.h		
Range:	1 - 255		
Configuration Class	Pre-Compile	X	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	It is the maximum among the total number of EB channels configured in each Spi configuration. For example if there are two post build configurations, one having 5 EB channels and another having 2 EB channels, SPI_MAXIMUM_EB_CHANNELS must be configured as 5.		
Scope:	Driver		
Dependency:	None		

4.17.6 SPI_MAXIMUM_IB_CHANNELS

Table 86 SPI Maximum IB Channels

Name:	SPI_MAXIMUM_IB_CHANNELS		
Source:	This is a derived configuration parameter		
Type as per Variant:	Variant PC	#define	
	Variant LT	--	
	Variant PB	#define	
File:	Spi_Cfg.h		
Range:	1 - 255		
Configuration Class	Pre-Compile	X	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	It is the maximum among the total number of IB channels configured in each Spi configuration. For example if there are two post build configurations, one having 3 IB channels and another having 6 IB channels, SPI_MAXIMUM_IB_CHANNELS must be configured as 6.		
Scope:	Driver		
Dependency:	None		

4.17.7 SPI_WRITELOCK_INDEX

Table 87 SPI Write Lock Index

Name:	SPI_WRITELOCK_INDEX		
Source:	This is a derived configuration parameter		
Type as per Variant:	Variant PC	--	
	Variant LT	--	
	Variant PB	#define	
File:	Spi_Cfg.h		
Range:	1 - 32		

Configuration Class	Pre-Compile	X	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	This value defines the size of the array Spi_ChannelWriteLock. This value is calculated based on the number of IB channels configured. Each bit of Spi_ChannelWriteLock variable which is uint8 holds the status of the IB channel. For ex: 10 IB channels configured, the value of SPI_WRITELOCK_INDEX is 2. The formula used $((10 / 8) + 1)$		
Scope:	Driver		
Dependency:	None		

4.17.8 SPI_SEQUENCE_ASIL_ARRAY_INDEX

Table 88 SPI ASIL Sequence Array Index

Name:	SPI_SEQUENCE_ASIL_ARRAY_INDEX		
Source:	This is a derived configuration parameter		
Type as per Variant:	Variant PC	--	
	Variant LT	--	
	Variant PB	#define	
File:	Spi_Cfg.h		
Range:	1 - 16		
Configuration Class	Pre-Compile	X	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	This parameter defines the size of the array/s Spi_AsilSeqStatus, Spi_AsilSeqCancelStatus which hold the status of sequences and cancelled sequences respectively. This value is calculated based on the number of sequences configured for Safety Master Module. Two bits of these array variables are required to hold the status of each sequence. These variables are of type uint32, which holds the status of sixteen sequences. For ex: 35 sequences configured, the value of SPI_SEQUENCE_ASIL_ARRAY_INDEX is 3. The formula used((35 / 16) + 1)		
Scope:	Driver		
Dependency:	None		

4.17.9 SPI_SEQUENCE_QM_ARRAY_INDEX

Table 89 SPI QM Sequence Array Index

Name:	SPI_SEQUENCE_QM_ARRAY_INDEX		
Source:	This is a derived configuration parameter		
Type as per Variant:	Variant PC	--	
	Variant LT	--	
	Variant PB	#define	
File:	Spi_Cfg.h		
Range:	1 - 16		

Configuration Class	Pre-Compile	X	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	This parameter defines the size of the array/s Spi_QmSeqStatus, Spi_QmSeqCancelStatus which hold the status of sequences and cancelled sequences respectively. This value is calculated based on the number of sequences configured for QM Master module/s. Two bits of these array variables are required to hold the status of each sequence. These variables are of type uint32, which holds the status of sixteen sequences. For ex: 35 sequences configured, the value of SPI_SEQUENCE_QM_ARRAY_INDEX is 3. The formula used $((35 / 16) + 1)$		
Scope:	Driver		
Dependency:	None		

4.17.10 SPI_JOB_ASIL_ARRAY_INDEX

Table 90 SPI ASIL Job Array Index

Name:	SPI_JOB_ASIL_ARRAY_INDEX		
Source:	This is a derived configuration parameter		
Type as per Variant:	Variant PC	--	
	Variant LT	--	
	Variant PB	#define	
File:	Spi_Cfg.h		
Range:	1 - 16		
Configuration Class	Pre-Compile	X	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	This value defines the size of the array Spi_AsilJobResult. This value is calculated based on the number of jobs configured for Safety Master Module. Two bits of this array variable are required to hold the status of each job. This variable is of type uint32, which holds the status of sixteen jobs. For ex: 35 jobs configured, the value of SPI_JOB_ASIL_ARRAY_INDEX is 3. The formula used((35 / 16) + 1)		
Scope:	Driver		
Dependency:	None		

4.17.11 SPI_JOB_QM_ARRAY_INDEX

Table 91 SPI QM Job Array Index

Name:	SPI_JOB_QM_ARRAY_INDEX	
Source:	This is a derived configuration parameter	
Type as per Variant:	Variant PC	--
	Variant LT	--
	Variant PB	#define
File:	Spi_Cfg.h	

Range:	1 - 16		
Configuration Class	Pre-Compile	X	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	This value defines the size of the array Spi_QmJobResult. This value is calculated based on the number of jobs configured for QM Master Module/s. Two bits of this array variable are required to hold the status of each job. This variable is of type uint32, which holds the status of sixteen jobs. For ex: 35 jobs configured, the value of SPI_JOB_QM_ARRAY_INDEX is 3. The formula used $((35 / 16) + 1)$		
Scope:	Driver		
Dependency:	None		

4.17.12 SPI_JOB_END_NOTIF_REQ

Table 92 Spi Job End Notification Required

Name:	SPI_JOB_END_NOTIF_REQ		
Source:	This is a derived parameter		
File:	Spi_Cfg.h		
Range:	STD_ON/STD_OFF		
Configuration Class	Pre-Compile	X	Variant PC
	Link Time	X	Variant LT
	Post Build	X	Variant PB
Description:	if STD_ON, Spi Job end notification configured if STD_OFF, Spi Job end notification is not configured.		
Scope:	Driver		
Dependency:	JobEndNotification		

4.17.13 SPI_SEQ_END_NOTIF_REQ

Name:	SPI_SEQ_END_NOTIF_REQ		
Source:	This is a derived parameter		
File:	Spi_Cfg.h		
Range:	TRUE/FALSE		
Configuration Class	Pre-Compile	X	Variant PC
	Link Time	X	Variant LT
	Post Build	X	Variant PB
Description:	if STD_ON, Spi sequence end notification configured if STD_OFF, Spi sequence end notification is not configured.		
Scope:	Driver		
Dependency:	SequenceEndNotification		

4.17.14 SPI_QSPIz_USED

Table 93 SPI QSPIz Module used

Name:	SPI_QSPIz_USED
--------------	----------------

Source:	This is a derived parameter		
Type as per Variant:	Variant PC	#define	
	Variant LT	--	
	Variant PB	#define	
File:	Spi_Cfg.h		
Range:	STD_ON/STD_OFF		
Configuration Class	Pre-Compile	X	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	if STD_ON, QSPIz is used if STD_OFF, QSPIz is not used Where range of z depends on microcontroller derivative. Refer [10.1] If a job configured to use Qspiz hardware, then the corresponding SPI_QSPIz_USED value is set as STD_ON.		
Scope:	Driver		
Dependency:	None		

4.17.15 SPI_QSPIz_MRIS_SEL

Table 94 SPI_QSPIz_MRIS_SEL

Name:	SPI_QSPIz_MRIS_SEL		
Source:	This is a derived parameter		
Type as per Variant:	Variant PC	#define	
	Variant LT	--	
	Variant PB	#define	
File:	Spi_Cfg.h		
Range:	0 - 7		
Configuration Class	Pre-Compile	X	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	This parameter specifies the MRIS (Master Mode Receive Input Select) selection based on MISO port pin #define SPI_QSPIz_MRIS_SEL (m) Where range of z depends on microcontroller derivative, refer [10.1] And range of m may vary from 0 to 7 based on MRST port pin. Refer [10.2] for the MRST pin availability for a particular derivative.		
Scope:	Driver		
Dependency:	This parameter shall be generated if SPI_QSPIz_USED is STD_ON		

4.17.16 SPI_QSPIz_HWTYPE

Table 95 SPI_QSPIz_HWTYPE

Name:	SPI_QSPIz_HWTYPE
Source:	This is a derived parameter

Type as per Variant:	Variant PC	#define	
	Variant LT	--	
	Variant PB	#define	
File:	Spi_Cfg.h		
Range:	SYNC_BUS – Synchronous Bus, ASYNCR_BUS - Asynchronous Bus		
Configuration Class	Pre-Compile	X	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	This parameter specifies if the QSPiZ Hw is used for Synchronous bus or Asynchronous Bus. Where range of z depends on microcontroller derivative, refer [10.1]		
Scope:	Driver		
Dependency:	This parameter shall be generated if SPI_QSPiZ_USED is STD_ON		

4.17.17 SPI_MAX_HW_UNIT

Table 96 SPI_MAX_HW_UNIT

Name:	SPI_MAX_HW_UNIT		
Source:	This is a derived configuration parameter		
File:	Spi_Cfg.h		
Range:	x, x depends on no of QSPI available in the particular microcontroller derivative (refer [10.1] for Qspi module availability for specific derivative.)		
Configuration Class	Pre-Compile	x	All Variants
	Link Time	-	-
	Post Build	-	-
Description:	This parameter specifies the number of HW SPI Modules (i.e. QSPI) available for the corresponding microcontroller derivative.		
Scope:	Driver		
Dependency:	None		

4.17.18 SPI_DEBUG_SUPPORT

Table 97 SPI_DEBUG_SUPPORT

Name:				SPI_DEBUG_SUPPORT			
Source:				This is a derived parameter			
Type as per Variant:				Variant PC		#define	
				Variant LT		--	
				Variant PB		#define	
File:				Spi_Cfg.h			
Range:				STD_ON/STD_OFF			
Configuration Class				Pre-Compile		X	All Variants
				Link Time		--	--
				Post Build		--	--

Description:	This pre-processor macro specifies if the module debug support is on/off. STD_ON – Module Debug is switched ON STD_OFF - Module Debug is switched OFF
Scope:	Driver
Dependency:	SpiDebugSupport

4.17.19 SPI_SUPPORT_CONCURRENT_SYNC_TRANSMIT

Table 98 SPI_SUPPORT_CONCURRENT_SYNC_TRANSMIT

Name:	SPI_SUPPORT_CONCURRENT_SYNC_TRANSMIT		
Source:	This is a derived parameter		
Type as per Variant:	Variant PC	#define	
	Variant LT	--	
	Variant PB	#define	
File:	Spi_Cfg.h		
Range:	STD_ON/STD_OFF		
Configuration Class	Pre-Compile	X	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	This pre-processor macro specifies if the concurrent sync transmit is allowed/not STD_ON – concurrent Spi_SyncTransmit() calls for different sequences allowed STD_OFF - concurrent Spi_SyncTransmit() calls for different sequences Not allowed		
Scope:	Driver		
Dependency:	SpiSupportConcurrentSyncTransmit		

4.17.20 SPI_SYNCTRANSMIT_TIMEOUT_COUNTER

Table 99 SPI_SYNCTRANSMIT_TIMEOUT_COUNTER

Name:	SPI_SYNCTRANSMIT_TIMEOUT_COUNTER		
Source:	This is a derived parameter		
Type as per Variant:	Variant PC	#define	
	Variant LT	--	
	Variant PB	#define	
File:	Spi_Cfg.h		
Range:	0 – 0xFFFFFFFF		
Configuration Class	Pre-Compile	X	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	This macro defines the timeout loop counter while waiting for data transmission/reception of 1 data element during Spi_SyncTransmit API call		
Scope:	Driver		
Dependency:	SpiSyncTransmitTimeoutDuration		

4.17.21 SPI_MASTER_MODULES_USED

Table 100 SPI_MASTER_MODULES_USED

Name:	SPI_MASTER_MODULES_USED		
Source:	This is a derived parameter		
Type as per Variant:	Variant PC	#define	
	Variant LT	--	
	Variant PB	#define	
File:	Spi_Cfg.h		
Range:	1 – 0x3F		
Configuration Class	Pre-Compile	X	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	This pre-processor macro specifies a bitmap of the modules which are configured as Master. Eg: #define SPI_MASTER_MODULES_USED (0x0005U) denotes that QSPI0(bit position 0) and QSPI2(bitposition 2) are configured as master modules.		
Scope:	Driver		
Dependency:	SpiAssignedHwModule,SpiSlaveSelectRef		

4.17.22 SPI_ASIL_MASTER_MODULE_USED

Table 101 SPI_ASIL_MASTER_MODULE_USED

Name:	SPI_ASIL_MASTER_MODULE_USED		
Source:	This is a derived parameter		
Type as per Variant:	Variant PC	#define	
	Variant LT	--	
	Variant PB	#define	
File:	Spi_Cfg.h		
Range:	1 to 32		
Configuration Class	Pre-Compile	X	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	This pre-processor macro specifies a bitmap of the modules which are configured as Master. Eg: #define SPI_ASIL_MASTER_MODULE_USED (0x0004U) denotes that QSPI2(bitposition 2) is configured as Safety master module.		
Scope:	Driver		
Dependency:	SpiSafetyMasterRef		

4.17.23 SPI_NUM_ASIL_MASTER_MODULES

Table 102 SPI_NUM_ASIL_MASTER_MODULES

Name:	SPI_NUM_ASIL_MASTER_MODULES
Source:	This is a derived parameter

Type as per Variant:	Variant PC	#define	
	Variant LT	--	
	Variant PB	#define	
File:	Spi_Cfg.h		
Range:	1		
Configuration Class	Pre-Compile	X	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	This pre-processor macro specifies the number of safety master modules.		
Scope:	Driver		
Dependency:	SpiSafetyEnable		

4.17.24 SPI_NUM_QM_ASYNC_MASTERS

Table 103 SPI_NUM_QM_ASYNC_MASTERS

Name:	SPI_NUM_QM_ASYNC_MASTERS		
Source:	This is a derived parameter		
Type as per Variant:	Variant PC	#define	
	Variant LT	--	
	Variant PB	#define	
File:	Spi_Cfg.h		
Range:	1-n (n is derivate specific)		
Configuration Class	Pre-Compile	X	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	This pre-processor macro specifies the Total number of QSPI modules configured as QM Asynchronous master.		
Scope:	Driver		
Dependency:	SpiAssignedHwModule		

4.17.25 SPI_NUM_ASIL_SLAVE_MODULES

Table 104 SPI_NUM_ASIL_SLAVE_MODULES

Name:	SPI_NUM_ASIL_SLAVE_MODULES		
Source:	This is a derived parameter		
Type as per Variant:	Variant PC	#define	
	Variant LT	--	
	Variant PB	#define	
File:	Spi_Cfg.h		
Range:	1		
Configuration Class	Pre-Compile	X	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	This pre-processor macro specifies the number of safety Slave modules.		

Scope:	Driver
Dependency:	SpiSafetyEnable

4.17.26 SPI_SLAVE_MODULE_INDEX

Table 105 SPI_SLAVE_MODULE_INDEX

Name:	SPI_SLAVE_MODULE_INDEX		
Source:	This is a derived parameter		
Type as per Variant:	Variant PC	#define	
	Variant LT	--	
	Variant PB	#define	
File:	Spi_Cfg.h		
Range:	0-n		
Configuration Class	Pre-Compile	X	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	This pre-processor macro specifies the Index of Slave module. Eg: If QSPI2 is configured as slave then the index would be SPI_QSPI2_INDEX		
Scope:	Driver		
Dependency:	SpiSlaveSelectRef		

4.17.27 SPI_SLAVE_ENABLE

Table 106 SPI_SLAVE_ENABLE

Name:	SPI_SLAVE_ENABLE		
Source:	This is a derived parameter		
Type as per Variant:	Variant PC	#define	
	Variant LT	--	
	Variant PB	#define	
File:	Spi_Cfg.h		
Range:	STD_ON, STD_OFF		
Configuration Class	Pre-Compile	X	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	This pre-processor macro specifies whether to enable slave APIs when safety is ON.		
Scope:	Driver		
Dependency:	SpiSafetyEnable, SpiSlaveSelectRef		

4.17.28 SPI_CONFIG_COUNT

Table 107 SPI_CONFIG_COUNT

Name:	SPI_CONFIG_COUNT	
Source:	This is a derived parameter	
Type as per	Variant PC	#define

Variant:	Variant LT	--	
	Variant PB	#define	
File:	Spi_Cfg.h		
Range:	1 - *		
Configuration Class	Pre-Compile	X	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	This pre-processor macro specifies the number of PB configurations.		
Scope:	Driver		
Dependency:	None		

5 Published Parameters

This section describes the parameters published by the SPI Driver.

5.1 VendorId

Table 108 Vendor ID

Syntax:	SPI_VENDOR_ID
Type:	#define (uint16)
File:	Spi_Cfg.h
Value:	17
Description:	Vendor ID of the dedicated implementation of this Driver according to the AUTOSAR vendor list

5.2 ModuleId

Table 109 Module ID

Syntax:	SPI_MODULE_ID
Type:	#define (uint8)
File:	Spi_Cfg.h
Value:	83 (0x53)
Description:	This macro gives the SPI handler driver ID as described by AUTOSAR : Wp1.1.2 Basic Software Module List

5.3 ARMajorVersion

Table 110 AUTOSAR Major Version

Syntax:	SPI_AR_RELEASE_MAJOR_VERSION
Type:	#define (uint8)
File:	Spi_Cfg.h
Value:	Refer Spi_Cfg.h
Description:	Major version number of AUTOSAR specification for SPI on which the driver implementation is based on.

5.4 ARMinorVersion

Table 111 AUTOSAR Minor Version

Syntax:	SPI_AR_RELEASE_MINOR_VERSION
Type:	#define (uint8)
File:	Spi_Cfg.h
Value:	Refer Spi_Cfg.h
Description:	Minor version number of AUTOSAR specification for SPI on which the driver implementation is based on.

5.5 ARPatchVersion

Table 112 AUTOSAR Revision Version

Syntax:	SPI_AR_RELEASE_REVISION_VERSION
Type:	#define (uint8)
File:	Spi_Cfg.h
Value:	Refer Spi_Cfg.h
Description:	Patch version number of AUTOSAR specification for SPI on which the driver implementation is based on.

5.6 SwMajorVersion

Table 113 Software Major Version

Syntax:	SPI_SW_MAJOR_VERSION
Type:	#define (uint8)
File:	Spi_Cfg.h
Value:	Refer Spi_Cfg.h
Description:	Vendor specific version number of the SPI handler driver

5.7 SwMinorVersion

Table 114 Software Minor Version

Syntax:	SPI_SW_MINOR_VERSION
Type:	#define (uint8)
File:	Spi_Cfg.h
Value:	Refer Spi_Cfg.h
Description:	Vendor specific version number of the SPI handler driver

5.8 SwPatchVersion

Table 115 Software Patch Version

Syntax:	SPI_SW_PATCH_VERSION
Type:	#define (uint8)
File:	Spi_Cfg.h
Value:	Refer Spi_Cfg.h
Description:	Vendor specific version number of the SPI handler driver

5.9 VendorApilInfix

Table 116 VendorApilInfix

Syntax:	VendorApilInfix
File:	Spi.xdm/.bmd
Value:	NA

Description:	Vendor Api infix of the driver. The parameter is empty in the configuration as the Spi driver doesn't have vendor Api infix.
---------------------	--

5.10 Release

Table 117 Release

Syntax:	Release
File:	Spi.xdm/.bmd
Value:	_TRICORE_TC275 for TC275 _TRICORE_TC277 for TC277
Description:	Aurix derivative used for the implementation

6 API Documentation

The API documentation describes the syntax and semantics of the API functions as well as it gives some additional implementation specific behavior description.

6.1 API Type Definitions

This section describes all the type definitions that are used by APIs.

6.1.1 Type Definition Spi_ConfigType

Table 118 Type Definition Spi_ConfigType

Syntax:	Spi_ConfigType
Type:	Struct
File:	Spi.h
Range:	This configuration structure contains the initialization data structure of the SPI. This structure contains configuration of SPI channels, SPI jobs, SPI sequences, SPI HW configuration.
Description:	This type definition is used to configure overall SPI configuration. The pointer to the object of this type is passed via Spi_Init API to initialize the SPI Handler Driver.

6.1.2 Type Definition Spi_StatusType

Table 119 Type Definition Spi_StatusType

Syntax:	Spi_StatusType
Type:	Enumeration
File:	Spi.h
Range:	SPI_UNINIT : The SPI Handler/Driver is not initialized or not usable SPI_IDLE : The SPI Handler/Driver is not currently transmitting any jobs. SPI_BUSY : The SPI Handler/Driver is performing a SPI job (transmit).
Description:	This type defines a range of specific status for SPI Handler/Driver. It informs about the SPI Handler/Driver status and can be obtained calling the API service Spi_GetStatus or the configurable Spi_GetHWUnitStatus.

6.1.3 Type Definition Spi_JobResultType

Table 120 Type Definition Spi_JobResultType

Syntax:	Spi_JobResultType
Type:	Enumeration
File:	Spi.h

Range:	SPI_JOB_OK: The last transmission of the job has been finished successfully. SPI_JOB_PENDING: The SPI Handler/Driver is performing a specific SPI job. The meaning of this status is equal to SPI_BUSY. SPI_JOB_FAILED: The last transmission of the job has failed. SPI_JOB_QUEUED: An asynchronous transmission Job has been accepted, while actual transmission for this Job has not started yet.
Description:	This type defines a range of specific Jobs status for SPI Handler/Driver. It informs about a SPI Handler/Driver job status and can be obtained calling the API service Spi_GetJobResult with the job ID.

6.1.4 Type Definition Spi_SeqResultType

Table 121 Type Definition Spi_SeqResultType

Syntax:	Spi_SeqResultType
Type:	Enumeration
File:	Spi.h
Range:	SPI_SEQ_OK: The last transmission of the sequence has been finished successfully. SPI_SEQ_PENDING: The SPI Handler/Driver is performing a specific SPI sequence. The meaning of this status is equal to SPI_BUSY. SPI_SEQ_FAILED: The last transmission of the sequence has failed. SPI_SEQ_CANCELED: The last transmission of the sequence has been cancelled by user
Description:	This type defines a range of specific sequences status for SPI Handler/Driver. It informs about a SPI Handler/Driver sequence status and can be obtained calling the API service Spi_GetSequenceResult with the sequence ID.

6.1.5 Type Definition Spi_DataType

Table 122 Type Definition spi_Datatype

Syntax:	Spi_DataType
Type:	uint8
File:	Spi.h
Range:	The buffer element size for the IB Buffer and EB buffer are of size: uint8. Note: This does not prevent the user from defining a channel data width for a transfer from 2 to 16bits. These are handled internally according to the following rules Channel Data Width 8: Straightforward send and receive. Channel Data Width <8: Send the lower part, ignore the upper part. Receive the lower part, extend with zero. Channel Data Width >8 and upto 16: In this case 2 buffer elements are written to or read from. Channel Data Width >16: In this case 4 buffer elements are written to or read from.

	Type uint8 is used to save RAM in Driver and application for 8-bit data transfer. For 16-bit transfer (Channel data width >8) user data have to be word aligned.
Description:	Type of application data buffer elements. Note: If channel width > 8 and <= 16, the SPI driver uses type uint16 to read or write the buffer. If Channel width > 16, the SPI driver uses type uint32 to read or write the buffer. The SPI driver will access the buffer as per little endian format (in accordance to architecture).

6.1.6 Type Definition Spi_NumberOfDataType

Table 123 Type Definition Spi_NumberOfDataType

Syntax:	Spi_NumberOfDataType
Type:	uint16
File:	Spi.h
Range:	Refer Parameters : SpiIBNBuffers SpiEBMaxLength
Description:	Type for defining the number of data elements of the type Spi_DataType to send and / or receive by channel.

6.1.7 Type Definition Spi_ChannelType

Table 124 Type Definition Spi_Channeltype

Syntax:	Spi_ChannelType
Type:	uint8
File:	Spi.h
Range:	Channels 0 to 255.
Description:	Specifies the identification (ID) for a Channel.

6.1.8 Type Definition Spi_JobType

Table 125 Type Definition Spi_JobType

Syntax:	Spi_JobType
Type:	uint8
File:	Spi.h
Range:	Jobs 0 to 65535 can be defined.
Description:	Specifies the identification (ID) for a job.

6.1.9 Type Definition Spi_SequenceType

Table 126 Type Definition Spi_SequenceType

Syntax:	Spi_SequenceType
Type:	uint8

File:	Spi.h
Range:	0 to 255 Sequences can be defined.
Description:	Specifies the identification (ID) for a sequence.

6.1.10 Type Definition Spi_HWUnitType

Table 127 Type Definition Spi_HWUnitType

Syntax:	Spi_HWUnitType
Type:	uint8
File:	Spi.h
Range:	Bit[0:3]: Module no - QSPIx where the range of x depends on the microcontroller derivative. Bit[4:7] : Channel no - Channel0-channel15
Description:	Specifies the module and channel no for a SPI hardware microcontroller peripheral.

6.1.11 Type Definition Spi_AsyncModeType

Table 128 Type Definition Spi_AsyncModeType

Syntax:	Spi_AsyncModeType
Type:	Enumeration
File:	Spi.h
Range:	SPI_POLLING_MODE: The asynchronous mechanism is ensured by polling, so interrupts related to the SPI busses handled asynchronously are disabled. SPI_INTERRUPT_MODE: The asynchronous mechanism is ensured by interrupts, so interrupts related to the SPI busses handled asynchronously are enabled.
Description:	Specifies the asynchronous mechanism mode for SPI busses handled asynchronously in Level2. This type is available or not according to the pre compile time parameter "SPI_LEVEL_DELIVERED". This is only relevant for LEVEL 2.

6.2 API Functions

This chapter describes the API interfaces provided by SPI Handler Driver.

6.2.1 Spi_Init

Table 129 Specification of API Spi_Init

Service name:	Spi_Init
Syntax:	void Spi_Init(const Spi_ConfigType* ConfigPtr)
Service ID:	0x00
Sync/ Async:	Synchronous
Origin	Autosar
Re-entrancy:	non re-entrant

Parameters (in):	ConfigPtr	Pointer to configuration set
Parameters (out):	None	
Return value:	None	
Description:	Service for SPI initialization. The initialization function initializes all SPI relevant registers with the values of the structure referenced by the parameter ConfigPtr. The Init function initializes all buffer pointers to a null value pointer. After having finished the initialization, the SPI Handler/Driver state is set to SPI_IDLE, set the sequences result to SPI_SEQ_OK and the jobs result is set to SPI_JOB_OK. For LEVEL 2, by default the init function sets the SPI Handler/Driver asynchronous mechanism mode to SPI_POLLING_MODE. Interrupts related to SPI buses are disabled. Attention: QSPI and it's related DMA interrupt priorities (SRPN) and type of service (TOS) should be initialized before calling this function. And later on QSPI interrupt related TOS/SRPN should not be updated by calling IrqSpi_Init().	
Caveats:	None	
Configuration:	None	
DET:	SPI_E_PARAM_CONFIG	Configuration Pointer is NULL.
	SPI_E_ALREADY_INITIALIZED	SPI already initilized
SAFE MCAL ERROR	Refer Safety errors [Table 163] Safety marker check is provided in Spi_Init. The marker value is generated as the first element of Config Root and will be checked to ensure valid initialization, when SAFETY is enabled. The value of safety marker is generated based on SPI_MODULE_ID and SPI_INSTANCE_ID	
DEM:	None	
Implementation Comments:	TXFM = single move mode	
I/O Mode:	This API can be invoked when the CPU is in User-0 Mode, provided configuration parameter SpiRunningInUser0Mode is 'True' and SpiUserModeInitApiEnable is 'True'. This API can be invoked when the CPU is in User-1 Mode, provided configuration parameter SpiRunningInUser0Mode is 'False' and SpiUserModeInitApiEnable is 'True'. This API can be invoked when the CPU is in Supervisor Mode, provided configuration parameter SpiRunningInUser0Mode is 'False' and SpiUserModeInitApiEnable is 'False'..	

6.2.2 Spi_InitCheck

Table 130 Specification of API Spi_InitCheck

Service name:	Spi_InitCheck
Syntax:	Std_ReturnType Spi_InitCheck (const Spi_ConfigType* ConfigPtr)
Service ID:	NA
Sync/ Async:	Synchronous
Origin	Vendor specific

Re-entrancy:	Non re-entrant	
Parameters (in):	ConfigPtr	Pointer to configuration set
Parameters (out):	None	
Return value:	E_OK	Initialization comparison successful.
	E_NOT_OK	Initialization comparison Incorrect.
Description:	Service for SPI initialization Check. It will check against all SFRs or variables initialized by Spi_Init() It will check if INTERRUPT mode is set by Spi_SetAsyncMode API If any failure in comparison, it reports E_NOT_OK	
Caveats:	None	
Configuration:	None	
DET:	None	
SAFE MCAL ERROR	None	
DEM:	None	
Implementation Comments:	None	
I/O Mode:	This API can be invoked when the CPU is in User-0 Mode, provided configuration parameter SpiRunningInUser0Mode is 'True' and SpiUserModelInitApiEnable is 'True'. This API can be invoked when the CPU is in User-1 Mode, provided configuration parameter SpiRunningInUser0Mode is 'False' and SpiUserModelInitApiEnable is 'True'. This API can be invoked when the CPU is in Supervisor Mode, provided configuration parameter SpiRunningInUser0Mode is 'False' and SpiUserModelInitApiEnable is 'False'..	

6.2.3 Spi_DeInit

Table 131 Specification of Spi_DeInit

Service name:	Spi_DeInit	
Syntax:	Std_ReturnType Spi_DeInit(void)	
Service ID:	0x01	
Sync/ Async:	Synchronous	
Origin	Autosar	
Re-entrancy:	Non Re-entrant	
Parameters (in):	None	
Parameters (out):	None	
Return value:	E_OK	Deinit Command was accepted.
	E_NOT_OK	Deinit Command was not accepted.
Description:	Service for SPI de-initialization. In case of a SPI_BUSY state, command is rejected, otherwise the deinitialization function will set all SPI hardware in reset mode. After having finished the de-initialization, the SPI Handler/Driver state is set to SPI_UNINIT.	
Caveats:	None	
Configuration:	All ports and port pins shall be configured by the configuration tool.	
DET:	SPI_E_UNINIT	SPI is not initialized.
DEM:	None	

SAFE MCAL ERROR	Refer Safety errors [Table 163]
Implementation Comments:	The common resources which are assumed to be used by other peripherals are not deinitialized.
I/O Mode:	This API can be invoked when the CPU is in User-0 Mode, provided configuration parameter SpiRunningInUser0Mode is 'True' and SpiUserModelInitApiEnable is 'True'. This API can be invoked when the CPU is in User-1 Mode, provided configuration parameter SpiRunningInUser0Mode is 'False' and SpiUserModelInitApiEnable is 'True'. This API can be invoked when the CPU is in Supervisor Mode, provided configuration parameter SpiRunningInUser0Mode is 'False' and SpiUserModelInitApiEnable is 'False'..

6.2.4 Spi_WriteIB

Table 132 Specification of Spi_WriteIB

Service name:	Spi_WriteIB	
Syntax:	<pre>Std_ReturnType Spi_Write (Spi_ChannelType Channel, const Spi_DataType* DataBufferPtr)</pre>	
Service ID:	0x02	
Sync/ Async:	Synchronous	
Origin	Autosar	
Re-entrancy:	Re-entrant	
Parameters (in):	Channel	Channel ID.
	DataBufferPtr	Pointer to source data buffer. If this pointer is null, it is assumed that the data to be transmitted is not relevant and the default transmit value of this channel will be used instead.
Parameters (out):	None	
Return value:	E_OK	Write Command was accepted.
	E_NOT_OK	Write Command was not accepted.
Description:	Service for writing one or more data to an IB SPI Handler/Driver channel specified by parameter. This service take over the given parameters, and save the pointed data to the internal buffer defined with Spi_Init(). Different Jobs, consequently also sequences, can have in common channels. But, the use of those shared channels is restricted. Read and write functions cannot guarantee the data integrity while data is transmitted. User must ensure that read and/or write functions are not called during transmission.	
Caveats:	The SPI Handler/Driver shall have been initialized before this service is	

	called. This service is only IB SPI Handler/Driver channel relevant.	
Configuration:	This function is pre compile time configurable by the parameter: SPI_CHANNEL_BUFFERS_ALLOWED. This service is only relevant for channels with IB.	
DET:	SPI_E_UNINIT	SPI is not initialized.
	SPI_E_PARAM_CHANNEL	Channel is not a valid IB channel
DEM:	None	
SAFE MCAL ERROR	Refer Safety errors [Table 163] Safety channel buffer marker check is provided in channel based APIs. The buffer marker value is generated as the last element of Channel Configuration Structure. This marker value will be checked to ensure valid channel configuration, when SAFETY is enabled. The value of safety marker is generated based on the configuration parameter, SpiSafetyBufferMarker. There will be only one buffer marker for a Data Buffer of any length per channel. Safety buffer marker is available irrespective of ASIL/QM channels	
Implementation Comments:	In case of 16bit channel data width the DataBufferPtr must be 16bit aligned. Similarly in case of 32 bit channel data, the DataBufferPtr must be 32 bit. Use the Memory Map Section: SPI_START_SEC_VAR_DMA_ACCESS for this purpose.	
I/O Mode:	This API can be invoked when the CPU is in User-0 Mode, provided configuration parameter SpiRunningInUser0Mode is 'True' and SpiUserModeInitApiEnable is 'True'. This API can be invoked when the CPU is in User-1 Mode, provided configuration parameter SpiRunningInUser0Mode is 'False' and SpiUserModeInitApiEnable is 'True'. This API can be invoked when the CPU is in Supervisor Mode, provided configuration parameter SpiRunningInUser0Mode is 'False' and SpiUserModeInitApiEnable is 'False'..	

6.2.5 Spi_AsyncTransmit

Table 133 Specification of Spi_AsyncTransmit

Service name:	Spi_AsyncTransmit	
Syntax:	Std_ReturnType Spi_AsyncTransmit (Spi_SequenceType Sequence)	
Service ID:	0x03	
Sync/ Async:	Asynchronous	
Origin	Autosar	
Re-entrancy:	Re-entrant	
Parameters (in):	Sequence	Sequence ID
Parameters (out):	None	
Return value:	E_OK	Transmit command was accepted.
	E_NOT_OK	Transmit command was not accepted.
Description:	Service to transmit data on the SPI bus. This service take over the given parameter, initiate a transmission, and set the SPI Handler/Driver status to SPI_BUSY, set the sequence result to SPI_SEQ_PENDING and return. Jobs' results (first Job of the Sequence included) are handled by the SPI	

	Handler/Driver when transmission Jobs are really started (result set to SPI_JOB_PENDING) and/or ended (set either to SPI_JOB_OK or SPI_JOB_FAILED) but none directly within this service. If the requested sequence is already in the state SPI_SEQ_PENDING the SPI Handler/Driver does not take in account this new request and this service returns value E_NOT_OK. The SPI Handler/Driver reports SPI_E_SEQ_PENDING error. If the requested sequence shares Jobs with another sequence that is in the state SPI_SEQ_PENDING, the SPI Handler/Driver does not take in account this new request and this service returns value E_NOT_OK. The SPI Handler/Driver reports the SPI_E_SEQ_PENDING error. With EB, when source data pointer has been provided as null using the Spi_SetupEB method the default transmit data configured for each channel will be transmitted. With EB, when destination data pointer has been provided as null using the Spi_SetupEB method the receiving data is ignored. When a transmission is requested for a sequence with linked Jobs, it will transmit from the first Job up to the last Job in the sequence. At the end of a sequence transmission, if configured, the sequence notification call-back function is executed after the last Job end notification if this one is also configured.	
Caveats:	The SPI should be initialized before this service is called. This method is called after a Spi_SetupEB method for EB channels or Spi_WriteIB method for IB channels but before the Spi_ReadIB method. For an asynchronous transmission with SpiChannelBasedCS Disabled, there would be a small delay before the transmission of last datawidth element of last SpiChannel assigned to a SpiJob	
Configuration:	This function is pre compile time selectable by the configuration parameter: SPI_LEVEL_DELIVERED. This function is only relevant for LEVEL 1 and LEVEL 2.	
DET:	SPI_E_UNINIT	SPI is not initialized.
	SPI_E_PARAM_SEQ	Channel not an IB channel and is not within 0 to SPI_MAX_CHANNEL
	SPI_E_SEQ_PENDING	The requested sequence is already pending. Another sequence sharing a job with this sequence is in pending state.
DEM:	SPI_E_HARDWARE_ERROR	HW Specific Error. A receive error is detected when a new data frame is completely received but the previous data was not read out of the receive buffer register.
	SPI_E_DMA_TRANSFER	DMA transfer error has occurred.
SAFE MCAL ERROR	Refer Safety errors [Table 163]	
Implementation Comments:	The Dem error Ids are defined in Dem_PBcfg.h.	

	If the buses are free, the first job of the sequence is started in this function and the job and sequence result are set to pending.
I/O Mode:	This API can be invoked when the CPU is in User-0 Mode, provided configuration parameter SpiRunningInUser0Mode is 'True' and SpiUserModeInitApiEnable is 'True'. This API can be invoked when the CPU is in User-1 Mode, provided configuration parameter SpiRunningInUser0Mode is 'False' and SpiUserModeInitApiEnable is 'True'. This API can be invoked when the CPU is in Supervisor Mode, provided configuration parameter SpiRunningInUser0Mode is 'False' and SpiUserModeInitApiEnable is 'False'..

Handling Notification functions

If jobs are scheduled across SPI buses, then some minor care should be taken with respect to the execution time of notification function. There is no flaw within driver execution however the order of return of notification function might vary. As an example Refer [Figure 5](#)

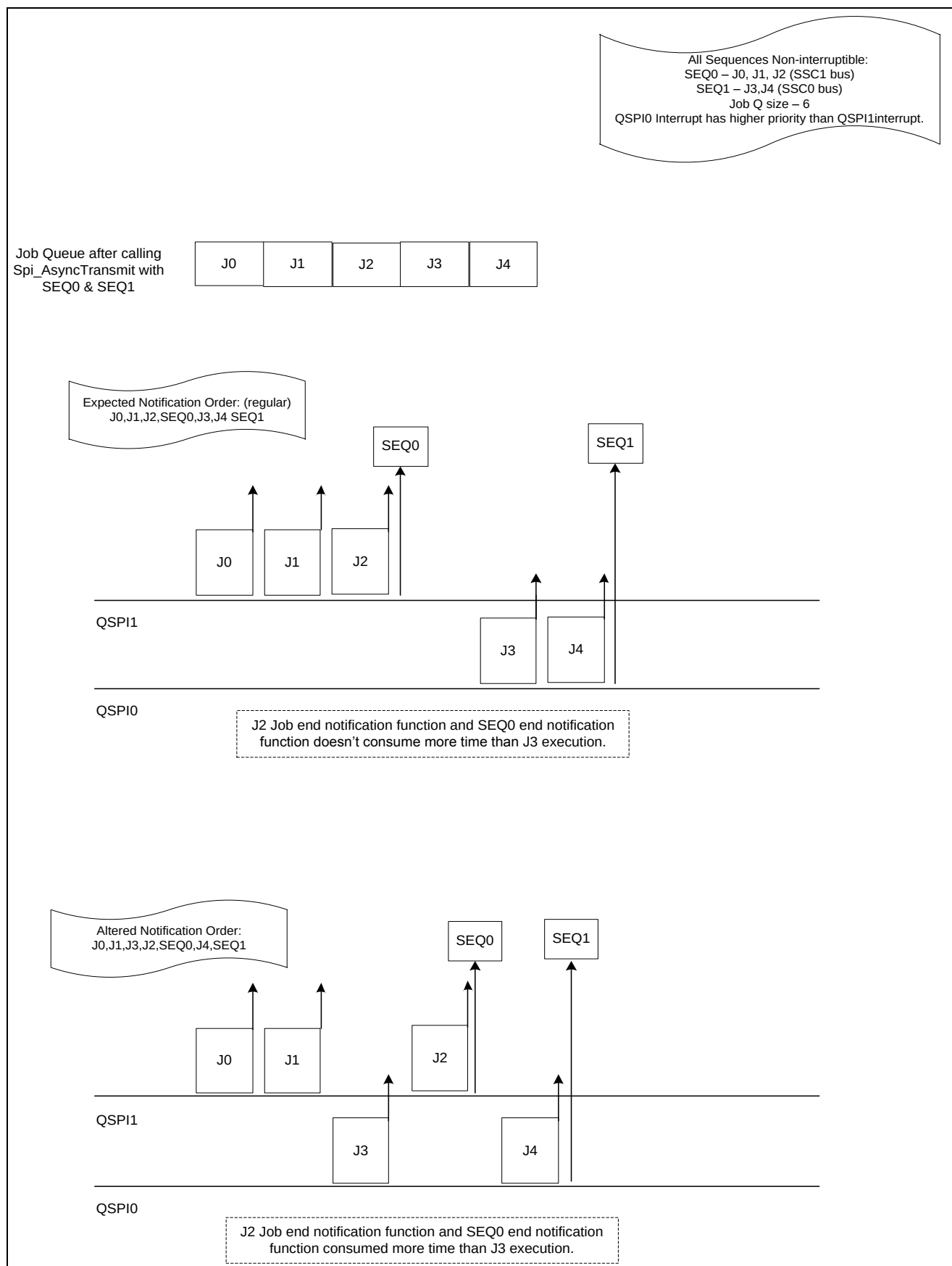


Figure 5 Handling Notification functions

fQspi = 60MHZ, SPI Jobs at Baudrate = 1Mbps and Job J3 (contains only one channel) transmits 20bits.

Considering above settings, J3 takes 20 (20 bits * (1/1M)) micro seconds (ISR execution time and other overheads not considered) for completing the job.

If J2 End Notification function consumes more time than J3 execution, then end notification functions are not returned in same order as fired. Refer Altered Notification Order of [Figure 5](#)

Note: It is suggested to have EndNotification callback function which consumes less time than execution time of on-going job.

Ordering of Jobs in Queue

Refer [Figure 6](#) for an example of interruptible sequences ordered in a queue according to priority

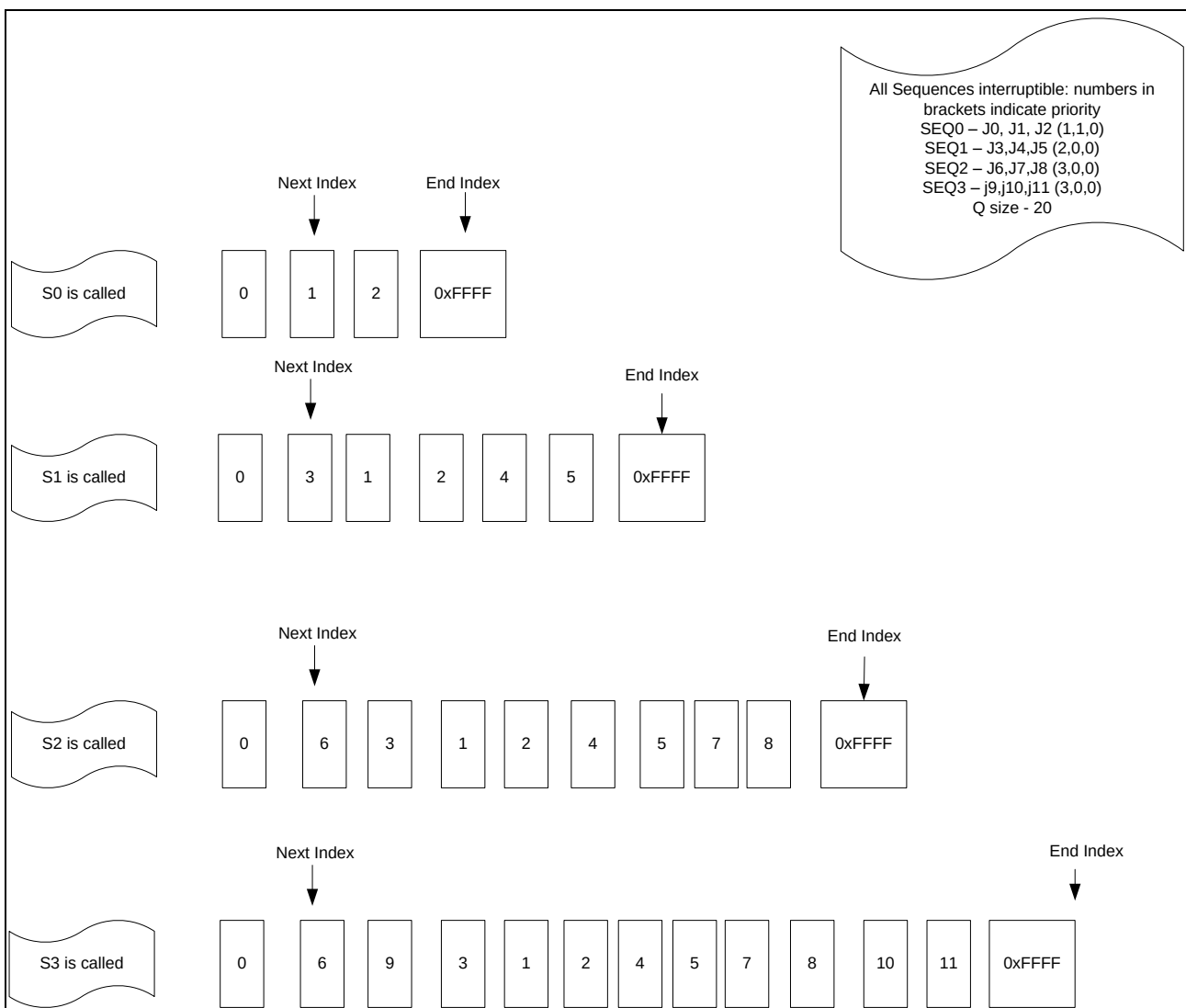


Figure 6 Use case 1: representing the interruptible sequence

In the case where some sequences are interruptible and some are non-interruptible, the jobs associated with the non-interruptible sequences have their priority made maximum (priority inversion). The non-interruptible sequences are not interrupted by any other sequences and the jobs will be queued in FIFO, therefore priority inversion would be helpful in queueing the Jobs in Spi_AsyncTransmit function. Refer [Figure 7](#) for an example of interruptible and non-interruptible sequences ordered in a queue

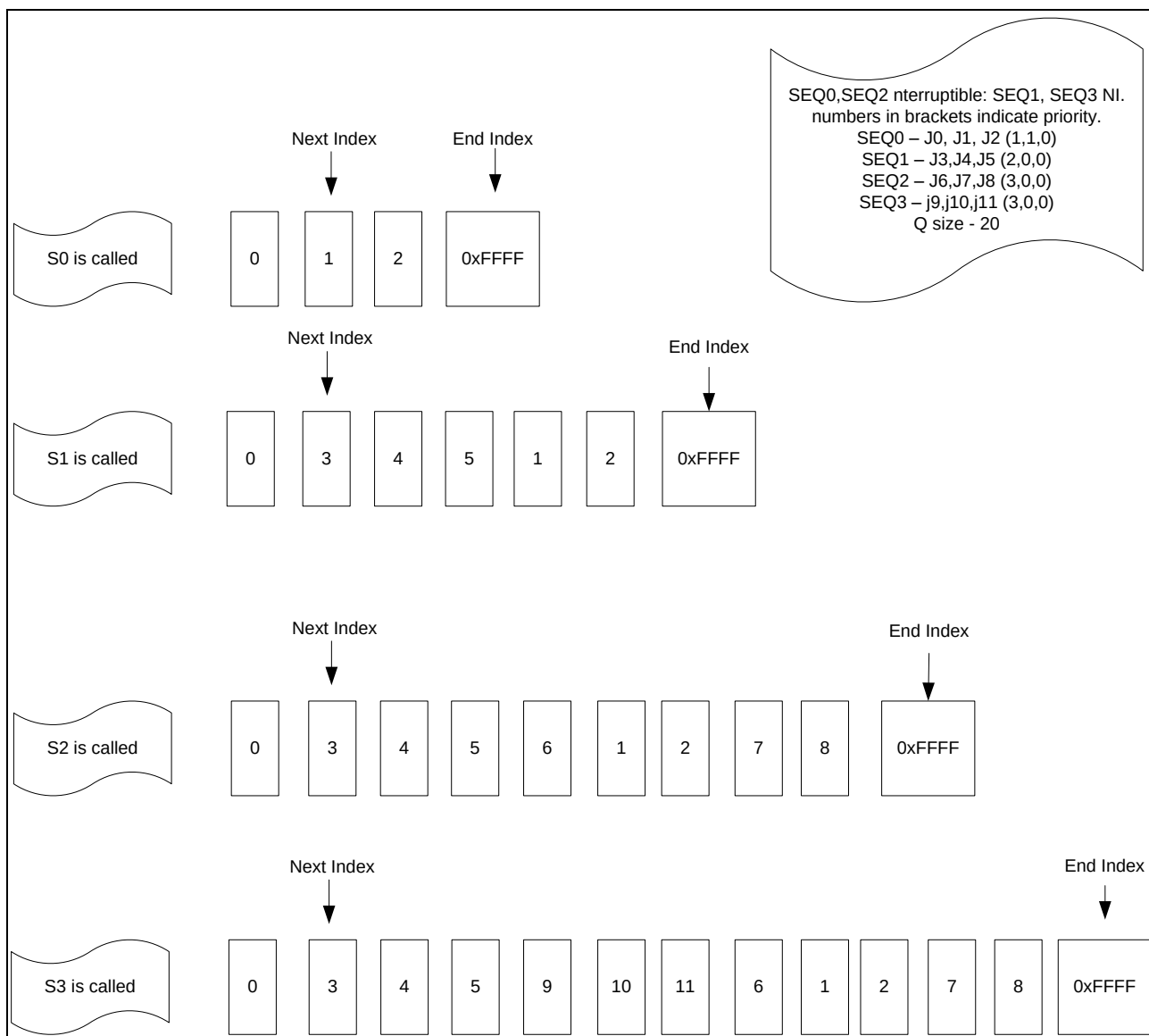


Figure 7 Use case 2: representing the interruptible and non-interruptible sequence

The below figure represents the Difference between SpiAsyncParallelTransmit ON/OFF

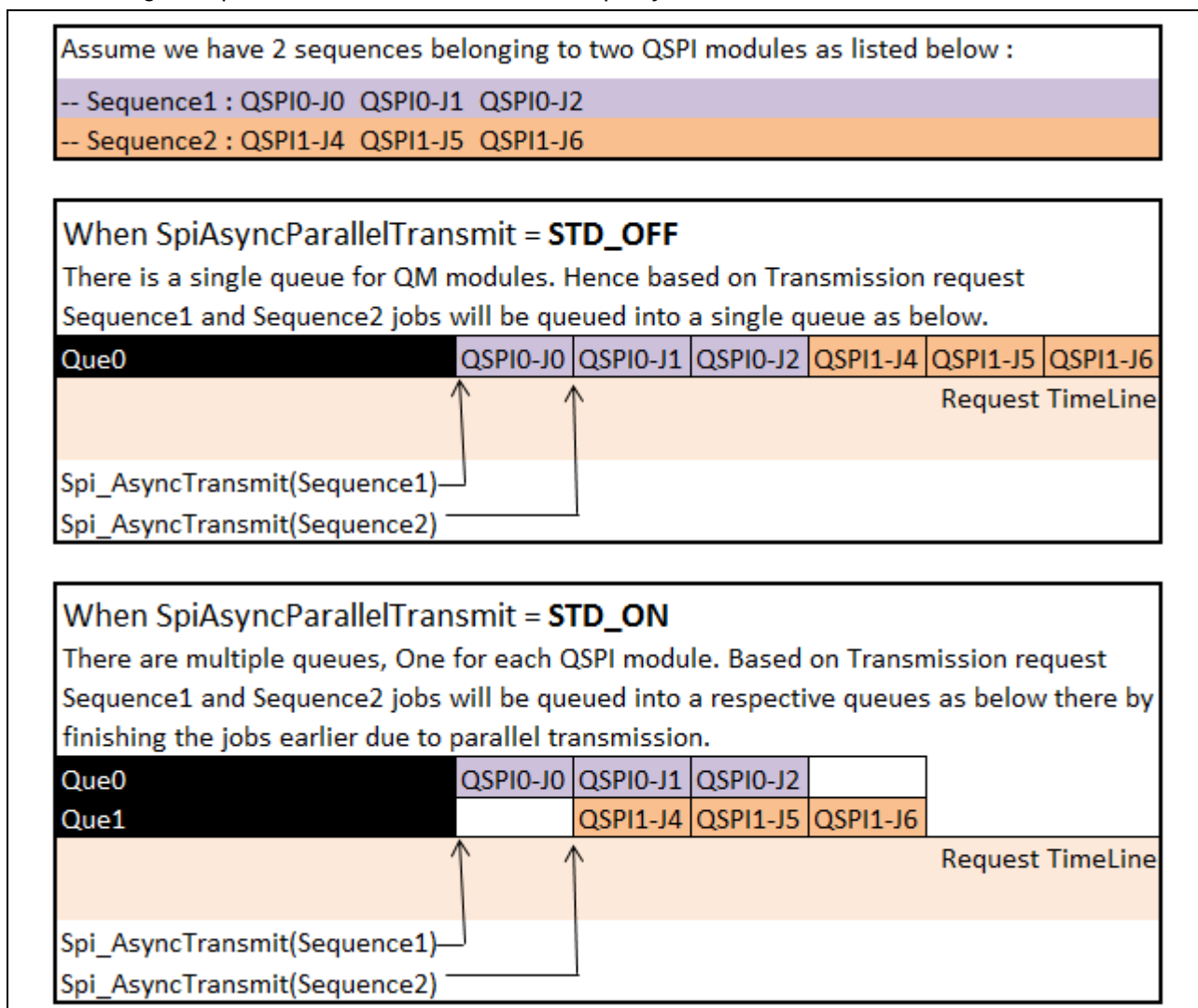


Figure 8 Use case 3: Difference between SpiAsyncParallelTransmit ON/OFF

Behaviour of Asynchronous transmission with SpiChannelBasedCS enabled/Disabled

Terminology,

Last byte delay - Delay between last two data buffers of last SpiChannel assigned to a SpiJob.

Frame Delays - Leading, Trailing, and Idle delays

Table 134 Asynchronous transmission with different Chip Select configurations

SpiChannelBasedCS	SpiCsSelection	Asynchronous transmission description
Enabled	CS_VIA_PERIPHERAL_ENGINE	No Last byte delay, with frame delays as configured. Chip Select assertion/deassertion between the transmissions of channel data buffers.

Disabled	CS_VIA_GPIO	Small Last byte delay, with frame delays more than configured. Chip Select assertion before start of job and deassertion after end of job
Disabled	CS_VIA_PERIPHERAL_ENGINE	Small Last byte delay, frame delays as configured. Chip Select assertion before start of job and deassertion after end of job
Enabled	CS_VIA_GPIO	No Last byte delay, with frame delays more than configured. Chip Select assertion before start of job and deassertion after end of job

6.2.6 Spi_ReadIB

Table 135 Specification of API Spi_ReadIB

Service name:	Spi_ReadIB	
Syntax:	Std_ReturnType Spi_ReadIB (Spi_ChannelType Channel, Spi_DataType *DataBufferPtr)	
Service ID:	0x03	
Sync/ Async:	Synchronous	
Origin	Autosar	
Re-entrancy:	Re-entrant	
Parameters (in):	Channel	Channel ID
	DataBufferPtr	Pointer to destination data buffer in RAM
Parameters (out):	None	
Return value:	E_OK	Read command was accepted.
	E_NOT_OK	Read command was not accepted.
Description:	Service for reading synchronously one or more data from an IB SPI Handler/Driver channel specified by parameter. Different Jobs, consequently also sequences, can have in common channels. But, the use of those shared channels is restricted. Read and write functions cannot guarantee the data integrity while channel data is transmitted. User must ensure that read and/or write functions are not called during transmission.	
Caveats:	The SPI Handler/Driver should be initialized before this service is called. This method is called after one transmit method to have relevant data within IB channel.	
Configuration:	This function is pre compile time configurable by the parameter: SPI_CHANNEL_BUFFERS_ALLOWED. This service is only relevant for channels with IB.	
DET:	SPI_E_UNINIT	SPI is not initialized.
	SPI_E_PARAM_CHANNEL	Channel is not a valid IB channel
DEM:	None	
SAFE MCAL ERROR	Refer Safety errors [Table 163] Safety channel buffer marker check is provided in channel based APIs. The	

	buffer marker value is generated as the last element of Channel Configuration Structure. This marker value will be checked to ensure valid channel configuration, when SAFETY is enabled. The value of safety marker is generated based on the configuration parameter, SpiSafetyBufferMarker. There will be only one buffer marker for a Data Buffer of any length per channel. Safety buffer marker is available irrespective of ASIL/QM channels
Implementation Comments:	In case of 16bit channel data width the DataBufferPtr must be 16bit aligned. In case of 32bit channel data width the DataBufferPtr must be 32bit aligned. Use the memory map section: SPI_START_SEC_VAR_DMA_ACCESS for this purpose.
I/O Mode:	This API can be invoked when the CPU is in User-0 Mode, provided configuration parameter SpiRunningInUser0Mode is 'True' and SpiUserModelInitApiEnable is 'True'. This API can be invoked when the CPU is in User-1 Mode, provided configuration parameter SpiRunningInUser0Mode is 'False' and SpiUserModelInitApiEnable is 'True'. This API can be invoked when the CPU is in Supervisor Mode, provided configuration parameter SpiRunningInUser0Mode is 'False' and SpiUserModelInitApiEnable is 'False'..

6.2.7 Spi_SetupEB

Table 136 Specification of API Spi_SetupEB

Service name:	Spi_SetupEB	
Syntax:	Std_ReturnType Spi_SetupEB (Spi_ChannelType Channel, const Spi_DataType* SrcDataBufferPtr, Spi_DataType* DesDataBufferPtr, Spi_NumberOfDataType Length) 	
Service ID:	0x05	
Sync/ Async:	Synchronous	
Origin	Autosar	
Re-entrancy:	non re-entrant	
Parameters (in):	Channel	Channel ID
	SrcDataBufferPtr	Pointer to source data buffer
	DesDataBufferPtr	Pointer to destination data buffer in RAM
	Length	Number of data transfers.
Parameters (out):	None	
Return value:	E_OK	Spi_SetupBuffers was accepted.
	E_NOT_OK	Spi_SetupBuffers was not accepted.
Description:	Service to setup the buffers and the length of data for the EB SPI Handler/Driver channel specified. This service updates the buffer pointers and length attributes of the specified channel with the provided values. As these attributes are persistent, they will be used for all succeeding calls to one transmit method (for the specified channel).	

	If SrcDataBufferPtr is a null pointer the default transmit value configured for the channel will be transmitted when a transmit method is requested. If DesDataBufferPtr is a null pointer the received data will be ignored when a transmit method is requested. Length can vary within the maximum size limit for the Channel buffer which is configured.	
Caveats:	The SPI Handler/Driver should be initialized before this service is called. This method is called one time for all channels with EB declared before to call a transmit method for them.	
Configuration:	This function is pre compile time configurable by the parameter: SPI_CHANNEL_BUFFERS_ALLOWED. This service is only relevant for channels with EB.	
DET:	SPI_E_UNINIT	SPI is not initialized.
	SPI_E_PARAM_CHANNEL	The channel is not a valid EB channel
	SPI_E_PARAM_LENGTH	If the Length of the buffer does not meet the conditions specified in configuration.
	SPI_E_DMA_ADR_ALIGN_ERROR	DMA implements a circular buffer with a maximum width of 32KB. So the source / destination ptrs should not span the 32KB Boundary if sequential data are to be transferred. This address alignment is checked. This check is performed only in async mode when DMA is used.
DEM:	None	
SAFE MCAL ERROR	Refer Safety errors [Table 163] Safety channel buffer marker check is provided in channel based APIs. The buffer marker value is generated as the last element of Channel Configuration Structure. This marker value will be checked to ensure valid channel configuration, when SAFETY is enabled. The value of safety marker is generated based on the configuration parameter, SpiSafetyBufferMarker. There will be only one buffer marker for a Data Buffer of any length per channel. Safety buffer marker is available irrespective of ASIL/QM channels	
Implementation Comments:	If the channel has a 16bit channel data width, the SrcDataBufferPtr and DesDataBufferPtr must be 16 bit Aligned. If the channel has a 32bit channel data width, the SrcDataBufferPtr and DesDataBufferPtr must be 32 bit Aligned. Additionally the buffer pointers + length should be within a 32KB boundary. Length is implemented as Number of data transfers(not as Bytes). For Example: • When 4 bytes i.e. 8 bit x 4 datawidth needs to be transferred Length = 4 • When 4 bytes i.e. 16 bit x 2 datawidth needs to be transferred Length = 2 • When 4 bytes i.e. 32 bit x1 datawidth needs to be transferred Length = 1 Note that DMA implements a circular buffer with a maximum width of 32KB. So the source/destination address should not cross the 32KB boundary if sequential data	

	are to be transferred. This address alignment is checked. ((address + length) & 0x00007FFF) <= 0x00008000) Use the memory map section: SPI_START_SEC_VAR_DMA_ACCESS for this purpose. This restriction is applicable only for Asynchronous transmissions which uses DMA.
I/O Mode:	This API can be invoked when the CPU is in User-0 Mode, provided configuration parameter SpiRunningInUser0Mode is 'True' and SpiUserModelInitApiEnable is 'True'. This API can be invoked when the CPU is in User-1 Mode, provided configuration parameter SpiRunningInUser0Mode is 'False' and SpiUserModelInitApiEnable is 'True'. This API can be invoked when the CPU is in Supervisor Mode, provided configuration parameter SpiRunningInUser0Mode is 'False' and SpiUserModelInitApiEnable is 'False'.

6.2.8 Spi_GetStatus

Table 137 Specification of API Spi_GetStatus

Service name:	Spi_GetStatus	
Syntax:	Spi_StatusType Spi_GetStatus (void)	
Service ID:	0x06	
Sync/ Async:	Synchronous	
Origin	Autosar	
Re-entrancy:	Re-entrant	
Parameters (in):	None	
Parameters (out):	None	
Return value:	Spi_StatusType	SPI_UNINIT : The SPI Handler/Driver is not initialized or not usable SPI_IDLE : The SPI Handler/Driver is not currently transmitting any jobs. SPI_BUSY : The SPI Handler/Driver is performing a SPI job (transmit).
Description:	This service returns the SPI Handler/Driver status.	
Caveats:	None	
Configuration:	None	
DET:	None	
DEM:	None	
SAFE MCAL ERROR	Refer Safety errors [Table 163]	
Implementation Comments:	None	
I/O Mode:	This API can be invoked when the CPU is in User-0 Mode, provided configuration parameter SpiRunningInUser0Mode is 'True' and SpiUserModelInitApiEnable is 'True'. This API can be invoked when the CPU is in User-1 Mode, provided configuration parameter SpiRunningInUser0Mode is 'False' and SpiUserModelInitApiEnable is 'True'.	

	This API can be invoked when the CPU is in Supervisor Mode, provided configuration parameter SpiRunningInUser0Mode is 'False' and SpiUserModelInitApiEnable is 'False'.
--	---

6.2.9 Spi_GetJobResult

Table 138 Specification of API Spi_GetJobResult

Service name:	Spi_GetJobResult	
Syntax:	<pre>Spi_JobResultType Spi_GetJobResult (Spi_JobType Job)</pre>	
Service ID:	0x07	
Sync/ Async:	Synchronous	
Origin	Autosar	
Re-entrancy:	Re-entrant	
Parameters (in):	Job	Job ID. An invalid job ID will return an undefined result.
Parameters (out):	None	
Return value:	Spi_JobResultType	SPI_JOB_OK: The last transmission of the job has been finished successfully. SPI_JOB_PENDING: The SPI Handler/Driver is performing a specific SPI job. The meaning of this status is equal to SPI_BUSY. SPI_JOB_FAILED: The last transmission of the job has failed. SPI_JOB_QUEUED: The Job has been accepted, while actual transmission for this Job has not started yet.
Description:	This service returns the last transmission result of the specified job. Every new transmit job that has been accepted by the SPI Handler/Driver overwrites the previous job result with SPI_JOB_PENDING. The user calls Spi_GetJobResult method to know if the job transmission succeeded (SPI_JOB_OK) or failed (SPI_JOB_FAILED).	
Caveats:	If SPI Handler/Driver has not been initialized before this service is called, the return value is undefined.	
Configuration:	None	
DET:	SPI_E_UNINIT	SPI is not initialized.
	SPI_E_PARAM_JOB	Invalid job Id
DEM:	None	
SAFE MCAL ERROR	Refer Safety errors [Table 163]	
Implementation Comments:	If SPI Handler/Driver has not been initialized before this service is called, the return value will be SPI_JOB_OK.	

I/O Mode:	This API can be invoked when the CPU is in User-0 Mode, provided configuration parameter SpiRunningInUser0Mode is 'True' and SpiUserModelInitApiEnable is 'True'. This API can be invoked when the CPU is in User-1 Mode, provided configuration parameter SpiRunningInUser0Mode is 'False' and SpiUserModelInitApiEnable is 'True'. This API can be invoked when the CPU is in Supervisor Mode, provided configuration parameter SpiRunningInUser0Mode is 'False' and SpiUserModelInitApiEnable is 'False'.
------------------	---

6.2.10 Spi_GetSequenceResult

Table 139 Specification of API Spi_GetSequenceResult

Service name:	Spi_GetSequenceResult	
Syntax:	<pre>Spi_SequenceResultType Spi_GetSequenceResult (Spi_SequenceType Sequence)</pre>	
Service ID:	0x08	
Sync/ Async:	Synchronous	
Origin	Autosar	
Re-entrancy:	Re-entrant	
Parameters (in):	Sequence	Sequence ID. An invalid sequence ID will return an undefined result.
Parameters (out):	None	
Return value:	Spi_SeqResultType	SPI_SEQ_OK: The last transmission of the sequence has been finished successfully. SPI_SEQ_PENDING: The SPI Handler/Driver is performing a specific SPI sequence. The meaning of this status is equal to SPI_BUSY. SPI_SEQ_FAILED: The last transmission of the sequence has failed. SPI_SEQ_CANCELED: The last transmission of the sequence has been cancelled by user
Description:	This service returns the last transmission result of the specified sequence. Every new transmit sequence that has been accepted by the SPI Handler/Driver overwrites the previous sequence result with SPI_SEQ_PENDING. The Spi_GetSequenceResult method is called to know if the fully sequence transmission succeeded (SPI_SEQ_OK) or failed (SPI_SEQ_FAILED)	
Caveats:	If SPI Handler/Driver has not been initialized before this service is called, the return value is undefined.	
Configuration:	None	
DET:	SPI_E_UNINIT	SPI is not initialized.
	SPI_E_PARAM_SEQ	Invalid Sequence Id
DEM:	None	

SAFE MCAL ERROR	Refer Safety errors [Table 163]
Implementation Comments:	If SPI Handler/Driver has not been initialized before this service is called, the return value will be SPI_SEQ_OK.
I/O Mode:	This API can be invoked when the CPU is in User-0 Mode, provided configuration parameter SpiRunningInUser0Mode is 'True' and SpiUserModelInitApiEnable is 'True'. This API can be invoked when the CPU is in User-1 Mode, provided configuration parameter SpiRunningInUser0Mode is 'False' and SpiUserModelInitApiEnable is 'True'. This API can be invoked when the CPU is in Supervisor Mode, provided configuration parameter SpiRunningInUser0Mode is 'False' and SpiUserModelInitApiEnable is 'False'.

6.2.11 Spi_GetVersionInfo

Table 140 Specification of API Spi_GetVersionInfo

Service name:	Spi_GetVersionInfo	
Syntax:	<pre>void Spi_GetVersionInfo { Std_VersionInfoType *VersionInfoPtr }</pre>	
Service ID:	0x09	
Sync/ Async:	Synchronous	
Origin	Autosar	
Re-entrancy:	Re-entrant	
Parameters (in):	None	
Parameters (out):	versioninfo	Pointer to where to store the version information of this Driver.
Return value:	None	
Description:	This service returns the version information of this driver. The version information includes: - Module Id - Vendor Id - Vendor specific version numbers (BSW00407).	
Caveats:	None	
Configuration:	This function is pre compile time configurable On/Off by the configuration parameter: SPI_VERSION_INFO_API	
DET:	SPI_E_UNINIT	SPI is not initialized.
	SPI_E_PARAM_POINTER	Input parameter is NULL
DEM:	None	
SAFE MCAL ERROR	Refer Safety errors [Table 163]	
Implementation Comments:	None	
I/O Mode:	This API can be invoked when the CPU is in User-0 Mode, provided configuration parameter SpiRunningInUser0Mode is 'True' and SpiUserModelInitApiEnable is 'True'. This API can be invoked when the CPU is in User-1 Mode, provided configuration parameter SpiRunningInUser0Mode is 'False' and SpiUserModelInitApiEnable is	

'True'.

This API can be invoked when the CPU is in Supervisor Mode, provided configuration parameter SpiRunningInUser0Mode is 'False' and SpiUserModelInitApiEnable is 'False'.

6.2.12 Spi_SyncTransmit

Table 141 Specification of API Spi_SyncTransmit

Service name:	Spi_SyncTransmit	
Syntax:	Std_ReturnType Spi_SyncTransmit { Spi_SequenceType Sequence } 	
Service ID:	0x0A	
Sync/ Async:	Synchronous	
Origin	Autosar	
Re-entrancy:	Re-entrant	
Parameters (in):	Sequence	Sequence ID
Parameters (out):	None	
Return value:	E_OK	Transmission command has been accepted
	E_NOT_OK	Transmission command has been rejected
Description:	Sync Service to transmit data on the SPI bus. This service take over the given parameter, initiate a transmission, and set the SPI Handler/Driver status to SPI_BUSY, set the sequence result to SPI_SEQ_PENDING, set the first job result to SPI_JOB_PENDING and return The LEVEL 0 SPI Handler/Driver offers a synchronous transfer service for SPI busses. When there is no on-going sequence transmission, the SPI Handler/Driver is set idle state (SPI_IDLE). Hence, the LEVEL 0 SPI Handler/Driver does not allow transmitting more than one sequence at the same time i.e.: during an ongoing transmission, all requests to transmit another sequence will be rejected. If a request is done while a sequence is on transmission, the SPI Handler/Driver does not take in account this new request and this service returns value E_NOT_OK. The SPI HandlerDriver reports SPI_E_SEQ_IN_PROCESS error.	
Caveats:	None	
Configuration:	This function is pre compile time selectable by the configuration parameter: SPI_LEVEL_DELIVERED. This function is only relevant for LEVEL 0 and LEVEL 2. When the function Spi_SyncTransmit is called while a sequence is on transmission and SPI_SUPPORT_CONCURRENT_SYNC_TRANSMIT is disabled or an-other sequence is on transmission on same bus, the SPI Handler/Driver shall not take into account this new transmission request and the function shall return the value E_NOT_OK. In Level 2, the API would be available if Atleast one synchronous job is configured.	
DET:	SPI_E_UNINIT	SPI is not initialized.
	SPI_E_PARAM_SEQ	Invalid sequence Id
	SPI_E_SEQ_IN_PROCESS	If a request is done while a sequence is on transmission.
DEM:	SPI_E_HARDWARE_ERROR	HW specific error. A receive error is detected when a new data frame is completely received but the previous data

		was not read out of the receive buffer register.
SAFE MCAL ERROR	Refer Safety errors [Table 163]	
Implementation Comments:	None	
I/O Mode:	This API can be invoked when the CPU is in User-0 Mode, provided configuration parameter SpiRunningInUser0Mode is 'True' and SpiUserModelInitApiEnable is 'True'. This API can be invoked when the CPU is in User-1 Mode, provided configuration parameter SpiRunningInUser0Mode is 'False' and SpiUserModelInitApiEnable is 'True'. This API can be invoked when the CPU is in Supervisor Mode, provided configuration parameter SpiRunningInUser0Mode is 'False' and SpiUserModelInitApiEnable is 'False'.	

6.2.13 Spi_GetHWUnitStatus

Table 142 Specification of API Spi_GetHWUnitStatus

Service name:	Spi_GetHWUnitStatus	
Syntax:	Spi_StatusType Spi_GetHWUnitStatus (Spi_HWUnitType HWUnit)	
Service ID:	0x0B	
Sync/ Async:	Synchronous	
Origin	Autosar	
Re-entrancy:	Re-entrant	
Parameters (in):	HWUnit	SPI Hardware microcontroller peripheral (unit) ID.
Parameters (out):	None	
Return value:	Spi_StatusType	SPI_UNINIT : The SPI Handler/Driver is not initialized or not usable SPI_IDLE : The SPI Handler/Driver is not currently transmitting any jobs. SPI_BUSY : The SPI Handler/Driver is performing a SPI job (transmit).
Description:	This service returns the status of the specified SPI hardware microcontroller peripheral. The Spi_GetHWUnitStatus method is called to know if the specified SPI hardware microcontroller peripheral is SPI_IDLE or SPI_BUSY.	
Caveats:	If SPI Handler/Driver has not been initialized before this service is called, the return value is undefined.	
Configuration:	This function is pre compile time configurable on / off by the configuration parameter: SPI_HW_STATUS_API.	
DET:	SPI_E_PARAM_UNIT	HWUnit is not within the specified range of values.
DEM:	None	
SAFE MCAL ERROR	Refer Safety errors [Table 163]	

Implementation Comments:	None
I/O Mode:	This API can be invoked when the CPU is in User-0 Mode, provided configuration parameter SpiRunningInUser0Mode is 'True' and SpiUserModelInitApiEnable is 'True'. This API can be invoked when the CPU is in User-1 Mode, provided configuration parameter SpiRunningInUser0Mode is 'False' and SpiUserModelInitApiEnable is 'True'. This API can be invoked when the CPU is in Supervisor Mode, provided configuration parameter SpiRunningInUser0Mode is 'False' and SpiUserModelInitApiEnable is 'False'.

6.2.14 Spi_Cancel

Table 143 Specification of API Spi_Cancel

Service name:	Spi_Cancel	
Syntax:	<pre>void Spi_Cancel (Spi_SequenceType Sequence)</pre>	
Service ID:	0x0C	
Sync/ Async:	Asynchronous	
Origin	Autosar	
Re-entrancy:	Re-entrant	
Parameters (in):	Sequence	Sequence ID
Parameters (out):	None	
Return value:	None	
Description:	This service cancels the specified on-going sequence transmission without cancelling any job transmission and the SPI Handler/Driver will set the sequence result to SPI_SEQ_CANCELED. With other words, the Spi_Cancel function stops a sequence transmission after a (possible) on transmission job ended and before a (potential) next job transmission starts. When the sequence is actually cancelled, if configured, the sequence notification call-back function is executed instead of starting a potential next job belonging to it.	
Caveats:	The SPI HandlerDriver is not responsible on external devices damages or undefined state due to cancelling a sequence transmission. The user must be aware of the consequences of cancelling a sequence transmission.	
Configuration:	This function is pre compile time configurable on / off by the configuration parameter: SPI_CANCEL_API.	
DET:	SPI_E_UNINIT	SPI is not initialized.
	SPI_E_PARAM_SEQ	Invalid sequence Id
DEM:	None	
SAFE MCAL ERROR	Refer Safety errors [Table 163]	
Implementation Comments:	The Cancel status of the sequence is maintained until it is started again.	

I/O Mode:	This API can be invoked when the CPU is in User-0 Mode, provided configuration parameter SpiRunningInUser0Mode is 'True' and SpiUserModelInitApiEnable is 'True'. This API can be invoked when the CPU is in User-1 Mode, provided configuration parameter SpiRunningInUser0Mode is 'False' and SpiUserModelInitApiEnable is 'True'. This API can be invoked when the CPU is in Supervisor Mode, provided configuration parameter SpiRunningInUser0Mode is 'False' and SpiUserModelInitApiEnable is 'False'..
------------------	--

6.2.15 Spi_SetAsyncMode

Table 144 Specification of API Spi_SetAsyncMode

Service name:	Spi_SetAsyncMode	
Syntax:	Std_ReturnType Spi_SetAsyncMode { Spi_AsyncModeType Mode } 	
Service ID:	0x0D	
Sync/ Async:	Synchronous	
Origin	Autosar	
Re-entrancy:	Non Re-entrant	
Parameters (in):	Mode	New Mode Required
Parameters (out):	None	
Return value:	E_OK	Setting command has been done
	E_NOT_OK	Setting command has not been accepted
Description:	Service to set the asynchronous mechanism mode for SPI buses handled asynchronously. This service sets the mode following the given parameter. If a request is done while the SPI Handler/Driver status is not in SPI_IDLE, it does not take in account this new request, the mode is kept as it is and this service returns value E_NOT_OK.	
Caveats:	None	
Configuration:	This function is pre compile time selectable by the configuration parameter: SPI_LEVEL_DELIVERED. This function is only relevant for LEVEL 2.	
DET:	SPI_E_UNINIT	SPI is not initialized.
DEM:	None	
SAFE MCAL ERROR	Refer Safety errors [Table 163]	
Implementation Comments:	In the LEVEL2 functionalities minimum two SPI buses required, Atleast one bus is assigned to synchronous transmission and other to asynchronous transmission.	
I/O Mode:	This API can be invoked when the CPU is in User-0 Mode, provided configuration parameter SpiRunningInUser0Mode is 'True' and SpiUserModelInitApiEnable is 'True'. This API can be invoked when the CPU is in User-1 Mode, provided configuration parameter SpiRunningInUser0Mode is 'False' and SpiUserModelInitApiEnable is 'True'. This API can be invoked when the CPU is in Supervisor Mode, provided configuration parameter SpiRunningInUser0Mode is 'False' and SpiUserModelInitApiEnable is 'False'.	

6.2.16 Spi_SlaveSetupEB

Table 145 Specification of API Spi_SlaveSetupEB

Service name:	Spi_SlaveSetupEB	
Syntax:	Std_ReturnType Spi_SlaveSetupEB (Spi_ChannelType Channel, Spi_DataType* DesDataBufferPtr, Spi_NumberOfDataType DataLength)	
Service ID:	0x11	
Sync/ Async:	Synchronous	
Origin	Vendor specific	
Re-entrancy:	Re-entrant	
Parameters (in):	Channel	Channel ID
	DesDataBufferPtr	Pointer to Destination data buffer
	Length	Number of data transfers
Parameters (out):	None	
Return value:	E_OK	Setting command has been done
	E_NOT_OK	Setting command has not been accepted
Description:	Service to setup the buffers and the length of data for the External Buffer of SPI SLAVE Channel Service is called when Master uses SetupEB service for External Buffer [EB] buffers. This service shall update the buffer pointers and length attributes of the specified Channel with the provided values. As these attributes are persistent, they will be used for all succeeding calls to one read method (for the specified Channel). To read a variable number of data, it is mandatory to call Spi_SlaveSetupEB function to store new parameters within Spi Slave before each Spi_SlaveAsyncRead / SpiSlaveSyncRead function calls. The size of the buffer provided by the user should be less than maximum size for Channel configured in configuration.	
Caveats:	In case of 16/32bit Channel Data Width the DataBufferPtr must be 16bit aligned. Different Jobs, consequently also Sequences, can have in common Channels. But, the use of those shared Channels is restricted. Slave SetupEB function cannot guarantee the data integrity while Channel data is being read. Application as to track and take care of common usage of channels across jobs. This method shall be called for all Channels with EB declared before call to corresponding Master EB method and a Read method for them.	
Configuration:	None	
DET:	None	
DEM:	None	
SAFE MCAL ERROR	Refer Safety errors [Table 163]	

Implementation Comments:	None
I/O Mode:	This API can be invoked when the CPU is in User-0 Mode, provided configuration parameter SpiRunningInUser0Mode is 'True' and SpiUserModelInitApiEnable is 'True'. This API can be invoked when the CPU is in User-1 Mode, provided configuration parameter SpiRunningInUser0Mode is 'False' and SpiUserModelInitApiEnable is 'True'. This API can be invoked when the CPU is in Supervisor Mode, provided configuration parameter SpiRunningInUser0Mode is 'False' and SpiUserModelInitApiEnable is 'False'..

6.2.17 Spi_SlaveReadPrepare

Table 146 Specification of API Spi_SlaveReadPrepare

Service name:	Spi_SlaveReadPrepare	
Syntax:	Std_ReturnType Spi_SlaveReadPrepare(Spi_SequenceType Sequence)	
Service ID:	0x12	
Sync/ Async:	Asynchronous	
Origin	Vendor specific	
Re-entrancy:	Re-entrant	
Parameters (in):	Sequence	Sequence ID
Parameters (out):	None	
Return value:	E_OK	Setting command has been done
	E_NOT_OK	Setting command has not been accepted
Description:	Slave Read Prepare to read/receive data on the SPI bus. This service takes over the given parameter 'Sequence', Queues jobs and Initializes QSPiX (connected to each job) to Slave Mode and starts receiving data on bus transmitted by QSPiY, configured in Master Mode. Jobs in each sequence are queued based on priority. If SlaveReadPrepare service is called with new sequence when current sequence is ongoing then depending on configured parameter 'Interruptible sequence' or 'non-interruptible sequence' Jobs of each sequence is queued by calling local function Spi_JobSeqQueueUpdate. This function is pre compile time selectable by the configuration parameter: SPI_LEVEL_DELIVERED. This function is only relevant for LEVEL 1 and LEVEL 2. This service also updates slave bus status to SPI_BUSY, set the sequence result to SPI_SEQ_PENDING, set the first job result to SPI_JOB_PENDING and return.	
Caveats:	None	
Configuration:	None	
DET:	None	
DEM:	None	
SAFE MCAL ERROR	Refer Safety errors [Table 163]	

Implementation Comments:	None
I/O Mode:	This API can be invoked when the CPU is in User-0 Mode, provided configuration parameter SpiRunningInUser0Mode is 'True' and SpiUserModelInitApiEnable is 'True'. This API can be invoked when the CPU is in User-1 Mode, provided configuration parameter SpiRunningInUser0Mode is 'False' and SpiUserModelInitApiEnable is 'True'. This API can be invoked when the CPU is in Supervisor Mode, provided configuration parameter SpiRunningInUser0Mode is 'False' and SpiUserModelInitApiEnable is 'False'.

6.3 Scheduled functions

These Functions should be called by Basic Software Scheduler.

6.3.1 Spi_MainFunction_Handling

Table 147 Specification of API Spi_MainFunction

Service name:	Spi_MainFunction_Handling
Syntax:	void Spi_MainFunction_Handling(void)
Service ID:	0x10
Sync/ Async:	Synchronous
Origin	Autosar
Re-entrancy:	Non- Reentrant
Parameters (in):	None
Parameters (out):	None
Return value:	None
Timing:	Fixed Cyclic The user has to determine the cycle rate depending upon the baud rate of the transmission and frequency of the job scheduling required.
Description:	Description: This function replaces the interrupt mechanism used to handle jobs on the bus. The function polls for the completion of a channel transmission (DMA transaction) and calls the bus handler function. In addition this function also checks for errors (production) on the SPI bus and performs the needed error handling.
Caveats:	List of assumptions about the environment in which the API call must operate.
Configuration:	Description of statically configurable attributes that affect this API call. For instance cycle time(s) in case of fixed cyclic timing.
DET:	None.
DEM:	None.
Implementation Comments:	None.
I/O Mode:	This API can be invoked when the CPU is in User-0 Mode, provided configuration parameter SpiRunningInUser0Mode is 'True' and SpiUserModelInitApiEnable is 'True'. This API can be invoked when the CPU is in User-1 Mode, provided configuration parameter SpiRunningInUser0Mode is 'False' and SpiUserModelInitApiEnable is 'True'. This API can be invoked when the CPU is in Supervisor Mode, provided configuration

parameter SpiRunningInUser0Mode is 'False' and SpiUserModelInitApiEnable is 'False'.
--

6.4 Interrupt Handling in SPI

The following interrupt nodes are used and have to be configured.

QSPIx:

- **QSPIx_ERR** – used to handle Qspi errors
- **QSPIx_RX** – used to interrupt Dma for data reception
- **QSPIx_TX** – used to interrupt Dma for Data transmission.
- **QSPIx_PT** – used to handle End of frame phase transition.
- **QSPIx_U** – used to handle DNA phase transition.

DMA:

- **DMA_Chz_SR**, where 'z' ranges from 0 to n (device specific). 'z' denotes the DMA channel used for data transmission/reception. Each Qspi uses one Dma channel for transmission and one for reception.
- **DMA_Err_SR** – used to handle DMA errors for all the DMA channels configured.

Below is the recommended interrupt priority configuration, where 'ISR_Prio' is the priority number assigned to the interrupt node.

- Priority for QSPI_Tx and QSPI_Rx should be the DMA channel number used for transmission and reception respectively. (Since TOS is DMA)
- Priority for QSPI_PT should be greater than the priority of Error interrupts
- Priority for Error interrupts should be greater than priority of QSPI_U, DMA Tx and DMA Rx channel interrupts
- Priority of QSPI_U should be greater than priority of DMA Tx and DMA Rx channel interrupt.

Also, priority for DMA Tx channel should be greater than DMA Rx channel.

i.e., $ISR_Prio(QSPI_PT) > ISR_Prio(DMA_Err/QSPIx_ERR) > ISR_Prio(QSPI_U)$,

$ISR_Prio(QSPI_U) > ISR_Prio(DMA_Tx_channel) > ISR_Prio(DMA_Rx_channel)$

Code Listing 1 Interrupt Priority configuration for QSPI2 node (Irq_Cfg.h)

```

001:  /* DMA Ch4 is the QSPI Reception dma channel*/
002:  /* DMA Ch5 is the QSPI Transmission dma channel*/
003:  /* Qspi Rx interrupt Priority Setting*/
004:  #define IRQ_QSPI2_RX_PRIO          0x4
005:
006:  /* Qspi Tx interrupt Priority Setting*/
007:  #define IRQ_QSPI2_TX_PRIO          0x5
008:
009:  /* Qspi Err interrupt Priority Setting*/
010:  #define IRQ_QSPI2_ERR_PRIO         0x2f
011:
012:  /* Qspi PT interrupt Priority Setting*/
013:  #define IRQ_QSPI2_PT_PRIO          0x34
014:
015:  /* Qspi UD interrupt Priority Setting*/
016:  #define IRQ_QSPI2_UD_PRIO          0x1b

```

Code Listing 1 Interrupt Priority configuration for QSPI2 node (Irq_Cfg.h)

```

017:
018:  /* DMA interrupt Priority settings */
019:  #define IRQ_DMA_CHANNEL4_SR_PRIO    0x14
020:  #define IRQ_DMA_CHANNEL5_SR_PRIO    0x15
021:  #define IRQ_DMA_ERR_SR_PRIO        0x2c

```

General Notes:

- The ISR is needed according to the peripheral configured. For example if QSPI1 is not used by any of the jobs then QSPI1_ERR, DMA_ME_CHn (where n is the DMA channel used for data transmission/reception) are not needed.
- In Level0 Driver mode (synchronous transmission mechanism) none of the interrupts are needed.
- The Level1 Driver mode, by default is realized using the interrupt mechanism. All the interrupts will be used here.
- For configurations, with parallel transmission on different QSPI hardware, the interrupts mentioned in the section below are re-entrant.

The functions to be called in the interrupt context are explained in the following sections.

6.4.1 Spi_IsrDmaQspiRx

Table 148 Specification of API Spi_IsrDmaQspiRx

Service name:	Spi_IsrDmaQspiRx
Syntax:	void Spi_IsrDmaQspiRx(uint32 Module)
Service ID:	0x34
Sync/ Async:	Synchronous
Origin	Vendor specific
Re-entrancy:	Non Re-entrant
Parameters (in):	Module
Parameters (out):	None
Return value:	None
Description:	Description: This routine should be called in the Rx-DMA Channel (Transaction complete / Error) Interrupt of QSPIx Module when no Dma Error has occurred. This function does the following activities if it is not end of the job: 1. Sets up a new DMA transaction for the next channels in the Channel Link. 2. If the channel transmitted is the last channel in a job link, and the Sequences is not interruptible, the next job in the Job link is setup If the Sequence is interruptible, the scheduler to schedule the next job based on priority is called. 3. At the end of the job if configured, a job end notification is called. 4. At the end of the Sequence, if configured, a seq end notification is called. If it is end of the job : 1. Enables the phase transition interrupt for the EOF phase
Caveats:	None
SAFE MCAL ERROR	Refer Safety errors [Table 163]
Configuration:	None

I/O Mode:	Supervisor Mode.
-----------	------------------

6.4.2 Spi_IsrDmaQspiTx

Table 149 Specification of API Spi_IsrDmaQspiTx

Service name:	Spi_IsrDmaQspiTx
Syntax:	void Spi_IsrDmaQspiTx (uint32 Module)
Service ID:	0x33
Sync/ Async:	Synchronous
Origin	Vendor specific
Re-entrancy:	Non Re-entrant
Parameters (in):	Module
Parameters (out):	None
Return value:	None
Description:	Description: This routine should be called in the TX-Dma Channel (Transaction complete / Error) Interrupt of QSPIx Module when no Dma Error has occurred. This function disables the Tx-Dma Channel interrupt and also the Qspi Tx interrupt. Then it sets the BACON.LAST to 1 and updates the last word to be transmitted on Qspix Hw. This transfers [i.e. TX] the last word of Last Channel of job and puts the Chipselect to inactive state
Caveats:	None
SAFE MCAL ERROR	Refer Safety errors [Table 163]
Configuration:	None
I/O Mode:	Supervisor Mode.

6.4.3 Spi_IsrQspiError

Table 150 Specification of API Spi_IsrQspiError

Service name:	Spi_IsrQspiError
Syntax:	void Spi_IsrQspiError (uint32 Module)
Service ID:	0x37
Sync/ Async:	Synchronous
Origin	Vendor specific
Re-entrancy:	Non Re-entrant
Parameters (in):	Module
Parameters (out):	None
Return value:	None
Description:	Description : This routine should be called in the Error Interrupt of QSPIx Module. This interrupt occurs in case of Receive / Phase Errors. This interrupt is mapped to the node: QSPIx_ERR This function calls the DEM with Error Code - SPI_E_HARDWARE_ERROR Additionally it sets the Job and Sequence Status to SPI_JOB_FAILED, SPI_SEQUENCE_FAILED respectively. Also it calls the Job end and sequence end notifications, if configured. CS is also Disabled.

Caveats:	None
SAFE MCAL ERROR	Refer Safety errors [Table 163]
Configuration:	None
I/O Mode:	Supervisor Mode.

6.4.4 Spi_IsrDmaError

Table 151 Specification of API Spi_IsrDmaError

Service name:	Spi_IsrDmaError
Syntax:	void Spi_IsrDmaError (uint32 Module)
Service ID:	0x32
Sync/ Async:	Synchronous
Origin	Vendor specific
Re-entrancy:	Non Re-entrant
Parameters (in):	Module
Parameters (out):	None
Return value:	None
Description:	Description : This routine should be called in the TX/Rx Dma Channel (Transaction complete / Error) Interrupt of QSPIx Module when a Dma Error has occurred. This interrupt occurs in case of DMA Error, when a trigger request is lost. This function calls the DEM with Error Code - SPI_DEM_DMA_TRANSFER_ERROR. Additionally it sets the Job and Sequence Status to SPI_JOB_FAILED, SPI_SEQUENCE_FAILED respectively. Also it calls the Job end and sequence end notification if configured. CS is also Disabled. It also disables the Qspi Tx interrupt in case if it is enabled.
Caveats:	None
SAFE MCAL ERROR	Refer Safety errors [Table 163]
Configuration:	None
I/O Mode:	Supervisor Mode.

6.4.5 Spi_IsrQspiPt

Table 152 Specification of API Spi_IsrQspiPt

Service name:	Spi_IsrQspiPt
Syntax:	void Spi_IsrQspiPt (uint32 Module)
Service ID:	0x35
Sync/ Async:	Synchronous
Origin	Vendor specific
Re-entrancy:	Non Re-entrant
Parameters (in):	Module
Parameters (out):	None
Return value:	None
Description:	Description : This routine should be called in the Phase transition of

	QSPiX Module. This interrupt occurs in case of Phase transition of EOF. This interrupt is mapped to the node: QSPiX_PT This function does the following activities: 1. Sets up a new DMA transaction for the next channels in the Channel Link. 2. If the channel transmitted is the last channel in a job link, and the Sequences is not interruptible, the next job in the Job link is setup If the Sequence is interruptible, the scheduler to schedule the next job based on priority is called. 3. At the end of the job if configured, a job end notification is called. 4. At the end of the Sequence, if configured, a seq end notification is called.
Caveats:	None
SAFE MCAL ERROR	Refer Safety errors [Table 163]
Configuration:	None
I/O Mode:	Supervisor Mode.

6.4.6 Spi_IsrQspiUsr

Table 153 Specification of API Spi_IsrQspiUsr

Service name:	Spi_IsrQspiUsr
Syntax:	void Spi_IsrQspiUsr (uint32 Module)
Service ID:	0x36
Sync/ Async:	Synchronous
Origin	Vendor specific
Re-entrancy:	Non Re-entrant
Parameters (in):	Module
Parameters (out):	None
Return value:	None
Description:	Description : This routine should be called in the user interrupt of QSPiX Module. This interrupt occurs in case of Phase transition of DNA (data not available / start of expect phase). This interrupt is mapped to the node: QSPiX_U This function does the following activities: 1. Update the BACON.LAST to 1. 2. Write last data word to QSPI FIFO.
Caveats:	None
SAFE MCAL ERROR	Refer Safety errors [Table 163]
Configuration:	None
I/O Mode:	Supervisor Mode.

6.4.7 Spi_IsrCheckDmaError

Table 154 Specification of API Spi_IsrCheckDmaError

Service name:	Spi_IsrCheckDmaError
Syntax:	void Spi_IsrCheckDmaError (uint32 Module)
Service ID:	0x31
Sync/ Async:	Synchronous
Origin	Vendor specific
Re-entrancy:	Non Re-entrant
Parameters (in):	Module
Parameters (out):	None
Return value:	uint32 - DMA Tx or Rx Error status
Description:	This routine shall be called in the Tx/Rx complete for DMA Channel (Transaction, reception complete / Error) Interrupt of QSPIx Module.
Caveats:	None
SAFE MCAL ERROR	Refer Safety errors [Table 163]
Configuration:	None
I/O Mode:	Supervisor Mode.

6.5 Exclusive Area (EA) Information

The exclusive areas (critical sections) are provided for any of the following reason:

- 1) To protect shared resources from the module interrupts. The resources are used in ISR's.
- 2) To protect shared resources from another API / same API

6.5.1 Spi_Init

Table 155 Specification of EA Spi_Init

API Names:	Spi_Init
Resource Name:	QSPIx_CLC Note: x is 0-5
ISR Name:	NA
Comment	These registers are ENDINIT protected and hence the protection unlocking and subsequent locking should be completed before the Watchdog Timeout, therefore the critical section is created.

6.5.2 Spi_DelInit

Table 156 Specification of EA Spi_DelInit

API Names:	Spi_DelInit
Resource Name:	QSPIx_CLC. Note: x is 0-5
ISR Name:	NA
Comment	These registers are ENDINIT protected and hence the protection

unlocking and subsequent locking should be completed before the Watchdog Timeout, therefore the critical section is created.

6.5.3 Spi_WriteIB

Table 157 Specification of EA Spi_WriteIB

API Names:	Spi_WriteIB
Resource Name:	Spi_ChannelWriteLock
ISR Name:	NA
Comment	While updating the Channel set lock and channel clear lock status the interrupts are suspended and resumed back. This primarily done to make sure that no other call is made to write on the same channel while the channels are being locked.

6.5.4 Spi_AsyncTransmit

Table 158 Specification of EA Spi_AsyncTransmit

API Names:	Spi_AsyncTransmit
Resource Name:	Spi_StartIndexExtractJobId, Spi_SeqStatus, Spi_SeqCancelStatus,
ISR Name:	Spi_IBusHandler
Comment	The interrupts are suspended and resumed back to ensure that the shared resources are not modified by the interrupt handlers. The status variables used in this API are also read and updated by other driver API's.

6.5.5 Spi_SyncTransmit (AS4.x)

Table 159 Specification of EA Spi_SyncTransmit

API Names:	Spi_SyncTransmit
Resource Name:	CurHwUsed (local static variable)
ISR Name:	NA
Comment	Required only in concurrent transmission. If the API is called from one task to use SPI HW unit (e.g. SPI0) then before the start of usage of this HW unit, a short duration of interrupt protection is required to prohibit an incorrect usage of the same API being called from another task for the same SPI HW unit.

6.5.6 Spi_GetSequenceResult

Table 160 Specification of EA Spi_GetSequenceResult

API Names:	Spi_GetSequenceResult
Resource Name:	Spi_StartIndexExtractJobId, Spi_SeqStatus, Spi_SeqCancelStatus,
ISR Name:	Spi_IBusHandler
Comment	The interrupts are suspended and resumed back to ensure that the shared resources are not modified by the interrupt handlers. The status variables used in this API are also read and updated by other driver API's.

6.5.7 Spi_Cancel

Table 161 Specification of EA Spi_Cancel

API Names:	Spi_Cancel,
Resource Name:	Spi_StartIndexExtractJobId, Spi_SeqStatus, Spi_SeqCancelStatus,
ISR Name:	Spi_IBusHandler
Comment	The interrupts are suspended and resumed back to ensure that the shared resources are not modified by the interrupt handlers. The status variables used in this API are also read and updated by other driver API's.

7 Error Classification

7.1 Error Classification and Error codes

Table 162 DET and DEM Error Classification and Error Codes

Error Code	Error	Relevance	Value (hex)
SPI_E_PARAM_CONFIG	API service called with wrong parameter	Development	0x0F
SPI_E_PARAM_CHANNEL	API service called with wrong parameter	Development	0x0A
SPI_E_PARAM_JOB	API service called with wrong parameter	Development	0x0B
SPI_E_PARAM_SEQ	API service called with wrong parameter	Development	0x0C
SPI_E_PARAM_LENGTH	API service called with wrong parameter	Development	0x0D
SPI_E_PARAM_UNIT	API service called with wrong parameter	Development	0x0E
SPI_E_UNINIT	API Service called without Spi_Init being called	Development	0x1A
SPI_E_SEQ_PENDING	Services called in a wrong sequence	Development	0x2A
SPI_E_SEQ_IN_PROCESS	Synchronous transmission service called at wrong time	Development	0x3A
SPI_E_ALREADY_INITIALIZED	After calling SPI Initialization, the same API called again before calling SPI deinitialization.	Development	0x4A
SPI_E_DMA_ADR_ALIGN_ERROR	Buffer pointers not aligned for DMA usage	Development	0x51
SPI_E_PARAM_POINTER	API service called with NULL parameter.	Development	0x10
SPI_E_HARDWARE_ERROR	A receive error is detected when a new data frame is completely received but the previous data was not read out of the receive buffer register RB.	Production	User Configurable
SPI_E_DMA_TRANSFER	DMA transfer error has occurred.	Production	User Configurable

Table 163 Safety Error Classification and Error Codes

Type or Error	Error Code	Value
API called with wrong Channel - Spi_WriteIB - Spi_ReadIB - Spi_SetupEB - Spi_SlaveSetupEB	SPI_E_PARAM_CHANNEL	0x0A
Api called with wrong job.	SPI_E_PARAM_JOB	0x0B

Type or Error	Error Code	Value
Spi_GetJobResult		
Api called with wrong sequence. - Spi_AsyncTransmit - Spi_GetSequenceResult - Spi_SyncTransmit - Spi_Cancel - Spi_SlaveReadPrepare	SPI_E_PARAM_SEQ	0x0C
Api called with incorrect length - Spi_SetupEB - Spi_SlaveSetupEB	SPI_E_PARAM_LENGTH	0x0D
Api called with wrong HW unit. Spi_GetHWUnitStatus	SPI_E_PARAM_UNIT	0x0E
Api called with wrong configuration parameter. Spi_Init.	SPI_E_PARAM_CONFIG	0x0F
Api called with wrong Source buffer pointer, i.e Last element of Buffer is not filled with channel's SpiSafetyBufferMarker value [See 4.6.8] - Spi_WriteIB - Spi_SetupEB	SPI_E_PARAM_SRC_POINTER	0xB1
Api called with wrong Destination buffer Pointer, i.e Last element of Buffer is not filled with channel's SpiSafetyBufferMarker value [See 4.6.8] - Spi_ReadIB - Spi_SetupEB - Spi_SlaveSetupEB	SPI_E_PARAM_DEST_POINTER	0xB2
Api called with invalid mode. Spi_SetAsyncMode	SPI_E_PARAM_MODE	0xB3
Interrupt service called with invalid QSPI Module	SPI_E_PARAM_MODULE	0xB4
QSPIn CLC initialization failed Spi_Init	SPI_E_CLC_ERROR	0xB5
Interrupt Source Check failed for ISRs, - Spi_IsrDmaQspiRx - Spi_IsrDmaQspiTx - Spi_IsrQspiPt - Spi_IsrQspiUsr - Spi_IsrQspiError	SPI_E_INTR_SRC_ERROR	0xB6

7.2 Description of Production errors

Table 164 Description of Production errors.

Dem EventId Description	Error Handler	Interrupt Function	SFR Bits and Description
SPI_E_DMA_TRANSFER: Occurs on DMA transaction request lost event.	Spi_IsrDmaError	DMA_ISR_QSPIxTX DMA_ISR_QSPIxRX (x = QSPI module)	DMA.TSRz (z = Dma Channel) .B.TRL
SPI_E_DMA_TRANSFER: Occurs on various DMA move engine errors.	Spi_IsrDmaError	DMA_ISR_QSPIxTX DMA_ISR_QSPIxRX (x = QSPI module)	DMA_ERRSRx (x = 0-1 move engine) Any of following error bits: .B.DLLER .B.SLLER .B.RAMER .B.SRIER .B.SPBER .B.DER

Dem EventId Description	Error Handler	Interrupt Function	SFR Bits and Description
SPI_E_HARDWARE_ERROR: Occurs on QSPI Queue - TXFIFO overflow - TXFIFO underflow - RXFIFO overflow - RXFIFO underflow - Parity Error	Spi_IsrQspiError	QSPiX_ISR_ESRC (x = QSPI module)	.B.SER QSPiX.STATUS (x = QSPI module) .B.ERRORFLAGS

7.3 Debug Macro used for Testing

In accordance to ISO26262-6:2011(E) section 9.4.3 the tests have been performed for software to achieve confidence in the absence of unintended functionality. There are debug code written within the driver to simulate certain HW conditions, however these code fragments are encapsulated with a DEBUG macro i.e. the final binary will not have these instructions.

These macros shall not be defined by the user within their environment.

Table 165 Description of Debug Macros.

Debug Macro	Purpose	Effect, if macro enabled.
IFX_SPI_DEBUG_TX	Simulates the TXF bit (STATUS register) with the help of a RAM variable Spi_IfxDebugStatus.	If IFX_SPI_DEBUG_TX is set to STD_ON. A compiler error would occur, since there is no definition of the variable done within the code. If the variable is also defined by the user, then the driver would read the status from this variable rather than HW register
IFX_SPI_DEBUG_ERR	Simulates the ERRORFLAGS bits (STATUS register) with the help of a RAM variable Spi_IfxDebugStatus.	If IFX_SPI_DEBUG_ERR is set to STD_ON. A compiler error would occur, since there is no definition of the variable done within the code. If the variable is also defined by the user, then the driver would read the status from this variable rather than HW register
IFX_SPI_DEBUG_RST	Simulates the RESET bits (Kernel reset register) with the help of a RAM variable Spi_IfxDebugStatus.	If IFX_SPI_DEBUG_RST is set to STD_ON. A compiler error would occur, since there is no definition of the variable done within the code. If the variable is also defined by the user, then the driver would read the status from this variable rather than HW register
IFX_SPI_DEBUG_CLC	Simulates CLC bits (Clock enable/disable) with the help of RAM variables Spi_IfxDebugStatus and Spi_IfxDebugClcSlave	If IFX_SPI_DEBUG_RST is set to STD_ON. A compiler error would occur, since there is no definition of the variable done within the code.

8 Example Usage

This chapter describes how the SPI Handler Driver can be configured and how to use different API provided by it.

8.1 Configuration of the driver

To configure SPI Handler Driver the following guidelines should be followed properly.

1. MCU driver: Configure the system clock refer [\[4\]](#).
2. PORT driver: Refer [\[5\]](#) to configure the Port driver. The port driver is required to configure the following port pins.
 - MOSI,MISO and CLK Pins used by the QSPI. This is done by using the alternate functionality of these pins.
 - CS Pins. The CS pins must be configured as the general purpose IO Pins. Additionally the initial polarity of these pins must also be configured.
3. Interrupt priority and TOS initialization: The priority of Qspi and Dma interrupts should be configured in IRQ driver and appropriate priorities should be given. Qspi Rx and Tx interrupt TOS should be configured to appropriate values based on DMA Channel used.
4. SPI handler driver: Select the required API configuration and choose channel dependant parameters like EB/IB, data width, number of data to transmitted, baudrate etc.

8.1.1 Sample Configuration of the Driver

Requirement: To Read and write to the EEPROM. Configure three EB channels, three jobs and three sequences.

In the configuration tool: Under SpiModuleDescription/SpiGeneral the following values are set.

SpiDevErrorDetect -> false

SpiVersionInfoApi -> false

SpiLevelDelivered -> 1

SpiChannelBuffersAllowed -> 1

SpiInterruptableSequenceAllowed -> true

SpiHwStatusApi -> false

SpiCancelApi -> false

The configuration header file is shown in the [Code Listing 2](#)

Code Listing 2 SPI Configuration Header file

```
022:  #define SPI_DEV_ERROR_DETECT    (STD_OFF)
023:  #define SPI_VERSION_INFO_API    (STD_OFF)
024:  #define SPI_LEVEL_DELIVERED     (1U)
025:  #define SPI_CHANNEL_BUFFERS_ALLOWED  (1U)
026:  #define SPI_INTERRUPTABLE_SEQUENCE_ALLOWED  (STD_ON)
027:  #define SPI_HW_STATUS_API       (STD_OFF)
028:  #define SPI_CANCEL_API          (STD_OFF)
029:  #define SPI_MAXIMUM_SEQUENCE    (3U)
030:  #define SPI_MAXIMUM_EB_CHANNELS  (3U)
031:  #define SPI_PB_FIXEDADDR        (STD_OFF)
```

Code Listing 2 SPI Configuration Header file

```

032:  #define SPI_SUPPORT_CONCURRENT_SYNC_TRANSMIT  (STD_OFF)
033:  #define SPI_SYNCTRANSMIT_TIMEOUT_COUNTER  (4294967295U)
034:  #define SPI_HW_ERROR_DEM_REPORT  (DISABLE_DEM_REPORT)
035:  #define SPI_DMA_ERROR_DEM_REPORT  (DISABLE_DEM_REPORT)
036:  #define SPI_RUNNING_IN_USER_0_MODE_ENABLE  (STD_OFF)
037:  #define SPI_USER_MODE_INIT_API_ENABLE  (STD_OFF)
038:  #define SPI_USER_MODE_DEINIT_API_ENABLE  (STD_OFF)
039:  #define SPI_USER_MODE_RUNTIME_API_ENABLE  (STD_OFF)

```

SPI channel configuration is configured as follows

(Parameters configured – SpiChannelType, SpiDataWidth, SpiTransferStart, SpiEbMaxLength, SpiDefaultData)

Set the SpiChannelType to EB Channel, SpiDataWidth to 8U, SpiTransferStart to MSB, SpiEbMaxLength to 300, set SpiDefaultData to 0.

The channel configuration parameter is shown in [Code Listing 3](#)

Code Listing 3 SPI Channel Configuration Parameter

```

001:  const Spi_ChannelConfigType Spi_kChannelConfig0[] =
002:  {
003:  /* EB Channel: EEP_CMD_CH Configuration */
004:  {
005:      /* Default Data, SPI_DEFAULT_DATA */
006:      (uint16)0x0,
007:      /* Data Configuration */
008:      Spi_DataConfig(8U, /* Data Width */
009:                    SPI_DATA_MSB_FIRST)/* Transfer Start */
010:      /* EB Channels : SPI112: Max EB Buffers */
011:      (Spi_NumberOfDataType)300,
012:      /* Channel Based Chip Select */
013:      (uint8)0U,
014:      /* Buffer Type, SPI_CHANNEL_TYPE */
015:      (uint8)SPI_EB_BUFFER,
016:  },
017:  };

```

SPI Job configuration is configured as follows.

The Parameter configured – QSPI0 Channel0 is assigned to SpiHwUnit, set SpiShiftClockIdleLevel to clock Idle level low, SpiDataShiftEdge to trailing edge, set the SpiBaudrate to 100kbaud, set the SpiCs to port3 pin 5, set the SpiCsPolarity to Low, set the SpiTimeClk2CS to 1microsec, SpiAutoCalcBaudAndDelayParams = true, SpiTrailingTime to 0.1 microsec, SpildleTime to 0.1 microsec.

The Job configuration parameter is shown in [Code Listing 4](#)

Code Listing 4 SPI Job Configuration Parameter

```

001:  static const Spi_JobConfigType Spi_kJobConfig0[] =
002:  {
003:
004:  /* Job: EEP_JOB_CMD Configuration */
005:  {
006:      /* Job End Notification: Spi_JobEndNotification, SPI118 */

```


Code Listing 4 SPI Job Configuration Parameter

```

007:     NULL_PTR,
008:     /* User given name outside the naming convention */
009:     /* Spi_ChannelLinkPtr */
010:     EEP_JOB_CMD_ChannelLinkPtr,
011:     /* Baud Rate (10000000 Hz) Hw configuration Parameters */
012:     Spi_BaudRateParams(/*TQ*/(0x0U), /*Q*/(0x0U),
013:         /*A*/(0x0U), /*B*/(0x2U), /*C*/(0x3U)),
014:
015:     /* Time Delay Configuration */
016:     (uint32)Spi_DelayParams(/*IPRE*/0x02,/*IDLE*/ 0x02,
017:         /*LPRE*/0x03, /*LEAD*/0x02,
018:         /*TPRE*/0x03, /*TRAIL*/0x02),
019:
020:     SPI_CS_HW, /* Hw CS Pin is Selected */
021:
022:     /*CS Active level Polarity*/
023:     SPI_CS_POLARITY_LOW,
024:
025:     /* Shift Clock Configuration : Clock Idle Polarity:
    SPI_SHIFT_CLOCK_IDLE_LEVEL,
026:                                     Clock Phase: SPI_DATA_SHIFT_EDGE */
027:     Spi_ShiftClkConfig(SPI_CLK_IDLE_LOW,
028:         SPI_DATA_SHIFT_TRAIL),
029:     /* Job Priority : SPI_JOB_PRIORITY, SPI002,SPI093 */
030:     SPI_JOB_PRIORITY_1,
031:
032:     /* Spi HW Unit. bit[7:4]: Channel no, bit[3:0]: hw module no */
033:     (uint8)((uint8)SPI_QSPI_CHANNEL7 << 4U) | (SPI_QSPI0_INDEX),
034:
035:     /* Channel Based Chip Select */
036:     (uint8)0U,
037: },
038: };

```

Spi sequence configuration is configured as follows

Set SpiInterruptibleSequence to true, set SpiSeqEndNotification to EepTest_EndOfCmdSeq

The sequence configuration parameter is shown in [Code Listing 5](#)

Code Listing 5 SPI Sequence Configuration Parameter

```

001:     const Spi_SequenceConfigType Spi_kSequenceConfig0[] =
002:     {
003:     /*Sequence: EEP_SEQ_CMD, Id: 0*/
004:     {
005:         /* Spi_SeqEndNotification */
006:         EepTest_EndOfCmdSeq,
007:         /* User given name outside the naming convention */
008:         EEP_SEQ_CMD_JobLinkPtr,
009:         /* Sequences that share jobs with this sequence */

```

Code Listing 5 SPI Sequence Configuration Parameter

```

010:     NULL_PTR,
011:     1U,      /* Sequence Interruptible or Not (SPI125, SPI126) */
012:     SPI_SEQ_INT_TRUE
013:
014: },
015: };

```

The [Code Listing 6](#) depicts hardware module and Dma configuration constant structures.

Code Listing 6 Hardware Module and Dma configuration

```

001:  const Spi_HWModuleConfigType Spi_kModuleConfig0[]=
002:  {
003:      /* QSPI0 Module */
004:      {
005:          /*Clock Settings:Sleep Control Disabled*/
006:          SPI_CLK_SLEEP_DISABLE,
007:          SPI_QSPI0_MRIS_SEL,
008:          &Spi_kDmaConfig0[0],
009:      },
010:      /* QSPI1 Module */
011:      {
012:          SPI_1_NOT_CONFIGURED,
013:          SPI_1_NOT_CONFIGURED,
014:          NULL_PTR,
015:      },
016:      /* QSPI2 Module */
017:      {
018:          SPI_2_NOT_CONFIGURED,
019:          SPI_2_NOT_CONFIGURED,
020:          NULL_PTR,
021:      },
022:      /* QSPI3 Module */
023:      {
024:          SPI_3_NOT_CONFIGURED,
025:          SPI_3_NOT_CONFIGURED,
026:          NULL_PTR,
027:      },
028:  };
029:
030:  const Spi_DmaConfigType Spi_kDmaConfig0[]=
031:  {
032:      { /* QSPI0 Module Hw Dma Config */
033:          DMA_CHANNEL2, /* Tx */
034:          DMA_CHANNEL1 /* Rx */
035:      },
036:      { /* QSPI1 Module Hw Dma Config */
037:          DMA_CHANNEL4, /* Tx */
038:          DMA_CHANNEL3 /* Rx */
039:      },

```

Code Listing 6 Hardware Module and Dma configuration

```

040:    { /* QSPI2 Module Hw Dma Config */
041:        DMA_CHANNEL6, /* Tx */
042:        DMA_CHANNEL5 /* Rx */
043:    },
044:    { /* QSPI3 Module Hw Dma Config */
045:        DMA_CHANNEL8, /* Tx */
046:        DMA_CHANNEL7 /* Rx */
047:    },
048: };

```

8.1.2 SPI Configuration

The run time configurable parameters are part of the configuration data structure Spi_ConfigType.

Spi configuration constant structure is configured in Spi_PBCfg.c

This configuration structure contains the initialization data structure of the SPI. This structure contains configuration of SPI channels, SPI jobs, SPI sequences, SPI HW configuration

Code Listing 7 Run time Configurable Parameters

```

049:    const Spi_ConfigType Spi_ConfigRoot[1] =
050:    {
051:        {
052:            Spi_kChannelConfig0,
053:            Spi_kJobConfig0,
054:            Spi_kSequenceConfig0,
055:            {
056:                &Spi_kModuleConfig0[0],
057:                &Spi_kModuleConfig0[1],
058:                &Spi_kModuleConfig0[2],
059:                &Spi_kModuleConfig0[3],
060:            },
061:            (sizeof(Spi_kChannelConfig0) / sizeof(Spi_ChannelConfigType)),
062:            (sizeof(Spi_kJobConfig0) / sizeof(Spi_JobConfigType)),
063:            (sizeof(Spi_kSequenceConfig0) / sizeof(Spi_SequenceConfigType)),
064:            NULL_PTR,
065:            NULL_PTR
066:        }
067:    }
068: };

```

8.1.3 Spi Driver Initialization

The [Code Listing 8](#) depicts the steps involved in the initialization of the SPI Handler Driver.

Note: The interrupt priority and TOS initialization should be done before the spi initialization. Also the qspi priority initialization function IrqSpi_Init() should be called only once before the Spi_Init.

Code Listing 8 SPI Driver Initialization

```

069:    /* MCU Initialization. */
070:    Mcu_Init(&Mcu_ConfigRoot[0]);
071:    /* MCU Clock Initialisation Clk: 60Mhz. */

```

Code Listing 8 SPI Driver Initialization

```

072:  Mcu_InitClock(0);
073:  while(Mcu_GetPllStatus(0) == 0);
074:  Mcu_DistributePllClock(0);
075:
076:  /* Configure DMA Module Interrupt Priority */
077:  IrqDma_Init();
078:  /* Configure SPI Module Interrupt Priority */
079:  IrqSpi_Init();
080:  /* Enable Global Interrupt Flag. */
081:  EnableAllInterrupts ();
082:  /* Port Initialization */
083:  Port_Init (&(Port_ConfigRoot[0]));
084:  /* SPI Initialization */
085:  Spi_Init(&Spi_ConfigRoot[0]);

```

8.1.4 Start SPI Transmission to access EEPROM

The [Code Listing 9](#) shows an example of the steps involved in setting up and starting a sequence for EEPROM access. This also shows the API usage. The actual configuration is available in the Demo application supplied.

Code Listing 9 SPI EEPROM Access

```

086:  #define EEP_TESTDATA_LENGTH 24
087:  const Spi_DataType EepTestWriteData[EEP_TESTDATA_LENGTH] =
088:  {
089:      'D','L','-','A','U','T','O','S',
090:      'A','R','-','E','N','G','I','N',
091:      'E','E','R','I','N','G',0x00,0x00
092:  };
093:  void Spi_EepromAccessDemo(void)
094:  {
095:      Spi_SetupEB(EEP_CMD_CH,&cmdWriteEn,NULL_PTR,1);
096:      Spi_AsyncTransmit(EEP_SEQ_CMD);
097:      while(EepTestSeqFlag == 1);
098:
099:      /* read status register */
0100:      Spi_SetupEB(EEP_CMD_CH,&cmdReadSR, NULL_PTR,1);
0101:      Spi_SetupEB(EEP_DATA_CH, NULL_PTR, (Spi_DataType *)&EepTestReadByte,
0102:      1);
0103:      Spi_AsyncTransmit(EEP_SEQ_STAT);
0104:      while(EepTestSeqFlag == 1);
0105:      printf ("\nRead data: ");
0106:      for (it=0;it<EEP_TESTDATA_LENGTH;it++) {
0107:          EepTestAddress [1] = it; //lsb.
0108:          ch = EepTest_ReadByte(EepTestAddress);
0109:          put_char(ch);
0110:      }
0111:      put_char('\n');
0112:

```

Code Listing 9 SPI EEPROM Access

```
0113:   printf("----- End of EEPROM access demo using SPI -----\n");
0114: }
```

8.1.5 Spi Slave Usage

SPI Slave is used for data readback which SPI Master communicates to the external slave.

Below diagram explains the pin connections of Spi slave readback,

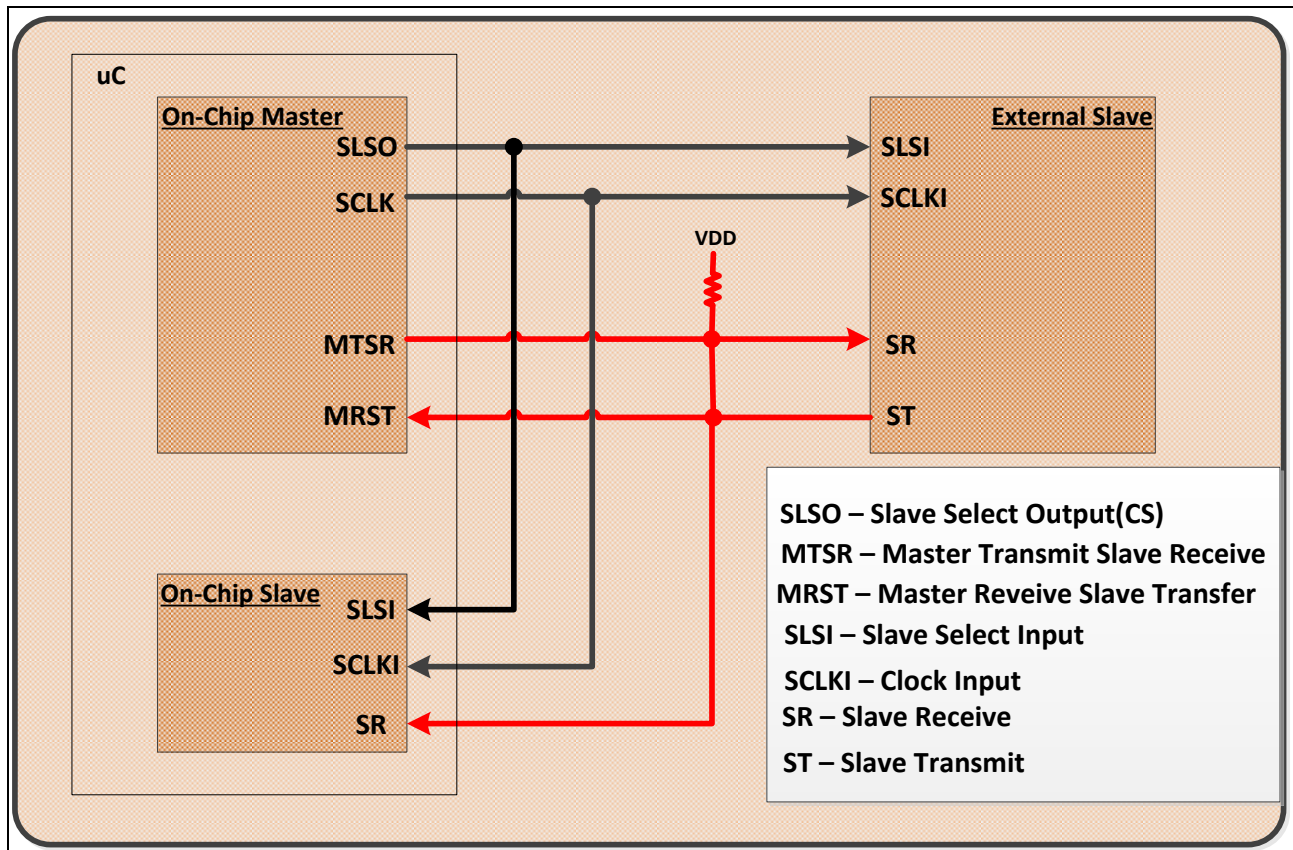


Figure 9 Spi Slave connections

Only one slave readback is present in the driver, which reads the data either on MTSR line or MRST line. Hence, with slave connections mentioned in the [Figure 9](#) should be used in half-duplex mode.

Port pin settings:

Pins of on-chip Master(MTSR, MRST), External Slave(SR, ST) and On-chipSlave(SR) are shorted and pulled up to form a wired-AND logic connection(i.e., Effective output level of the system is LOW, if any one output pin connected is LOW and effective output level is HIGH, if all the output pins in the connection are HIGH)

MTSR(Master) and ST(external slave) should be configured to OPEN DRAIN OUTPUT mode in PORT module.

Sample slave configuration:

SpiLevelDelivered -> 2

SpiSafetyMasterRef -> QSPI0

SpiSlaveSelectRef -> QSPI2

SpiSlaveFlipTransferStart -> FALSE

Define a sequence "SEQUENCE_MASTER_SLAVE", assign an asynchronous job with EB channel "CHANNEL_MASTER_SLAVE" (8-bit datawidth) on the QSPI0 hardware.

The below code listing depicts the readback of the slave:

Code Listing 10 Spi Slave readback calling procedure(Master Receive data, half-duplex)

```

001:  /* pre-Configure following buffers with Safety Buffer marker and
002:      appropriate data */
003:  /* Spi_SlaveDesEBBuff_8, Spi_MasterSrcEBBuff_8, Spi_MasterDesEBBuff_8*/
004:
005:  /* Configure slave receive buffer */
006:  Spi_SlaveSetupEBResult_1 = Spi_SlaveSetupEB(CHANNEL_MASTER_SLAVE,
    &Spi_SlaveDesEBBuff_8, 8U);
007:
008:  /* Configure master receive buffer */
009:  Spi_SetupEBResult_1 = Spi_SetupEB(CHANNEL_MASTER_SLAVE,
    &Spi_MasterSrcEBBuff_8, &Spi_MasterDesEBBuff_8, 8U);
010:
011:  /* Call Slave API */
012:  Spi_SlaveReadPrepareResult =
    Spi_SlaveReadPrepare(SEQUENCE_MASTER_SLAVE);
013:
014:  /* Call Master API */
015:  Spi_AsyncTransmitResult_1 = Spi_AsyncTransmit(SEQUENCE_MASTER_SLAVE);
016:
017:  while(SPI_IDLE != Spi_GetStatus())
018:  {
019:      ;
020:  }
021:
022:  /* data received by master in buffer "Spi_MasterDesEBBuff_8"
023:      And data received by slave in buffer "Spi_SlaveDesEBBuff_8"
024:      Should be same and verifies safe acquisition via different
025:      independent paths i.e., QSPI0 and QSPI2*/
026:

```

9 Limitations and Assumptions

9.1 Assumptions and Deviations from Software Specification

- According to Bugzilla 48763 (http://www.autosar.org/bugzilla/show_bug.cgi?id=48763), the parameter SpiHwUnitSynchronous (SWS ID: SPI129_Conf) is moved to SpiJob container from SpiChannel container.
- According to Bugzilla 48762 (http://www.autosar.org/bugzilla/show_bug.cgi?id=48762), the parameter SpiEnableCs (SWS ID: SPI212_Conf) which is of type BooleanParamDef enables or disables the Chip Select handling functions. If this parameter is enabled then parameter SpiCsSelection further details the type of chip selection.
The parameter SpiCsSelection specifies if the chip select is handled automatically by Peripheral HW engine or via GPIO by SPI driver and shall have the values,

CS_VIA_PERIPHERAL_ENGINE - SPI IP drives the chip-select lines autonomously without explicit chip select control by the driver.
CS_VIA_GPIO, - SPI driver shall drive the chip select lines explicitly via GPIO pin.
- According to Bugzilla 49716 (http://www.autosar.org/bugzilla/show_bug.cgi?id=49716), hardware error should be reported only to DEM as specified by SPI267 and SPI083 shall be removed.
- Support of debug module and exporting variables from BSW is done according to Bugzilla 48672 (http://www.autosar.org/bugzilla/show_bug.cgi?id=48672).
- **QSPI Interrupt Priority:** QSPIn_U and QSPI_PT interrupt nodes should not be enabled externally (eg., OS environment) before start of asynchronous transmission, since the QSPI interrupts are runtime enabled/disabled by the SPI driver during transmission.
Generally SPI driver will enable all required SPI interrupts, therefore it is recommended to configure OS such that the SPI interrupts are not enabled.
- Autosar requirement SPI270 is deviated. Rationale is to support the Job end notification being able to reconfigure SPI pins before starting next job. (Refer UTP: AI00239711)
- SpiMainFunctionPeriod parameter is implemented according to Bugzilla #60168
- Autosar requirement SPI046 is deviated for the API Spi_GetHWUnitStatus. Rationale, if the SPI driver is not initialized, the API returns a defined status of SPI_UNINIT. Hence, SPI_E_UNINIT DET is avoided.

9.2 Limitations

The SPI Driver imposes the following configuration limitation:

A non-interruptible sequence must have all jobs that belong to the same HW Unit. It cannot have jobs belonging to different HW Units.

Configuration Limitations:

- 1) In compliance with Autosar specification, the maximum number of sequences that can be handled by driver is 255. Similarly the maximum number of channels is 255 and jobs are 65535.
- 2) For optimal implementation, the configuration will place all the IB channels first followed by all the EB channels. In configuration the user is free to give in any order, the generation will be with this prescribed order.
- 3) Following point 3, Spi_ChannelID, Spi_JobId, Spi_SequenceId is automatically derived from the configuration tool and the user is expected to use the generated macros for the same.

- 4) The Configuration restricts the mixed 8-/16-bit IB channels in a way that only 8-bit channels with an even number of configured bytes are allowed. This to ensure that the addresses are 16-bit aligned.
- 5) The Configuration restricts the mixed 8-/32-bit IB channels in a way that only 8-bit channels with a multiple of 4 no of configured bytes are allowed. This to ensure that the addresses are 32-bit aligned.
- 6) The configuration restricts the range of parameter SpiDataWidth to 2...32, this is in accordance with the hardware specification.
- 7) For QSPI module configured as slave SpiHwDmaChannelTransmissionRef shall not be configured as slave module is used only for read back.
- 8) If SpiLevelDelivered is configured as 2 then the user shall not configure SpiHwDmaChannelTransmissionRef and SpiHwDmaChannelReceptionRef channels for QSPI module used in synchronous communication.
- 9) DMA channel selection done in SPI driver overrules the DMA channel configuration of IRQ priority number selected in IRQ module. The DMA channel number used by SPI will be used as the priority number as per HW recommendation.
- 10) Separate DMA channels shall be used for every kernel, hence, SpiHwDmaChannelTransmissionRef/SpiHwDmaChannelReceptionRef shall not be configured as the same for different QSPI devices. Tool will not generate error for this case, hence, user shall not configure same Dma channels for different QSPI hardware units..

Functional Limitations:

1. If the SpiExternalDevice uses chip select via gpio feature (i.e SpiCsSelection = CS_VIA_GPIO), then the actual Lead/Trail time (i.e SpiTimeClk2CS or SpiQSpiParamLPre, SpiQSpiParamLead) will be more than the configured value.
2. If SpiChannelBasedCS feature is enabled for a Job; in order to reduce the chipselect deactivation time between each datawidth transmission; configured idle time of the job shall be reduced.
3. The clock idle level after Spi initialization shall be LOW irrespective of value of SpiShiftClockIdleLevel. The value of SpiShiftClockIdleLevel shall be set to proper idle level at the beginning of first job transmission and shall remain at the same idle level until it is reconfigured to different idle level.
4. There may be a small delay in job transmit getting completed and the Spi_GetStatus API returning to SPI_IDLE state since the hardware takes some time to return to wait state.

Safety usage Limitation:

1. During interruptible sequence transmission/reception (with SPI read back), user shall ensure that a high priority job is rescheduled only after a job notification is received or within good timing interval (i.e. there is no change in status of queue between the call Spi_SlaveReadPrepare and Spi_SyncTransmit/Spi_AsyncTransmit. If rescheduled at arbitrary times there could be a mismatch between master and slave queue.

9.3 Hardware Features not supported

The slave of the QSPI is implemented with limited functionality to read back from the Spi Safety master.

10 Used Aurix Derivates

10.1 QSPI Modules Availability

Table 166 Aurix Derivates Information

Derivative	Information
TC299 (BGA516)	Number of QSPI Modules : 6 Available Modules: QSPI0,QSPI1,QSPI2,QSPI3,QSPI4,QSPI5
TC298 (BGA416)	Number of QSPI Modules : 4 Available Modules: QSPI1,QSPI2,QSPI3,QSPI4
TC297 (BGA292)	Number of QSPI Modules : 5 Available Modules: QSPI0,QSPI1,QSPI2,QSPI3,QSPI4
TC277 (BGA292)	Number of QSPI Modules : 4 Available Modules: QSPI0,QSPI1,QSPI2,QSPI3
TC275 (QFP176)	Number of QSPI Modules : 4 Available Modules: QSPI0,QSPI1,QSPI2,QSPI3
TC267 (QFP292)	Number of QSPI Modules : 4 Available Modules: QSPI0,QSPI1,QSPI2,QSPI3
TC265 (QFP176)	Number of QSPI Modules : 4 Available Modules: QSPI0,QSPI1,QSPI2,QSPI3
TC264/TC244 (QFP144)	Number of QSPI Modules : 4 Available Modules: QSPI0,QSPI1,QSPI2,QSPI3
TC237 (BGA292)	Number of QSPI Modules : 4 Available Modules: QSPI0,QSPI1,QSPI2,QSPI3
TC234 (QFP144)	Number of QSPI Modules : 4 Available Modules: QSPI0,QSPI1,QSPI2,QSPI3
TC233 (QFP100)	Number of QSPI Modules : 4 Available Modules: QSPI0,QSPI1,QSPI2,QSPI3
TC224/TC214 (QFP144)	Number of QSPI Modules : 4 Available Modules: QSPI0,QSPI1,QSPI2,QSPI3
TC223/TC213 (TQFP100)	Number of QSPI Modules : 4 Available Modules: QSPI0,QSPI1,QSPI2,QSPI3
TC222/TC212 (QFP80)	Number of QSPI Modules : 4 Available Modules: QSPI0,QSPI1,QSPI2,QSPI3

10.2 Port-Pins mapping

10.2.1 TC299 (516pin) Derivative Port-Pin mapping

Table 167 QSPI MRST(MISO), MTSR(MOSI), SCLK and Chip Select Pin Connections for TC299

QSPI Module	MRST(MISO) Master Input	MTSR (MOSI) Slave Input	SCLK Slave Clock	ChipSelect
QSPI0	MRST0A: P20.12 MRST0B: P22.9 MRST0C: P22.6	MTSR0: P20.14 MTSR0: P22.10 MTSR0: P22.5	SCLK0: P20.11 SCLK0: P22.7 SCLK0: P22.8	SLSO0-0: P20.8 SLSO0-1: P20.9 SLSO0-10: P22.11 SLSO0-11: P23.6 SLSO0-12: P22.4 SLSO0-13: P15.0 SLSO0-2: P20.13 SLSO0-3: P11.10 SLSO0-4: P11.11 SLSO0-5: P11.2 SLSO0-6: P20.10 SLSO0-7: P33.5 SLSO0-8: P20.6 SLSO0-9: P20.3
QSPI1	MRST1A: P10.1 MRST1B: P11.3	MTSR1: P10.3 MTSR1: P10.4 MTSR1: P11.9	SCLK1: P10.2 SCLK1: P11.6	SLSO1-0: P20.8 SLSO1-1: P20.9 SLSO1-10: P10.0 SLSO1-2: P20.13 SLSO1-3: P11.10 SLSO1-4: P11.11 SLSO1-5: P11.2 SLSO1-6: P33.10 SLSO1-7: P33.5 SLSO1-8: P10.4 SLSO1-9: P10.5
QSPI2	MRST2A: P15.4 MRST2B: P15.7 MRST2CN: P21.2 MRST2CP: P21.3 MRST2D: P34.4 MRST2E: P15.2	MTSR2: P15.5 MTSR2: P15.6 MTSR2: P34.5	SCLK2: P15.3 SCLK2: P15.8 SCLK2: P33.14	SLSO2-0: P15.2 SLSO2-1: P14.2 SLSO2-10: P33.2 SLSO2-10: P34.3 SLSO2-11: P33.15 SLSO2-11: P33.6 SLSO2-12: P32.6 SLSO2-12: P33.4 SLSO2-2: P14.6 SLSO2-3: P14.3 SLSO2-4: P14.7 SLSO2-5: P15.1 SLSO2-6: P33.13 SLSO2-7: P20.10 SLSO2-8: P20.6

QSPI Module	MRST(MISO) Master Input	MTSR (MOSI) Slave Input	SCLK Slave Clock	ChipSelect
				SLSO2-9: P20.3
QSPI3	MRST3A: P02.5 MRST3B: P10.7 MRST3C: P01.5	MTSR3: P01.6 MTSR3: P02.6 MTSR3: P10.6	SCLK3: P01.7 SCLK3: P02.7 SCLK3: P10.8	SLSO3-0: P02.4 SLSO3-1: P02.0 SLSO3-10: P01.4 SLSO3-2: P02.1 SLSO3-3: P00.5 SLSO3-3: P02.2 SLSO3-4: P00.2 SLSO3-4: P02.3 SLSO3-5: P02.12 SLSO3-5: P02.8 SLSO3-6: P00.8 SLSO3-6: P02.15 SLSO3-7: P00.9 SLSO3-7: P02.13 SLSO3-8: P10.5 SLSO3-9: P01.3
QSPI4	MRST4A: P33.13 MRST4B: P22.1 MRST4CN: P21.2 MRST4CP: P21.3 MRST4DN: P21.0 MRST4DP: P21.1	MTSR4: P33.12 MTSR4: P22.0	SCLK4: P22.3 SCLK4: P33.11	SLSO4-0: P33.10 SLSO4-1: P33.9 SLSO4-2: P33.8 SLSO4-3: P22.2 SLSO4-4: P02.12 SLSO4-4: P23.5 SLSO4-5: P02.15 SLSO4-5: P23.4 SLSO4-6: P02.13 SLSO4-6: P23.1 SLSO4-7: P02.1 SLSO4-7: P33.7
QSPI5	MRST5A: P15.10	MTSR5: P15.14	SCLK5: P15.15	SLSO5-0: P15.13 SLSO5-1: P15.12 SLSO5-2: P15.11 SLSO5-3: P15.6 SLSO5-4: P13.14 SLSO5-5: P13.9

10.2.2 TC298 (416pin) Derivative Port-Pin mapping

Table 168 QSPI MRST(MISO), MTSR(MOSI), SCLK and Chip Select Pin Connections for TC298

QSPI Module	MRST(MISO) Master Input	MTSR (MOSI) Slave Input	SCLK Slave Clock	ChipSelect
QSPI0	Not Available	Not Available	Not Available	SLSO0-11: P23.6 SLSO0-3: P11.10 SLSO0-4: P11.11 SLSO0-5: P11.2 SLSO0-7: P33.5
QSPI1	MRST1A: P10.1 MRST1B: P11.3	MTSR1: P10.3 MTSR1: P10.4 MTSR1: P11.9	SCLK1: P10.2 SCLK1: P11.6	SLSO1-10: P10.0 SLSO1-3: P11.10 SLSO1-4: P11.11 SLSO1-5: P11.2 SLSO1-6: P33.10 SLSO1-7: P33.5 SLSO1-8: P10.4 SLSO1-9: P10.5
QSPI2	MRST2A: P15.4 MRST2B: P15.7 MRST2CN: P21.2 MRST2CP: P21.3 MRST2D: P34.4 MRST2E: P15.2	MTSR2: P15.5 MTSR2: P15.6 MTSR2: P34.5	SCLK2: P15.3 SCLK2: P15.8 SCLK2: P33.14	SLSO2-0: P15.2 SLSO2-1: P14.2 SLSO2-10: P33.2 SLSO2-10: P34.3 SLSO2-11: P33.15 SLSO2-11: P33.6 SLSO2-12: P32.6 SLSO2-12: P33.4 SLSO2-2: P14.6 SLSO2-3: P14.3 SLSO2-4: P14.7 SLSO2-5: P15.1 SLSO2-6: P33.13
QSPI3	MRST3A: P02.5 MRST3B: P10.7 MRST3C: P01.5	MTSR3: P01.6 MTSR3: P02.6 MTSR3: P10.6	SCLK3: P01.7 SCLK3: P02.7 SCLK3: P10.8	SLSO3-0: P02.4 SLSO3-1: P02.0 SLSO3-10: P01.4 SLSO3-2: P02.1 SLSO3-3: P00.5 SLSO3-3: P02.2 SLSO3-4: P00.2 SLSO3-4: P02.3 SLSO3-5: P02.12 SLSO3-5: P02.8 SLSO3-6: P00.8 SLSO3-6: P02.15 SLSO3-7: P00.9 SLSO3-7: P02.13 SLSO3-8: P10.5 SLSO3-9: P01.3
QSPI4	MRST4A: P33.13	MTSR4: P33.12	SCLK4: P22.3	SLSO4-0: P33.10

CONFIDENTIAL

Used Aurix Derivates

QSPI Module	MRST(MISO) Master Input	MTSR (MOSI) Slave Input	SCLK Slave Clock	ChipSelect
	MRST4B: P22.1 MRST4CN: P21.2 MRST4CP: P21.3 MRST4DN: P21.0 MRST4DP: P21.1	MTSR4: P22.0	SCLK4: P33.11	SLSO4-1: P33.9 SLSO4-2: P33.8 SLSO4-3: P22.2 SLSO4-4: P02.12 SLSO4-4: P23.5 SLSO4-5: P02.15 SLSO4-5: P23.4 SLSO4-6: P02.13 SLSO4-6: P23.1 SLSO4-7: P02.1 SLSO4-7: P33.7
QSPI5	Not Available	Not Available	Not Available	SLSO5-3: P15.6 SLSO5-4: P13.14 SLSO5-5: P13.9

10.2.3 TC297 (292pin) Derivative Port-Pin mapping

Table 169 QSPI MRST(MISO), MTSR(MOSI), SCLK and Chip Select Pin Connections for TC297

QSPI Module	MRST(MISO) Master Input	MTSR (MOSI) Slave Input	SCLK Slave Clock	ChipSelect
QSPI0	MRST0A: P20.12 MRST0B: P22.9 MRST0C: P22.6	MTSR0: P20.14 MTSR0: P22.10 MTSR0: P22.5	SCLK0: P20.11 SCLK0: P22.7 SCLK0: P22.8	SLSO0-0: P20.8 SLSO0-1: P20.9 SLSO0-10: P22.11 SLSO0-11: P23.6 SLSO0-12: P22.4 SLSO0-13: P15.0 SLSO0-2: P20.13 SLSO0-3: P11.10 SLSO0-4: P11.11 SLSO0-5: P11.2 SLSO0-6: P20.10 SLSO0-7: P33.5 SLSO0-8: P20.6 SLSO0-9: P20.3
QSPI1	MRST1A: P10.1 MRST1B: P11.3	MTSR1: P10.3 MTSR1: P10.4 MTSR1: P11.9	SCLK1: P10.2 SCLK1: P11.6	SLSO1-0: P20.8 SLSO1-1: P20.9 SLSO1-10: P10.0 SLSO1-2: P20.13 SLSO1-3: P11.10 SLSO1-4: P11.11 SLSO1-5: P11.2 SLSO1-6: P33.10 SLSO1-7: P33.5 SLSO1-8: P10.4 SLSO1-9: P10.5
QSPI2	MRST2A: P15.4 MRST2B: P15.7 MRST2CN: P21.2 MRST2CP: P21.3 MRST2D: P34.4 MRST2E: P15.2	MTSR2: P15.5 MTSR2: P15.6 MTSR2: P34.5	SCLK2: P15.3 SCLK2: P15.8 SCLK2: P33.14	SLSO2-0: P15.2 SLSO2-1: P14.2 SLSO2-10: P33.2 SLSO2-10: P34.3 SLSO2-11: P33.15 SLSO2-11: P33.6 SLSO2-12: P32.6 SLSO2-12: P33.4 SLSO2-2: P14.6 SLSO2-3: P14.3 SLSO2-4: P14.7 SLSO2-5: P15.1 SLSO2-6: P33.13 SLSO2-7: P20.10 SLSO2-8: P20.6 SLSO2-9: P20.3
QSPI3	MRST3A: P02.5 MRST3B: P10.7	MTSR3: P01.6 MTSR3: P02.6	SCLK3: P01.7 SCLK3: P02.7	SLSO3-0: P02.4 SLSO3-1: P02.0

QSPI Module	MRST(MISO) Master Input	MTSR (MOSI) Slave Input	SCLK Slave Clock	ChipSelect
	MRST3C: P01.5	MTSR3: P10.6	SCLK3: P10.8	SLSO3-10: P01.4 SLSO3-2: P02.1 SLSO3-3: P00.5 SLSO3-3: P02.2 SLSO3-4: P00.2 SLSO3-4: P02.3 SLSO3-5: P02.8 SLSO3-6: P00.8 SLSO3-7: P00.9 SLSO3-8: P10.5 SLSO3-9: P01.3
QSPI4	MRST4A: P33.13 MRST4B: P22.1 MRST4CN: P21.2 MRST4CP: P21.3 MRST4DN: P21.0 MRST4DP: P21.1	MTSR4A: P33.12 MTSR4: P22.0	SCLK4: P22.3 SCLK4: P33.11	SLSO4-0: P33.10 SLSO4-1: P33.9 SLSO4-2: P33.8 SLSO4-3: P22.2 SLSO4-4: P23.5 SLSO4-5: P23.4 SLSO4-6: P23.1 SLSO4-7: P02.1 SLSO4-7: P33.7
QSPI5	Not Available	Not Available	Not Available	SLSO5-3: P15.6

10.2.4 TC277 (292pin) Derivative Port-Pin mapping

Table 170 QSPI MRST(MISO), MTSR(MOSI), SCLK and Chip Select Pin Connections for TC277

QSPI Module	MRST(MISO) Master Input	MTSR (MOSI) Slave Input	SCLK Slave Clock	ChipSelect
QSPI0	MRST0A: P20.12 MRST0B: P22.9 MRST0C: P22.6	MTSR0: P20.14 MTSR0: P22.10 MTSR0: P22.5	SCLK0: P20.11 SCLK0: P22.7 SCLK0: P22.8	SLSO0-0: P20.8 SLSO0-1: P20.9 SLSO0-10: P22.11 SLSO0-11: P23.6 SLSO0-12: P22.4 SLSO0-13: P15.0 SLSO0-2: P20.13 SLSO0-3: P11.10 SLSO0-4: P11.11 SLSO0-5: P11.2 SLSO0-6: P20.10 SLSO0-7: P33.5 SLSO0-8: P20.6 SLSO0-9: P20.3
QSPI1	MRST1A: P10.1 MRST1B: P11.3	MTSR1: P10.3 MTSR1: P10.4 MTSR1: P11.9	SCLK1: P10.2 SCLK1: P11.6	SLSO1-0: P20.8 SLSO1-1: P20.9 SLSO1-10: P10.0 SLSO1-2: P20.13 SLSO1-3: P11.10 SLSO1-4: P11.11 SLSO1-5: P11.2 SLSO1-6: P33.10 SLSO1-7: P33.5 SLSO1-8: P10.4 SLSO1-9: P10.5
QSPI2	MRST2A: P15.4 MRST2B: P15.7 MRST2CN: P21.2 MRST2CP: P21.3 MRST2D: P34.4 MRST2E: P15.2	MTSR2: P15.5 MTSR2: P15.6 MTSR2: P34.5	SCLK2: P15.3 SCLK2: P15.8 SCLK2: P33.14	SLSO2-0: P15.2 SLSO2-1: P14.2 SLSO2-10: P34.3 SLSO2-11: P33.15 SLSO2-12: P32.6 SLSO2-2: P14.6 SLSO2-3: P14.3 SLSO2-4: P14.7 SLSO2-5: P15.1 SLSO2-6: P33.13 SLSO2-7: P20.10 SLSO2-8: P20.6 SLSO2-9: P20.3
QSPI3	MRST3A: P02.5 MRST3B: P10.7 MRST3C: P01.5 MRST3D: P33.13 MRST3E: P22.1	MTSR3: P01.6 MTSR3: P02.6 MTSR3: P10.6 MTSR3: P22.0 MTSR3: P33.12	SCLK3: P01.7 SCLK3: P02.7 SCLK3: P10.8 SCLK3: P22.3 SCLK3: P33.11	SLSO3-0: P02.4 SLSO3-1: P02.0 SLSO3-1: P33.9 SLSO3-10: P01.4 SLSO3-11: P33.10

QSPI Module	MRST(MISO) Master Input	MTSR (MOSI) Slave Input	SCLK Slave Clock	ChipSelect
	MRST3FN: P21.2 MRST3FP: P21.3			SLSO3-12: P22.2 SLSO3-13: P23.1 SLSO3-2: P02.1 SLSO3-2: P33.8 SLSO3-3: P02.2 SLSO3-4: P02.3 SLSO3-4: P23.5 SLSO3-5: P02.8 SLSO3-5: P23.4 SLSO3-6: P00.8 SLSO3-7: P00.9 SLSO3-7: P33.7 SLSO3-8: P10.5 SLSO3-9: P01.3

10.2.5 TC275 (176pin) Derivative Port-Pin mapping

Table 171 QSPI MRST(MISO), MTSR(MOSI), SCLK and Chip Select Pin Connections for TC275

QSPI Module	MRST(MISO) Master Input	MTSR (MOSI) Slave Input	SCLK Slave Clock	ChipSelect
QSPI0	MRST0A: P20.12	MTSR0: P20.14	SCLK0: P20.11	SLSO0-0: P20.8 SLSO0-1: P20.9 SLSO0-13: P15.0 SLSO0-2: P20.13 SLSO0-3: P11.10 SLSO0-4: P11.11 SLSO0-5: P11.2 SLSO0-6: P20.10 SLSO0-7: P33.5 SLSO0-8: P20.6 SLSO0-9: P20.3
QSPI1	MRST1A: P10.1 MRST1B: P11.3	MTSR1: P10.3 MTSR1: P10.4 MTSR1: P11.9	SCLK1: P10.2 SCLK1: P11.6	SLSO1-0: P20.8 SLSO1-1: P20.9 SLSO1-10: P10.0 SLSO1-2: P20.13 SLSO1-3: P11.10 SLSO1-4: P11.11 SLSO1-5: P11.2 SLSO1-6: P33.10 SLSO1-7: P33.5 SLSO1-8: P10.4 SLSO1-9: P10.5
QSPI2	MRST2A: P15.4 MRST2B: P15.7 MRST2CN: P21.2 MRST2CP: P21.3 MRST2E: P15.2	MTSR2: P15.5 MTSR2: P15.6	SCLK2: P15.3 SCLK2: P15.8	SLSO2-0: P15.2 SLSO2-1: P14.2 SLSO2-2: P14.6 SLSO2-3: P14.3 SLSO2-4: P14.7 SLSO2-5: P15.1 SLSO2-6: P33.13 SLSO2-7: P20.10 SLSO2-8: P20.6 SLSO2-9: P20.3
QSPI3	MRST3A: P02.5 MRST3B: P10.7 MRST3D: P33.13 MRST3E: P22.1 MRST3FN: P21.2 MRST3FP: P21.3	MTSR3: P02.6 MTSR3: P10.6 MTSR3: P22.0 MTSR3: P33.12	SCLK3: P02.7 SCLK3: P10.8 SCLK3: P22.3 SCLK3: P33.11	SLSO3-0: P02.4 SLSO3-1: P02.0 SLSO3-1: P33.9 SLSO3-11: P33.10 SLSO3-12: P22.2 SLSO3-13: P23.1 SLSO3-2: P02.1 SLSO3-2: P33.8 SLSO3-3: P02.2 SLSO3-4: P02.3 SLSO3-4: P23.5

CONFIDENTIAL

Used Aurix Derivates

QSPI Module	MRST(MISO) Master Input	MTSR (MOSI) Slave Input	SCLK Slave Clock	ChipSelect
				SLSO3-5: P02.8 SLSO3-5: P23.4 SLSO3-6: P00.8 SLSO3-7: P00.9 SLSO3-7: P33.7 SLSO3-8: P10.5

10.2.6 TC267 (292pin) Derivative Port-Pin mapping

Table 172 QSPI MRST(MISO), MTSR(MOSI), SCLK and Chip Select Pin Connections for TC267

QSPI Module	MRST(MISO) Master Input	MTSR (MOSI) Slave Input	SCLK Slave Clock	ChipSelect
QSPI0	MRST0A: P20.12	MTSR0: P20.14	SCLK0: P20.11	SLSO0-0: P20.8 SLSO0-1: P20.9 SLSO0-13: P15.0 SLSO0-2: P20.13 SLSO0-3: P11.10 SLSO0-4: P11.11 SLSO0-5: P11.2 SLSO0-6: P20.10 SLSO0-7: P33.5 SLSO0-8: P20.6 SLSO0-9: P20.3
QSPI1	MRST1A: P10.1 MRST1B: P11.3	MTSR1: P10.3 MTSR1: P10.4 MTSR1: P11.9	SCLK1: P10.2 SCLK1: P11.6	SLSO1-0: P20.8 SLSO1-1: P20.9 SLSO1-10: P10.0 SLSO1-2: P20.13 SLSO1-3: P11.10 SLSO1-4: P11.11 SLSO1-5: P11.2 SLSO1-6: P33.10 SLSO1-7: P33.5 SLSO1-8: P10.4 SLSO1-9: P10.5
QSPI2	MRST2A: P15.4 MRST2B: P15.7 MRST2CN: P21.2 MRST2CP: P21.3 MRST2E: P15.2	MTSR2: P15.5 MTSR2: P15.6	SCLK2: P15.3 SCLK2: P15.8	SLSO2-0: P15.2 SLSO2-1: P14.2 SLSO2-2: P14.6 SLSO2-3: P14.3 SLSO2-4: P14.7 SLSO2-5: P15.1 SLSO2-6: P33.13 SLSO2-7: P20.10 SLSO2-8: P20.6 SLSO2-9: P20.3
QSPI3	MRST3A: P02.5 MRST3B: P10.7 MRST3D: P33.13 MRST3E: P22.1 MRST3FN: P21.2 MRST3FP: P21.3	MTSR3: P02.6 MTSR3: P10.6 MTSR3: P22.0 MTSR3: P33.12	SCLK3: P02.7 SCLK3: P10.8 SCLK3: P22.3 SCLK3: P33.11	SLSO3-0: P02.4 SLSO3-1: P02.0 SLSO3-1: P33.9 SLSO3-11: P33.10 SLSO3-12: P22.2 SLSO3-13: P23.1 SLSO3-2: P02.1 SLSO3-2: P33.8 SLSO3-3: P02.2 SLSO3-4: P02.3 SLSO3-4: P23.5

CONFIDENTIAL

Used Aurix Derivates

QSPI Module	MRST(MISO) Master Input	MTSR (MOSI) Slave Input	SCLK Slave Clock	ChipSelect
				SLSO3-5: P02.8 SLSO3-5: P23.4 SLSO3-6: P00.8 SLSO3-7: P00.9 SLSO3-7: P33.7 SLSO3-8: P10.5

10.2.7 TC265 (176pin) Derivative Port-Pin mapping

Table 173 QSPI MRST(MISO), MTSR(MOSI), SCLK and Chip Select Pin Connections for TC265

QSPI Module	MRST(MISO) Master Input	MTSR (MOSI) Slave Input	SCLK Slave Clock	ChipSelect
QSPI0	MRST0A: P20.12	MTSR0: P20.14	SCLK0: P20.11	SLSO0-0: P20.8 SLSO0-1: P20.9 SLSO0-13: P15.0 SLSO0-2: P20.13 SLSO0-3: P11.10 SLSO0-4: P11.11 SLSO0-5: P11.2 SLSO0-6: P20.10 SLSO0-7: P33.5 SLSO0-8: P20.6 SLSO0-9: P20.3
QSPI1	MRST1A: P10.1 MRST1B: P11.3	MTSR1: P10.3 MTSR1: P10.4 MTSR1: P11.9	SCLK1: P10.2 SCLK1: P11.6	SLSO1-0: P20.8 SLSO1-1: P20.9 SLSO1-10: P10.0 SLSO1-2: P20.13 SLSO1-3: P11.10 SLSO1-4: P11.11 SLSO1-5: P11.2 SLSO1-6: P33.10 SLSO1-7: P33.5 SLSO1-8: P10.4 SLSO1-9: P10.5
QSPI2	MRST2A: P15.4 MRST2B: P15.7 MRST2CN: P21.2 MRST2CP: P21.3 MRST2E: P15.2	MTSR2: P15.5 MTSR2: P15.6	SCLK2: P15.3 SCLK2: P15.8	SLSO2-0: P15.2 SLSO2-1: P14.2 SLSO2-2: P14.6 SLSO2-3: P14.3 SLSO2-4: P14.7 SLSO2-5: P15.1 SLSO2-6: P33.13 SLSO2-7: P20.10 SLSO2-8: P20.6 SLSO2-9: P20.3
QSPI3	MRST3A: P02.5 MRST3B: P10.7 MRST3D: P33.13 MRST3E: P22.1 MRST3FN: P21.2 MRST3FP: P21.3	MTSR3: P02.6 MTSR3: P10.6 MTSR3: P22.0 MTSR3: P33.12	SCLK3: P02.7 SCLK3: P10.8 SCLK3: P22.3 SCLK3: P33.11	SLSO3-0: P02.4 SLSO3-1: P02.0 SLSO3-1: P33.9 SLSO3-11: P33.10 SLSO3-12: P22.2 SLSO3-13: P23.1 SLSO3-2: P02.1 SLSO3-2: P33.8 SLSO3-3: P02.2 SLSO3-4: P02.3 SLSO3-4: P23.5

CONFIDENTIAL

Used Aurix Derivates

QSPI Module	MRST(MISO) Master Input	MTSR (MOSI) Slave Input	SCLK Slave Clock	ChipSelect
				SLSO3-5: P02.8 SLSO3-5: P23.4 SLSO3-6: P00.8 SLSO3-7: P00.9 SLSO3-7: P33.7 SLSO3-8: P10.5

10.2.8 TC264/TC244 (144pin) Derivative Port-Pin mapping

Table 174 QSPI MRST(MISO), MTSR(MOSI), SCLK and Chip Select Pin Connections for TC264/TC244

QSPI Module	MRST(MISO) Master Input	MTSR (MOSI) Slave Input	SCLK Slave Clock	ChipSelect
QSPI0	MRST0A: P20.12	MTSR0: P20.14	SCLK0: P20.11	SLSO0-0: P20.8 SLSO0-1: P20.9 SLSO0-13: P15.0 SLSO0-2: P20.13 SLSO0-3: P11.10 SLSO0-4: P11.11 SLSO0-5: P11.2 SLSO0-6: P20.10 SLSO0-7: P33.5 SLSO0-8: P20.6 SLSO0-9: P20.3
QSPI1	MRST1A: P10.1 MRST1B: P11.3	MTSR1: P10.3 MTSR1: P11.9	SCLK1: P10.2 SCLK1: P11.6	SLSO1-0: P20.8 SLSO1-1: P20.9 SLSO1-2: P20.13 SLSO1-3: P11.10 SLSO1-4: P11.11 SLSO1-5: P11.2 SLSO1-6: P33.10 SLSO1-7: P33.5 SLSO1-9: P10.5
QSPI2	MRST2A: P15.4 MRST2B: P15.7 MRST2CN: P21.2 MRST2CP: P21.3 MRST2E: P15.2	MTSR2: P15.5 MTSR2: P15.6	SCLK2: P15.3 SCLK2: P15.8	SLSO2-0: P15.2 SLSO2-1: P14.2 SLSO2-2: P14.6 SLSO2-3: P14.3 SLSO2-5: P15.1 SLSO2-6: P33.13 SLSO2-7: P20.10 SLSO2-8: P20.6 SLSO2-9: P20.3
QSPI3	MRST3A: P02.5 MRST3D: P33.13 MRST3E: P22.1 MRST3FN: P21.2 MRST3FP: P21.3	MTSR3: P02.6 MTSR3: P10.6 MTSR3: P22.0 MTSR3: P33.12	SCLK3: P02.7 SCLK3: P22.3 SCLK3: P33.11	SLSO3-0: P02.4 SLSO3-1: P02.0 SLSO3-1: P33.9 SLSO3-11: P33.10 SLSO3-12: P22.2 SLSO3-13: P23.1 SLSO3-2: P02.1 SLSO3-2: P33.8 SLSO3-3: P02.2 SLSO3-4: P02.3 SLSO3-5: P02.8 SLSO3-6: P00.8 SLSO3-7: P00.9

CONFIDENTIAL

Used Aurix Derivates

QSPI Module	MRST(MISO) Master Input	MTSR (MOSI) Slave Input	SCLK Slave Clock	ChipSelect
				SLSO3-7: P33.7 SLSO3-8: P10.5

10.2.9 TC237 (292pin) Derivative Port-Pin mapping

Table 175 QSPI MRST(MISO), MTSR(MOSI), SCLK and Chip Select Pin Connections for TC237

QSPI Module	MRST(MISO) Master Input	MTSR (MOSI) Slave Input	SCLK Slave Clock	ChipSelect
QSPI0	MRST0A: P20.12 MRST0D: P14.6	MTSR0: P20.14	SCLK0: P20.11	SLSO0-0: P20.08 SLSO0-1: P20.09 SLSO0-12: P22.04 SLSO0-13: P15.00 SLSO0-2: P20.13 SLSO0-3: P11.10 SLSO0-4: P11.11 SLSO0-5: P11.02 SLSO0-6: P20.10 SLSO0-7: P33.05 SLSO0-8: P20.06 SLSO0-9: P20.03
QSPI1	MRST1A: P10.1 MRST1B: P11.3	MTSR1: P10.3 MTSR1: P11.9 MTSR1: P11.8	SCLK1: P10.2 SCLK1: P11.6	SLSO1-0: P20.08 SLSO1-1: P20.09 SLSO1-10: P11.08 SLSO1-2: P20.13 SLSO1-3: P11.10 SLSO1-4: P11.11 SLSO1-5: P11.02 SLSO1-6: P33.10 SLSO1-7: P33.05 SLSO1-9: P10.05
QSPI2	MRST2A: P15.4 MRST2B: P15.7 MRST2E: P15.2	MTSR2: P15.5 MTSR2: P15.6	SCLK2: P15.3 SCLK2: P15.8	SLSO2-0: P15.02 SLSO2-1: P14.02 SLSO2-10: P34.03 SLSO2-2: P14.06 SLSO2-3: P14.03 SLSO2-4: P14.07 SLSO2-5: P15.01 SLSO2-7: P20.10 SLSO2-8: P20.06 SLSO2-9: P20.03
QSPI3	MRST3A: P02.5 MRST3E: P22.1	MTSR3: P02.6 MTSR3: P10.6 MTSR3: P22.0 MTSR3: P33.12	SCLK3: P02.7 SCLK3: P22.3 SCLK3: P33.11	SLSO3-0: P02.04 SLSO3-1: P02.00 SLSO3-1: P33.09 SLSO3-11: P33.10 SLSO3-12: P22.02 SLSO3-13: P23.01 SLSO3-2: P02.01 SLSO3-2: P33.08 SLSO3-3: P02.02 SLSO3-4: P02.03 SLSO3-5: P02.08

CONFIDENTIAL

Used Aurix Derivates

QSPI Module	MRST(MISO) Master Input	MTSR (MOSI) Slave Input	SCLK Slave Clock	ChipSelect
				SLSO3-6: P00.08 SLSO3-7: P00.09 SLSO3-7: P33.07 SLSO3-8: P10.05

10.2.10 TC234 (144pin) Derivative Port-Pin mapping

Table 176 QSPI MRST(MISO), MTSR(MOSI), SCLK and Chip Select Pin Connections for TC234

QSPI Module	MRST(MISO) Master Input	MTSR (MOSI) Slave Input	SCLK Slave Clock	ChipSelect
QSPI0	MRST0A: P20.12 MRST0D: P14.06	MTSR0: P20.14	SCLK0: P20.11	SLSO0-0:P20.08 SLSO0-1:P20.09 SLSO0-12:P22.04 SLSO0-13:P15.00 SLSO0-2:P20.13 SLSO0-3:P11.10 SLSO0-4:P11.11 SLSO0-5:P11.02 SLSO0-6:P20.10 SLSO0-7:P33.05 SLSO0-8:P20.06 SLSO0-9:P20.03
QSPI1	MRST1A: P10.1 MRST1B: P11.3	MTSR1: P10.3 MTSR1: P11.8 MTSR1: P11.9	SCLK1: P10.2 SCLK1: P11.6	SLSO1-0:P20.08 SLSO1-1:P20.09 SLSO1-10:P11.08 SLSO1-2:P20.13 SLSO1-3:P11.10 SLSO1-4:P11.11 SLSO1-5:P11.02 SLSO1-6:P33.10 SLSO1-7:P33.05 SLSO1-9:P10.05
QSPI2	MRST2A: P15.4 MRST2B: P15.7 MRST2E: P15.2	MTSR2: P15.5 MTSR2: P15.6	SCLK2: P15.3 SCLK2: P15.8	SLSO2-0:P15.02 SLSO2-1:P14.02 SLSO2-10:P34.03 SLSO2-2:P14.06 SLSO2-3:P14.03 SLSO2-4:P14.07 SLSO2-5:P15.01 SLSO2-7:P20.10 SLSO2-8:P20.06 SLSO2-9:P20.03
QSPI3	MRST3A: P02.5 MRST3B: P10.6 MRST3E: P22.1	MTSR3: P02.6 MTSR3: P10.6 MTSR3: P22.0 MTSR3: P33.12	SCLK3: P02.7 SCLK3: P22.3 SCLK3: P33.11	SLSO3-0:P02.04 SLSO3-1:P02.00 SLSO3-1:P33.09 SLSO3-11:P33.10 SLSO3-12:P22.02 SLSO3-13:P23.01 SLSO3-2:P02.01 SLSO3-2:P33.08 SLSO3-3:P02.02 SLSO3-4:P02.03 SLSO3-5:P02.08

CONFIDENTIAL

Used Aurix Derivates

QSPI Module	MRST(MISO) Master Input	MTSR (MOSI) Slave Input	SCLK Slave Clock	ChipSelect
				SLSO3-6:P00.08 SLSO3-7:P00.09 SLSO3-7:P33.07 SLSO3-8:P10.05

10.2.11 TC233 (100pin) Derivative Port-Pin mapping

Table 177 QSPI MRST(MISO), MTSR(MOSI), SCLK and Chip Select Pin Connections for TC233

QSPI Module	MRST(MISO) Master Input	MTSR (MOSI) Slave Input	SCLK Slave Clock	ChipSelect
QSPI0	MRST0A: P20.12 MRST0D: P14.06	MTSR0: P20.14	SCLK0: P20.11	SLSO0-0:P20.08 SLSO0-1:P20.09 SLSO0-13:P15.00 SLSO0-2:P20.13 SLSO0-3:P11.10 SLSO0-4:P11.11 SLSO0-5:P11.02 SLSO0-6:P20.10 SLSO0-7:P33.05
QSPI1	MRST1B: P11.3	MTSR1: P11.8 MTSR1: P11.9	SCLK1: P10.2 SCLK1: P11.6	SLSO1-0:P20.08 SLSO1-1:P20.09 SLSO1-10:P11.08 SLSO1-2:P20.13 SLSO1-3:P11.10 SLSO1-4:P11.11 SLSO1-5:P11.02 SLSO1-6:P33.10 SLSO1-7:P33.05 SLSO1-9:P10.05
QSPI2	MRST2E: P15.2	MTSR2: P15.5	SCLK2: P15.3	SLSO2-0:P15.02 SLSO2-2:P14.06 SLSO2-3:P14.03 SLSO2-5:P15.01 SLSO2-7:P20.10
QSPI3	MRST3A: P02.5	MTSR3: P01.6 MTSR3: P02.6 MTSR3: P10.6	SCLK3: P02.7	SLSO3-0:P02.04 SLSO3-1:P02.00 SLSO3-1:P33.09 SLSO3-11:P33.10 SLSO3-13:P23.01 SLSO3-2:P02.01 SLSO3-2:P33.08 SLSO3-3:P02.02 SLSO3-4:P02.03 SLSO3-5:P02.08 SLSO3-7:P33.07 SLSO3-8:P10.05

10.2.12 TC224/TC214 (144pin) Derivative Port-Pin mapping

Table 178 QSPI MRST(MISO), MTSR(MOSI), SCLK and Chip Select Pin Connections for TC224/TC214

QSPI Module	MRST(MISO) Master Input	MTSR (MOSI) Slave Input	SCLK Slave Clock	ChipSelect
QSPI0	MRST0A:P20.12 MRST0D:P14.06	MTSR0A:P20.14	SCLK0A:P20.11	SLSO0-0:P20.08 SLSO0-1:P20.09 SLSO0-2:P20.13 SLSO0-3:P11.10 SLSO0-4:P11.11 SLSO0-6:P20.10 SLSO0-7:P33.05 SLSO0-8:P20.06 SLSO0-9:P20.03 SLSO0-12:P22.04 SLSO0-13:P15.00
QSPI1	MRST1A:P10.01 MRST1B:P11.03	MTSR1A:P10.03 MTSR1B:P11.09 MTSR1C:P11.08	SCLK1A:P10.02 SCLK1B:P11.06	SLSO1-0:P20.08 SLSO1-1:P20.09 SLSO1-2:P20.13 SLSO1-3:P11.10 SLSO1-4:P11.11 SLSO1-6:P33.10 SLSO1-7:P33.05 SLSO1-9:P10.05 SLSO1-10:P11.08
QSPI2	MRST2A:P15.04 MRST2B:P15.07 MRST2E:P15.02	MTSR2A:P15.05 MTSR2B:P15.06	SCLK2A:P15.03 SCLK2B:P15.08	SLSO2-0:P15.02 SLSO2-1:P14.02 SLSO2-2:P14.06 SLSO2-3:P14.03 SLSO2-4:P14.07 SLSO2-5:P15.01 SLSO2-7:P20.10 SLSO2-8:P20.06 SLSO2-9:P20.03 SLSO2-10:P34.03
QSPI3	MRST3A:P02.05 MRST3E:P22.01	MTSR3A:P02.06 MTSR3B:P10.06 MTSR3D:P33.12 MTSR3E:P22.00	SCLK3A:P02.07 SCLK3D:P33.11 SCLK3E:P22.03	SLSO3-0:P02.04 SLSO3-1:P02.00 SLSO3-1:P33.09 SLSO3-2:P02.01 SLSO3-2:P33.08 SLSO3-3:P02.02 SLSO3-4:P02.03 SLSO3-5:P02.08 SLSO3-6:P00.08 SLSO3-7:P00.09 SLSO3-7:P33.07 SLSO3-8:P10.05

CONFIDENTIAL

Used Aurix Derivates

QSPI Module	MRST(MISO) Master Input	MTSR (MOSI) Slave Input	SCLK Slave Clock	ChipSelect
				SLSO3-11:P33.10 SLSO3-12:P22.02 SLSO3-13:P23.01

10.2.13 TC223/TC213 (100pin) Derivative Port-Pin mapping

Table 179 QSPI MRST(MISO), MTSR(MOSI), SCLK and Chip Select Pin Connections for TC223/TC213

QSPI Module	MRST(MISO) Master Input	MTSR (MOSI) Slave Input	SCLK Slave Clock	ChipSelect
QSPI0	MRST0A:P20.12 MRST0D:P14.06	MTSR0A:P20.14	SCLK0A:P20.11	SLSO0-0:P20.08 SLSO0-1:P20.09 SLSO0-2:P20.13 SLSO0-3:P11.10 SLSO0-4:P11.11 SLSO0-5:P11.02 SLSO0-6:P20.10 SLSO0-7:P33.05 SLSO0-13:P15.00
QSPI1	MRST1B:P11.03	MTSR1C:P11.08 MTSR1B:P11.09	SCLK1B:P11.06	SLSO1-0:P20.08 SLSO1-1:P20.09 SLSO1-2:P20.13 SLSO1-3:P11.10 SLSO1-4:P11.11 SLSO1-5:P11.02 SLSO1-6:P33.10 SLSO1-7:P33.05 SLSO1-9:P10.05 SLSO1-10:P11.08
QSPI2	MRST2E:P15.02	MTSR2A:P15.05 MTSR3A:P02.06	SCLK2A:P15.03	SLSO2-0:P15.02 SLSO2-2:P14.06 SLSO2-3:P14.03 SLSO2-5:P15.01 SLSO2-7:P20.10
QSPI3	MRST3A:P02.05	MTSR3B:P10.06	SCLK3A:P02.07	SLSO3-0:P02.04 SLSO3-1:P02.00 SLSO3-1:P33.09 SLSO3-2:P02.01 SLSO3-2:P33.08 SLSO3-3:P02.02 SLSO3-4:P02.03 SLSO3-5:P02.08 SLSO3-7:P33.07 SLSO3-8:P10.05 SLSO3-11:P33.10 SLSO3-13:P23.01

10.2.14 TC222/TC212 (80pin) Derivative Port-Pin mapping

Table 180 QSPI MRST(MISO), MTSR(MOSI), SCLK and Chip Select Pin Connections for TC222/TC212

QSPI Module	MRST(MISO) Master Input	MTSR (MOSI) Slave Input	SCLK Slave Clock	ChipSelect
QSPI0	MRST0A:P20.12 MRST0D:P14.06	MTSR0A:P20.14	SCLK0A:P20.11	SLSO0-0:P20.08 SLSO0-1:P20.09 SLSO0-2:P20.13 SLSO0-3:P11.10 SLSO0-4:P11.11 SLSO0-5:P11.02 SLSO0-7:P33.05 SLSO0-13:P15.00
QSPI1	MRST1B:P11.03	MTSR1B:P11.09	SCLK1B:P11.06	SLSO1-0:P20.08 SLSO1-1:P20.09 SLSO1-2:P20.13 SLSO1-3:P11.10 SLSO1-4:P11.11 SLSO1-5:P11.02 SLSO1-6:P33.10 SLSO1-7:P33.05 SLSO1-9:P10.05
QSPI2	MRST2E:P15.02	MTSR2A:P15.05	SCLK2A:P15.03	SLSO2-0:P15.02 SLSO2-2:P14.06 SLSO2-3:P14.03 SLSO2-5:P15.01
QSPI3	MRST3A:P02.05	MTSR3A:P02.06 MTSR3B:P10.06	SCLK3A:P02.07	SLSO3-0:P02.04 SLSO3-1:P02.00 SLSO3-1:P33.09 SLSO3-2:P02.01 SLSO3-2:P33.08 SLSO3-3:P02.02 SLSO3-4:P02.03 SLSO3-5:P02.08 SLSO3-7:P33.07 SLSO3-8:P10.05 SLSO3-11:P33.10 SLSO3-13:P23.01

www.infineon.com